# UNIVERSITE DE FIANARANTSOA

# ECOLE NATIONALE D'INFORMATIQUE

MEMOIRE DE FIN D'ETUDE POUR L'OBTENTION DU DIPLOME DE LICENCE PROFESSIONNELLE

- Mention : Informatique
- Parcours : Informatique Générale

Intitulé :

# IMPLEMENTATION D'UN PIPELINE CI/CD SECURISE ET SCALABLE POUR LE DEPLOIEMENT AUTOMATISE D'UNE APPLICATION WEB


**Présenté le :** 09 Janvier 2025

**Par :** Monsieur RANAIVO Hary Manana Zo Mahefa


**Membres du jury :**

- Président : Monsieur RAZAFINDRANDRIANTSIMANIRY Marie Dieu Donnée Michel , Maître de Conférences
- Examinateur : Monsieur RATOVONDRAHONA Alain Josué, Docteur
- Rapporteurs :
  - Madame RAKOTOASIMBAHOAKA Antsa Cyprienna, Maître de Conférences
  - Monsieur RANDRIANATOANDRO Miandrisoa, Master en économie et gestion

Année universitaire : 2024-2025

# CURRICULUM VITAE

0342075279
Ampasambazaha
zomahefaranaivo@gmail.com
18 Decembre 2005

**RANAIVO**
**Hary Manana Zo Mahefa**



## Formation et diplôme

- 2024-2025 : Troisième année de licence professionnelle à l'Ecole Nationale d'Informatique Fianarantsoa
- 2023-2024 : Deuxième année de licence professionnelle à l'Ecole Nationale d'Informatique Fianarantsoa
- 2022-2023 : Première année de licence professionnelle à l'Ecole Nationale d'Informatique Fianarantsoa
- 2021-2022 : Obtention du Bacc à LPJC Ambohimahasoa

## Stages et expériences professionnelles

- 2024-2025:
  - Septembre 2025:Stagiaire au JIRAMA
  - Thème: Mise en place d'un pipeline CI/CD sécurisé et scalable pour une application web
- 2023-2024:
  - Septembre 2024: Stagiaire au SPRAY INFO
  - Thème: Mise en place d'un serveur Moodle

## Compétences en informatique

- Système d'exploitation:Windows,Linux
- Technologie de virtualisation: Virtualbox, Vmware
- SGBD: Mysql, PostgresSQL
- Outil de supervision: Prometheus & grafana,ELK
- Conteneurisation: Docker

- Orchestration: K8s,Docker compose
- Routeur: Pfsens, Mikrotik
- Langage de programmation : Javascript,python,Java,Php
- Technologie web: HTML,CSS,Javascript
- Librairies : React.js,  Bootstrap
- Framework : Laravel, FastAPI, Spring Boot, Express.js
- outils CI/CD : Gitlab CI/CD,Github actions

## Langues

- *FRANÇAIS*

| PARLER | ECRIRE | COMPRENDRE | LIRE |
|---|---|---|---|
| Bien | Bien | Bien | Bien |

- *ANGLAIS*

| PARLER | ECRIRE | COMPRENDRE | LIRE |
|---|---|---|---|
| Assez-bien | Assez-bien | Assez-bien | Assez-bien |

## Centre d’intérêt et divers

**- Passion** : Lecture

**- Passe temps**:Jouer au Basket-ball,des jeux vidéos et regarder la télé

# REMERCIEMENTS

Tout d'abord, je tiens à remercier Dieu Tout Puissant de m'avoir donné la force de pouvoir réaliser ce mémoire. Ainsi, je tiens aussi à remercier :

- Monsieur HAJALALAINA Aimé Richard, Professeur, Président de l'Université de Fianarantsoa, qui a bien organisé l'année universitaire ;
- Monsieur MAHATODY Thomas, Professeur, Directeur de l'Ecole Nationale d'Informatique de Fianarantsoa, qui m'a donné l'opportunité de terminer mon étude en Troisième année de formation professionnelle ;
- Monsieur RANDRIANATOANDRO Miandrisoa, chef JIRAMA Ambohimahasoa, qui est mon encadreur Professionnel et qui m'a permis d'effectuer ce stage au sein de son département et pour son précieux conseil et son soutien tout au long de la période de stage;
- Monsieur RALAIVAO Jean Christian, Maître de Conférences, Responsable de la Mention Informatique de nous avoir accepté au sein de sa mention ;
- Monsieur RAZAFINDRAMONJA Clément Aubert, Assistant d'Enseignement Supérieur et de Recherche, Responsable du parcours Informatique Générale de nous avoir acceptés dans son parcours ;
- Madame RAKOTOASIMBAHOAKA Antsa Cyprienna, Maître de conférences mon encadreur pédagogique pour sa disponibilité sans oublier ses conseils qui m'a permis de réaliser ce travail dans les meilleures conditions;
- Les membres du jury,le président du jury RAZAFINDRANDRIANTSIMANIRY Marie Dieu Donnée Michel, mon examinateur RATOVONDRAHONA Alain Josué et mon encadreur RAKOTOASIMBAHOAKA Antsa Cyprienna pour l'intérêt qu'ils m'ont porté en acceptant d'examiner mon travail et de l'enrichir par leur proposition ;
- Tous les enseignants au sein de l'école Nationale d'Informatique, pour toutes les connaissances qu'ils nous ont transmises ;
- Finalement, je tiens à remercier les membres de ma famille de m'avoir soutenu que ce soit moralement, matériellement et financièrement tout au long de ce stage.

# SOMMAIRE

CURRICULUM VITAE.....I
REMERCIEMENTS.....III
SOMMAIRE.....IV
LISTE DES FIGURES.....VI
LISTE DES TABLEAUX.....VIII
LISTE DES ABREVIATIONS.....IX
INTRODUCTION GÉNÉRALE.....1
PARTIE I: PRESENTATION GENERALE.....2
chapitre 1: Présentation de l'Ecole Nationale d'Informatique.....3
1.1. Information d'ordre générale.....3
1.2. Missions et historiques.....3
1.3. Organigramme institutionnel.....5
1.4. Domaine de spécialisation.....6
1.5. Architecture des formations pédagogiques.....6
1.6. Relation de l'ENI avec les organismes externes.....7
1.7. Débouchés professionnels et diplômés.....8
1.8. Ressources humaines.....10
chapitre 2: Présentation de l'entreprise JIRAMA.....11
2.1. Présentation générale de la Société JIRAMA Madagascar.....11
2.2. Historique.....11
2.3. Indentité juridique.....12
2.4. Activité de la société.....13
2.5. Organigramme de la société.....15
chapitre 3: Description du projet.....16
3.1. Formulation.....16
3.2. Objectifs et besoins de l'entreprise.....16
3.3. Moyens nécessaires à la réalisation du projet.....16
3.4. Résultats attendus.....17
3.5. Chronogramme des travaux.....18
PARTIE II: ANALYSE ET CONCEPTION.....19
chapitre 4: Analyse préalable.....20

4.1. Analyse de l'existant....20
4.2. Critique de l'existant....24
4.3. Proposition de solution....25
4.4. Solution retenue....26
chapitre 5: Choix des outils....27
5.1. Conteneurisation....27
5.2. systèmes de gestion de code source....29
5.3. Système d'intégration et de déploiement continus(CI/CD)....31
5.4. Orchestration de conteneurs....33
5.5. Surveillance et monitoring....35
5.6. Serveur proxy....39
PARTIE 3: REALISATION....41
chapitre 6: Installation et configuration....42
6.1. Configuration de github....42
6.2. Mise en place d'un environnement de développement conteneurisé et automatisé....43
6.3. Phase de staging : validation locale avant mise en production....60
6.4. Mise en place de l'environnement de production....79
chapitre 7: Test et validation....115
7.1. Architecture finale du projet....115
7.2. Validation des workflows CI/CD....119
7.3. Tests de supervision et alerting....127
CONCLUSION....135
REFERENCES BIBLIOGRAPHIQUES....X
REFERENCES WEBOGRAPHIQUES....XI
GLOSSAIRE....XIII
TABLE DES MATIERES....XVI

# LISTE DES FIGURES

Figure 1: Organigramme de l'Ecole Nationale d'Informatique.....5
Figure 2: Architecture des études correspondant au système LMD.....7
Figure 3 : Implémentation géographique du JIRAMA.....13
Figure 4: Organigramme du jirama.....15
Figure 5: Topologie réseau de la JIRAMA.....20
Figure 6: Graphe de la bande passante générale d'un employé.....22
Figure 7: Graphe de la protocole http d'un employé.....22
Figure 8: Architecture de docker.....29
Figure 9: Fonctionnement de github actions.....33
Figure 10: Architecture de docker compose.....35
Figure 11: Architecture de prometheus.....37
Figure 12: Architecture de l'environnement de développement.....44
Figure 13: Liste des variables et secrets github en staging.....78
Figure 14: Interface de configuration du VPS.....79
Figure 15: Création de l'utilisateur non root.....80
Figure 16: Commande de configuration du dépôt docker.....81
Figure 17: Fichier .env.global pour la production.....83
Figure 18: Contenue du fichier my-exporter.cnf.....83
Figure 19: Création de l'utilisateur mysql-exporter.....84
Figure 20: Configuration dns du serveur.....85
Figure 21: Vérification du répertoire letsenscrypt.....86
Figure 22: Création des volumes dans le VPS.....87
Figure 23: Architecture d'un CI/CD.....95
Figure 24: Liste de secrets mise en place dans github.....96
Figure 25: Source de données dans grafana.....109
Figure 26: Dashboard grafana.....109
Figure 27: Création de l'application grafana alerts dans slack.....110
Figure 28: Création d'un webhook dans slack.....111
Figure 29: Installation de l'application grafana-alerts dans slack.....111
Figure 30: Création du contact point dans grafana.....112
Figure 31: Architecture final du projet.....116

Figure 32: Validation du workflow development.yml.....119
Figure 33: Validation du workflow staging.yml.....119
Figure 34: Visualisation des images backend dédié à la staging sur dockerhub.....120
Figure 35: Visualisation des images frontend dédié à la staging sur dockerhub.....120
Figure 36: Validation du workflow ci.yml.....121
Figure 37: Validation du workflow cd.yml.....121
Figure 38: Visualisation des images backend dédié à la production sur dockerhub.....122
Figure 39: Visualisation des images backend dédié à la production sur dockerhub.....122
Figure 40: Utilisation du commande makefile.....123
Figure 41: Visualisation de l'application en phase de développement.....124
Figure 42: Visualisation de l'application en phase de staging.....125
Figure 43: Visualisation du résultat du workflow cd.yml.....126
Figure 44: Visualisation de l'interface de l'application en production.....127
Figure 45: Demande d'authentification pour accédez à l’interface de grafana.....128
Figure 46: Demande d'authentification pour accédez à l’interface de prometheus.....128
Figure 47: Vérification des job dans l'interface de prometheus.....129
Figure 48: Dashboard de supervision de prometheus.....130
Figure 49: Dashboard node-exporter pour la supervision du VPS.....130
Figure 50: Dashboard de supervision du backend Django.....131
Figure 51: Dashboard mysq-exporter pour la supervision de la base de données mysql.....131
Figure 52: Dashboard Loki pour la supervision des logs docker et système.....132
Figure 53: Visualisation des alertes dans grafana.....133
Figure 54: Visualisation des alertes dans l'application Slack.....134

# LISTE DES TABLEAUX

Tableau 1: Organisation du système de formation pédagogique de l'Ecole.....6
Tableau 2: Mention et parcours au sein de l'ENI.....6
Tableau 3: Liste des formations existantes à l'ENI.....7
Tableau 4: Débouchés éventuels des jeunes diplômés.....9
Tableau 5: Moyen matériel.....16
Tableau 6: Liste des moyens logiciel.....17
Tableau 7: Chronogramme de travaux.....18
Tableau 8: Liste de connexion internet.....20
Tableau 9: Liste de passerelle.....21
Tableau 10: Liste de poste de travail.....21
Tableau 11: Critique de l'existant De l'architecture actuel de la JIRAMA.....24
Tableau 12: Tableau comparatif des principaux types de conteneurs.....27
Tableau 13 :Tableau comparatif des systèmes de codes sources.....29
Tableau 14:Comparaison des outils CI/CD.....31
Tableau 15: Comparaison des outils d'orchestration de conteneur.....33
Tableau 16: Tableau comparatif des outils de monitoring.....36
Tableau 17: Environnement cible.....42
Tableau 18: Rôle principale des outils de développement.....44
Tableau 19: Listes des rapports générés dans github.....60
Tableau 20: Explication des rôles des fichiers docker-compose en staging.....62
Tableau 21: Les tags des images docker.....97

# LISTE DES ABREVIATIONS

- **API** : Application Programming Interface

- **ASR**: Administration des Systèmes et Réseaux

- **AUF** : Agence Universitaire de la Francophonie

- **BP** : Boîte Postale

- **BT** : Basse Tension

- **CI/CD** : Continuous Integration / Continuous Deployment

- **CNH** : Commission Nationale d'Habilitation

- **DG**: Directeur Général .

- **DNS** : Domain Name System

- **EEM** : Électricité et Eaux de Madagascar

- **ENI**: École Nationale d'Informatique

- **GB**: Génie Logiciel et Base de Données

- **GID**: Gouvernance et Ingénierie de Données

- **HA** : Haute Disponibilité

- **HT** : Haute Tension

- **HTTP**: HyperText Transfer Protocol

- **IA**: Intelligence Artificielle

- **IG**: Informatique Générale

- **JIRAMA**: Jiro sy Rano Malagasy

- **JSON** : JavaScript Object Notation

- **LMD**: Licence-Master-Doctorat

- **LogQL**: Log Query Language

- **MDi**: Métiers du Digital

- **MT** : Moyenne Tension

- **NIF**: Numéro d'Identification Fiscale

- **OCC** : Objets Connectés et Cybersécurité

- **PL-SQL**: Procedural Language/Structured Query Language.

- **PromQL**: Prometheus Query Language

- **RCS**: Registre du Commerce et des Sociétés

- **SEM**: Société des Eaux de Madagascar

- **SLO**: Service Level Objective

- **SQL**: Structured Query Language

- **SSH** : Secure Shell

- **TIC** : Technologies de l'Information et de la Communication

- **TLS**: Transport Layer Security

- **UFW** : Uncomplicated Firewall

- **UPST**: Université Paul Sabatier de Toulouse

- **VPS**: Virtual Private Server

-**YAML**:Yet Another Markup Language

# INTRODUCTION GENERALE

Dans un contexte où la transformation numérique s'impose comme un levier stratégique pour les entreprises, la JIRAMA, compagnie nationale d'eau et d'électricité de Madagascar, fait face à des défis opérationnels majeurs dans la gestion des pannes signalées par les usagers. La communication entre les citoyens et les services techniques reste limitée, et les processus de déploiement des applications web sont encore largement manuels, entraînant des risques d'erreurs, des temps d'arrêt prolongés, et une absence de supervision en temps réel.

Face à cette situation, une problématique centrale émerge : comment mettre en place une infrastructure DevOps fiable, automatisée et sécurisée pour le déploiement continu d'une application web, tout en garantissant une haute disponibilité, une supervision proactive, et une maîtrise totale de l'environnement de production dans un contexte à ressources limitées ?

Pour répondre à cette problématique, ce travail propose la mise en place d’une approche basée sur l’automatisation des processus de développement et de déploiement, ainsi que sur l’amélioration de la collaboration entre les phases de conception, de mise en production et d’exploitation. Cette approche repose sur l’intégration de pratiques permettant d’assurer une livraison continue des applications, une meilleure maîtrise des environnements et une visibilité accrue sur le fonctionnement des systèmes en production.

Ce mémoire s'articule en trois parties, la première partie du mémoire détaillera la présentation de l’Ecole Nationale d’Informatique et celle de l’entreprise JIRAMA,ainsi que la description du projet, La deuxième partie du mémoire sera consacrée à l’analyse préalable et la conception.Enfin, la réalisation et la validation seront abordées dans la dernière partie.

# PARTIE I:  PRESENTATION GENERALE

# chapitre 1: Présentation de l'Ecole Nationale d'Informatique

L'École Nationale d'Informatique (ENI) est reconnue comme un établissement de référence à Madagascar dans le domaine de la formation en informatique.

## 1.1. Information d'ordre générale

L'Ecole Nationale d'Informatique, en abrégé ENI, est un établissement d'enseignement supérieur rattaché académiquement et administrativement à l'Université de Fianarantsoa. Le siège de l'Ecole se trouve à Tanambao-Antaninarenina à Fianarantsoa. L'adresse pour la prise de contact avec l'Ecole est la suivante : Ecole Nationale d'Informatique (ENI) Tanambao, Fianarantsoa. Le numéro de sa boîte postale est 1487 avec le code postal 301. Téléphone : 038 96 566 96 ou 034 76 591 95. Son adresse électronique est la suivante : **eni@eni.mg**. Il dispose également d'un site web : **www.eni.mg**

## 1.2. Missions et historiques

L'ENI se positionne sur l'échiquier socio-éducatif malgache comme étant le plus puissant secteur de diffusion et de vulgarisation des connaissances et des technologies informatiques.

Cette Ecole Supérieure peut être considérée aujourd'hui comme la vitrine et la pépinière des élites informaticiennes du pays.

De façon formelle, l'ENI était créée par le décret N° 83- 185 du 24 Mai 1983, comme étant le seul établissement Universitaire Professionnalisé au niveau national, destiné à former des techniciens et des Ingénieurs de haut niveau, aptes à répondre aux besoins et exigences d'Informatisation des entreprises, des sociétés et des organes implantés à Madagascar.

L'ENI a pour conséquent pour mission de former des spécialistes informaticiens compétents et opérationnels de différents niveaux notamment :

- En fournissant à des étudiants des connaissances de base en informatique ;
- En leur transmettant le savoir-faire requis, à travers la professionnalisation des formations dispensées et en essayant une meilleure adéquation des formations par rapport aux besoins évolutifs des sociétés et des entreprises ;
- En initiant les étudiants aux activités de recherche dans les différents domaines des Technologies de l'Information et de la Communication (TIC) ;

La filière de formation d'Analystes Programmeurs a été mise en place à l'Ecole en 1983, et a été gelée par la suite en 1996, tandis que la filière de formation d'ingénieurs a été ouverte à l'Ecole en 1986.

Une formation de troisième cycle a été ouverte à l'Ecole depuis l'année 2003 – 2004 grâce à la coopération académique et scientifique entre l'Université de Fianarantsoa pour le compte de l'ENI et l'Université Paul Sabatier de Toulouse (UPST).

Cette filière avait pour objectif de former certains étudiants à la recherche dans les différents domaines de l'Informatique, et notamment pour préparer la relève des Enseignants-Chercheurs qui étaient en poste.

Pendant l'année 2007-2008, la formation en vue de l'obtention du diplôme de Licence Professionnelle en Informatique a été mise en place à l'ENI avec les deux parcours de formation:

- Génie Logiciel et base de Données.
- Administration des Système et réseaux.

La mise en place à l'Ecole de ces deux options de formation devait répondre au besoin de basculement vers le système Licence – Master – Doctorat (LMD).

En vue de surmonter les difficultés de limitation de l'effectif des étudiants accueillis à l'Ecole, notamment à cause du manque d'infrastructures, un système de « Formation Hybride » a été mise en place à partir de l'année 2010. Il s'agit en effet d'un système de formation semi présentielle et à distance avec l'utilisation de la visioconférence pour la formation à distance. Le système de formation hybride a été ainsi créé à Fianarantsoa ainsi qu'Université de Toliara. Cette formation est à l'origine du parcours Informatique Générale.

En 2023, la mention Intelligence Artificielle (IA) a été ouverte au sein de l'Ecole pour répondre les besoins des entreprises. La formation est destinée aux étudiants titulaires du diplôme de licence (Bac +3) en Mathématiques ou en Statistiques ou en Informatique, etc. La mention IA comporte deux parcours :

- Gouvernance et Ingénierie de Données (GID),
- Objets connectés et Cybersécurité (OCC).

Le principe de l'enseignement pour le parcours GID offre aux étudiants des compétences scientifiques et techniques spécialisées en Science de données. Pour le parcours OCC, les étudiants octroient la double spécialité premièrement en internet des objets et deuxièmement en cybersécurité. La formation de master est axée sur l'ensemble d'applications de l'Intelligence Artificielle.

Depuis l'année universitaire 2024-2025, la mention Expertise Digitale est officiellement ouverte au grand public. Cette mention propose deux (2) parcours distincts :

- Métiers du Digital (MDi) – disponible aux niveaux Licence et Master. Ce parcours est destiné aux étudiants titulaires du diplôme de Baccalauréat, toutes séries confondues. Il est élaboré pour équiper les étudiants des compétences essentielles dans les domaines du marketing digital, de la communication digitale, et de la gestion de projets digitaux.
- Audit des Systèmes d'Information (ASI) – proposé au niveau Master uniquement. Ce parcours s'adresse aux étudiants titulaires d'une licence (Bac +3) en Mathématiques, en

Statistiques ou en Informatique. Il est structuré pour approfondir les connaissances théoriques et pratiques nécessaires à l'audit efficace des systèmes d'information.

Le système d'enseignement adopté pour cette mention est un système en ligne. Ainsi, les cours et ressources pédagogiques sont accessibles en permanence via la plateforme numérique Moodle. Par ailleurs, des regroupements en présentiel ou en ligne seront organisés périodiquement, conformément au planning défini par les responsables pédagogiques.

## 1.3. Organigramme institutionnel

L'organigramme de l'Ecole est inspiré des dispositions du décret N° 83-185 du 24 Mai 1983. L'ENI est administrée par un Conseil d'Ecole, et dirigée par un directeur nommé par un décret adopté en Conseil des Ministres. Le Collège des enseignants regroupant tous les enseignants-chercheurs permanents de l'Ecole est chargé de résoudre les problèmes liés à l'organisation pédagogique des enseignements. Le Conseil Scientifique propose les orientations pédagogiques et scientifiques de l'établissement, en tenant compte notamment de l'évolution du marché de travail et de l'adéquation des formations dispensées par rapport aux besoins des entreprises.

La figure 1 représente l'organigramme actuel de l'Ecole Nationale d'Informatique.

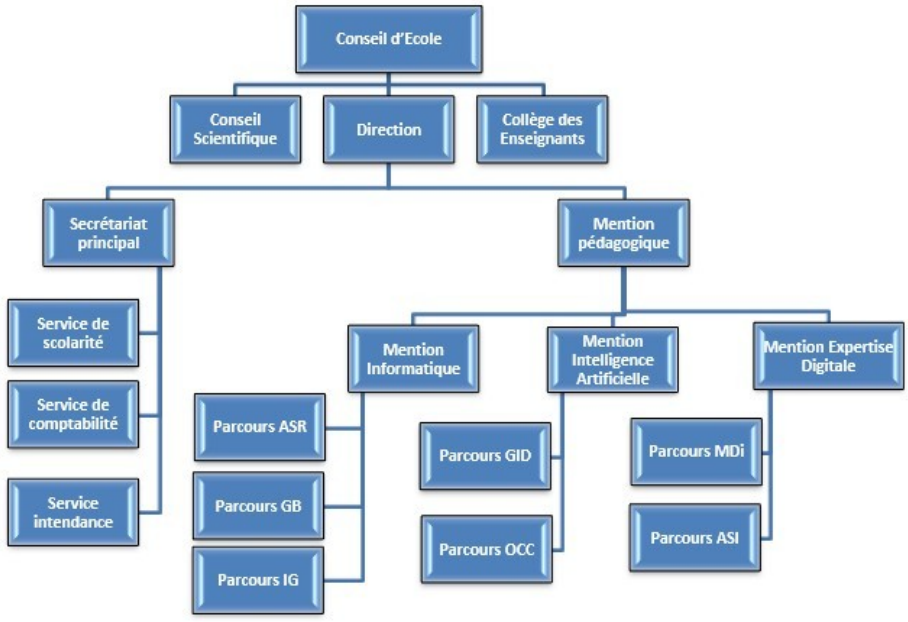



*Figure 1: Organigramme de l'Ecole Nationale d'Informatique*

## 1.4. Domaine de spécialisation

Les activités de formation et de recherche organisées à l'ENI portent sur les domaines suivants :

- Génie logiciel et Base de Données ;
- Administration des Systèmes et Réseaux ;
- Informatique Générale ;
- Modélisation informatique et mathématique des Systèmes complexes ;
- Intelligence artificielle.

Le tableau 1 décrit l'organisation du système de formation pédagogique de l'Ecole.

*Tableau 1: Organisation du système de formation pédagogique de l'Ecole*

| Formation Théorique | Formation Pratique |
|---|---|
| ➢ Enseignement théorique<br>➢ Travaux dirigés<br>➢ Travaux pratiques<br>➢ Conférences | ➢ Etude de cas<br>➢ Travaux de réalisation<br>➢ Projets/ Projets tutorés<br>➢ Voyages d'Etudes<br>➢ Stages en entreprise |

## 1.5. Architecture des formations pédagogiques

Le recrutement des étudiants à l'ENI se fait uniquement par voie de concours d'envergure nationale en première année. Les offres de formation organisées à l'Ecole ont été validées par la Commission Nationale d'Habilitation (CNH). Au sein de l'ENI, il existe deux mentions et cinq parcours. Le tableau 2 récapitule les mentions et les parcours au sein de l'Ecole :

*Tableau 2: Mention et parcours au sein de l'ENI*

| **Mention** | **Parcours** |
|---|---|
| **Informatique** | Génie logiciel et Base de Données (GB) |
| | Administration des Systèmes et Réseaux (ASR) |
| | Informatique Générale (IG) |
| **Intelligence Artificielle** | Gouvernance et Ingénierie de Données (GID) |
| | Objets Connectés et Cyber sécurités (OCC) |
| **Expertise Digitale** | Métiers du Digital (MDi) |
| | Audit des Systèmes d'Information (ASI) |

La figure 2 représente l'architecture des études correspondant au système LMD.

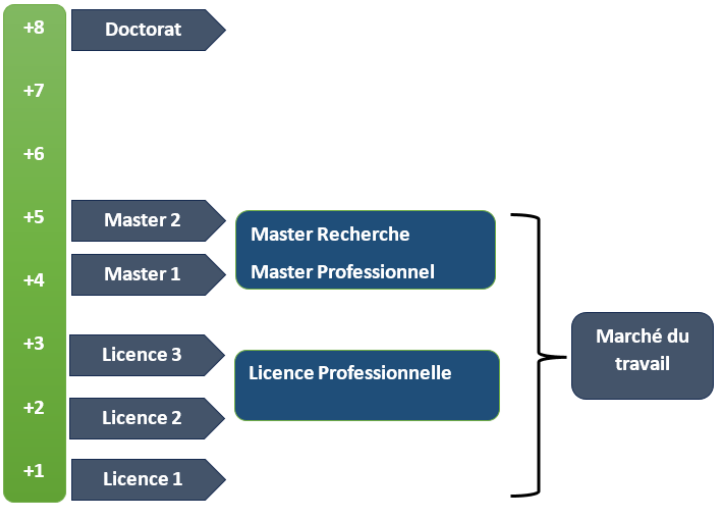



*Figure 2: Architecture des études correspondant au système LMD*

La licence peut avoir une vocation générale ou professionnelle. Le master peut avoir une vocation professionnelle ou de recherche. L'accès en première année de MASTER se fait automatiquement pour les étudiants de l'Ecole qui ont obtenu le diplôme de Licence Professionnelle.

Le tableau 3 illustre la liste des formations existantes à l'ENI.

*Tableau 3: Liste des formations existantes à l'ENI*

| | FORMATION | |
|---|---|---|
| | LICENCE PROFESSIONNELLE | MASTER |
| Condition admission | Par voie de concours | Par voie de concours pour la mention IA |
| Condition d'accès | Bac de série C, D ou Technique | Être titulaire de licence professionnelle |
| Durée de Formation | 3 ans | 2 ans |
| Diplôme délivré | Diplôme de Licence Professionnelle | Diplôme de Master Professionnel<br>Diplôme de Master Recherche |

Le Master Recherche permet à son titulaire de poursuivre directement des études en doctorat et de s'inscrire directement dans une Ecole Doctorale.

Les étudiants diplômés de l'Ecole sont plutôt bien accueillis dans les instituts universitaires étrangères (Canada, Suisse, France, …)

## 1.6. Relation de l'ENI avec les organismes externes

Les stages effectués chaque année par les étudiants mettent l'Ecole en rapport permanent avec plus de 400 entreprises et organismes publics, semi-publics et privés, nationaux et internationaux.

L'Ecole dispose ainsi d'un réseau d'entreprises, de sociétés et d'organismes publics et privés qui sont des partenaires par l'accueil en stage de ses étudiants, et éventuellement pour le recrutement après l'obtention des diplômes par ces derniers. Les compétences que l'Ecole cherche à développer chez ses étudiants sont l'adaptabilité, le sens de la responsabilité, du travail en équipe, le goût de l'expérimentation et l'innovation.

En effet, la vocation de l'ENI est de former des licenciés et des ingénieurs de niveau MASTER avec des qualités scientifiques, techniques et humaines reconnues, capables d'évoluer professionnellement dans des secteurs d'activité variés intégrant l'informatique. Les stages en milieu professionnel permettent de favoriser une meilleure adéquation entre les formations à l'Ecole et les besoins évolutifs du marché de l'emploi.

Parmi les sociétés, les entreprises et les organismes partenaires de l'Ecole, on peut citer : ACCENTURE Mauritius, AKATA Goavana, Air Madagascar, Ambre Associates, Airtel, Agence Universitaire de la Francophonie (AUF), AXIAN, B2B, Banque Centrale, , BIANCO, BlueLine, CNaPS, Bureau National de Gestion des Risques et des Catastrophes (BNGRC), CEDII-Fianarantsoa, Data Consulting, Central Test, Centre National Antiacridien, CNRE, COLAS, Direction Générale des Douanes, DLC, E-Tech Consulting, , FID, FIHARY Soft, FTM, GNOSYS, GENIUS AT WORK, Hello Tana, IBONIA, INGENOSIA, INSTAT, IOGA, JIRAMA, JOUVE, MADADEV, MAEP, MANAO, MEF, MEN, MESupRES, MFB, , MININTER, Min des Postes/Télécommunications et du Développement Numérique, NEOV MAD, Ny Havana, Madagascar National Parks, OMNITEC, ORANGE, OTME, PRACCESS, QMM Fort-Dauphin, SG Madagasikara SMMC, SMMEC, SNEDADRS Antsirabe, Sénat, Société d'Exploitation du Port de Toamasina (SEPT), SOFTWELL, Strategy Consulting, TELMA, VIVETEC, Société LAZAN'I BETSILEO, WWF, UGD, ARATO, MANAO, MNDPT, NG ACADEMY.NG, Relia, Spoon Consulting …

## 1.7. Débouchés professionnels et diplômés

Les formations proposées par l'Ecole permettent aux diplômés d'être immédiatement opérationnels sur le marché du travail avec la connaissance d'un métier complet lié à l'informatique aux TIC.

L'Ecole apporte à ses étudiants un savoir-faire et un savoir-être qui les accompagnent tout au long de leur vie professionnelle. Elle a une vocation professionnalisante. Les diplômés en LICENCE et en MASTER issus de l'ENI peuvent faire carrière dans différents secteurs.

L'Ecole bénéficie aujourd'hui de 40 années d'expériences pédagogiques et de reconnaissance auprès des sociétés, des entreprises et des organismes. C'est une Ecole Supérieure de référence en matière informatique.

D’une manière générale, les diplômés de l’ENI n’éprouvent pas de difficultés particulières à être recrutés au terme de leurs études. Cependant, l’ENI recommande à ses diplômés de promouvoir l’entrepreneuriat en TIC et de créer des cybercafés, des SSII ou des bureaux d’études.

Le tableau 4 représente les débouchés éventuels selon la mention.

*Tableau 4: Débouchés éventuels des jeunes diplômés*

| **Mention** | **Débouchés professionnels** |
|---|---|
| INFORMATIQUE | GB : Développeur logiciel / développeur full-stack, Ingénieur en développement d’applications (desktop, web, mobile), Architecte logiciel, Intégrateur d’applications, Développeur d’API, Administrateur de bases de données (DBA), Ingénieur en conception de bases de données, Développeur SQL / PL-SQL, Spécialiste en migration et optimisation de bases de données, Data engineer (ingénieur en traitement de données), Analyste programmeur, Analyste fonctionnel, Chef de projet informatique, Scrum master / Product owner (en méthodologie Agile), Ingénieur QA (Quality Assurance), Testeur logiciel / testeur automatisé, Spécialiste maintenance applicative, Expert en refactoring et optimisation de code, Analyste de données (Data analyst), Concepteur d’outils décisionnels (BI), Ingénieur en Big Data, Machine Learning Engineer, Consultant fonctionnel ou freelance<br>ASR : Administrateur systèmes et réseaux, Ingénieur systèmes et réseaux, Administrateur systèmes Linux / Windows, Administrateur cloud (AWS, Azure, Google Cloud), Ingénieur virtualisation (VMware, Hyper-V, Proxmox), Administrateur sécurité (Security Administrator), Ingénieur cybersécurité, Analyste SOC (Security Operations Center), Spécialiste firewall / VPN, Responsable sécurité informatique (RSSI – avec expérience), Technicien support informatique (N2/N3), Technicien réseaux et télécoms, Ingénieur exploitation, Administrateur messagerie (Exchange, Microsoft 365), Technicien datacenter, Architecte réseaux, Architecte systèmes, Consultant infrastructure, Chef de projet systèmes et réseaux, Ingénieur DevOps, Administrateur Kubernetes / Docker, Cloud engineer, Spécialiste automatisation (Ansible, Terraform)<br>IG : GB et SR en même temps |
| INTELLIGENCE ARTIFICIELLE | GID : Développeur IA, Data Scientist, Entrepreneur IA, Consultant IA, Responsable marketing IA, Business Developer IA, Product Owner, Consultant en cybersécurité, Directeur de l’Innovation ; OCC : Analyste en cybersécurité, Ingénieur en cybersécurité, Spécialiste en réponse aux incidents, Architecte de sécurité, Consultant en cybersécurité, Auditeur en cybersécurité, Expert en sécurité des |

| | |
|---|---|
| | données, Responsable de la sécurité des informations, Chercheur en cybersécurité, Formateur en cybersécurité, Ingénieur IoT, Développeur IoT, Architecte IoT, Spécialiste en sécurité IoT, Analyste de données |
| EXPERTISE DIGITALE | MDi : Spécialiste en Marketing Digital, Gestionnaire de Contenu Web, Analyste UX/UI, Product Owner, Chef de Projet Digital, Stratège en Médias Sociaux, Analyste de Performance Digitale, Chef de Projet Digital, Stratège en Médias Sociaux, Concepteur-Rédacteur Web Analyste de Performance Digitale, Expert en Commerce Électronique, Expert en Commerce Électronique, Spécialiste SEO/SEM, Manager de Communauté (Community Manager), Conseiller en Stratégie Digitale, Responsable CRM (Customer Relationship Management), Spécialiste en Analytique et Intelligence d'Affaires (Business Intelligence), Consultant en Transformation Digitale, Responsable des Partenariats Digitaux, Spécialiste en Cybersécurité Marketing, Directeur de l'Innovation Digitale, Manager de l'E-réputation<br>ASI : Auditeur des Systèmes d'Information, Consultant en Sécurité des Systèmes d'Information, Analyste de Risques IT, Chef de Projet SI, Responsable de la Conformité IT, Spécialiste en Cyber Forensics, Directeur de l'Audit et de la Sécurité des Systèmes d'Information, Expert en Gouvernance IT, Formateur ou Éducateur en Sécurité des Chercheur en Sécurité IT |

## 1.8. Ressources humaines

Les ressources humaines sont citées ci-dessous selon leurs responsabilités :

- Directeur de l'Ecole : Monsieur MAHATODY Thomas, Professeur
- Responsable de la Mention « Informatique » : Monsieur RALAIVAO Jean Christian, Maître de Conférences
- Responsable de la Mention « Intelligence Artificielle » : Monsieur DIMBISOA William Germain, Maître de Conférences

L'ENI compte douze (12) enseignants permanents dont un (01) Professeur Titulaire, deux (02) Professeurs, Sept (07) Maîtres de Conférences, Deux (02) Assistants d'Enseignement Supérieur et de Recherche, dix (10) enseignants vacataires, quarante un (41) personnel administratif.

# chapitre 2: Présentation de l'entreprise JIRAMA

Dans ce chapitre, on peut voir une présentation de la société JIRAMA MADAGASCAR. Cette présentation est utile d'une part pour comprendre l'évolution de la société depuis sa création jusqu'à maintenant et d'autre part pour analyser sur le fonctionnement de la société.

## 2.1. Présentation générale de la Société JIRAMA Madagascar

En général, toute entreprise a pour objectif principal « la formation brute de capital fixe ou le FBCF ».En raison du progrès et de l'évolution économique, la JIRAMA ne s'empêche pas de rechercher des meilleures solutions ou orientations dans le cas de gestion de ressources et le maintien de son perfectionnement.

On va voir ci-dessous la présentation de la société JIRAMA Madagascar depuis sa création jusqu'à maintenant.

## 2.2. Historique

La JIRAMA fait partie des usines industrielles très puissantes à Madagascar. Elle a été créée par l'ordonnance N° 75024 du 17 octobre 1975 sous tutelle du Ministère de l'énergie et de mine (MEM). En 1972,elle est devenue « société malgache de l'eau et électricité(SMEE) ».

Depuis 1975 : la compagnie est actuellement dénommée « JIro sy RAno MAlagasy(JIRAMA) . La JIRAMA est présente dans toutes les villes de Madagascar malgré la libération du secteur de production d'électricité et d'eau, elle domine toujours ce secteur d'activité. Elle est le premier distributeur public d'eau potable et d'électricité apparue à Antananarivo à la fin du 20$^{ème}$ siècle.

En 1905 : l'Etat a confirmé son monopole en créant la Société Malagasy de l'eau et de l'électricité par l'attribution de l'actif et du passif du bilan de l'EEM. Il a également regroupé la passation de la totalité des ressources et des emplois de la SMEE et de l'EEM à l'interieur d'une nouvelle société appelée Société d'Intérêt Nationale de l'Eau et de l'Electricité(SINEE)

En octobre 1975 : La SINEE est dissoute et le nom Jiro sy Rano Malagasy a été créée en relation étroite avec l'ordonnance N°75024 du 17 octobre 1975. Un Malagasy, Monsieur RAKOTOARIVELA Caleb a pris la Direction Générale.

En 1983 : un accord a été vu le jour par la JIRAMA avec les Bailleurs de fonds étrangers à tire de projet ENERGIE I pour les réalisations telles que les barrages Andekaleka- Antananarivo qui est le principal barrage hydraulique d'énergie de Madagascar et aussi de Namorona- Fianarantsoa, Et de même pour les travaux de réhabilitation des équipements de la centrale Hydro-électrique de Volobe-Toamasina . Le projet énergie I visera l'amélioration des perspectives de croissance durable de Madagascar en assurant un approvisionnement adéquat en électricité à moyen terme ainsi qu'à un meilleur accès au service de la population urbaine et rurale.

En 1992 : Un nouvel accord de financement avec la Banque Mondial pour le Développement de ces services et pour assurer les projets d'investissements d'électricité pour les Cinq années à venir.

De 2005 jusqu'à 2007 , un étranger Monsieur BERNARD ROMAN dirigeait la JIRAMA. Le contrat de gestion était confié à un Allemand, lequel fut constitué jusqu'à la fin de mois de mars 2008, le directeur générale étranger a été remplacé pour un Malagasy Monsieur RASIDY Désiré

## 2.3. Indentité juridique

La JIRAMA est une société de prestation de service, elle distribue d'électricité et d'eau potable à Madagascar. C'est une société semi-privée régie par le droit commun des sociétés anonymes.

La majorité de son capital appartient à l'Etat. Il se chiffre à 17533084304.20 Ariary. Cette entreprise est identifiable fiscalement sous le NIF 0021411, inscrite aux registres de commerce 7120 Antananarivo, Son siège social se trouve au 19 Rue Rainandriamampandry Ambohijatovo, Antananarivo selon l'ordonnance N°75-024 du 17 octobre 1975.Il se compose d'un Conseil Administration de Neuf membres dont Six représentants du personnel de la Société.

La JIRAMA est encore dans le cadre de la direction publique. Son siège social est joignable par téléphone sous le N°22 200 31. Quant à la JIRAMA à Fianarantsoa, elle est joignable par téléphone sous le N° 75 510 22 ou aussi par e-mail : **www.jirama.mg**

La figure 3 montre l'implémentation du JIRAMA à Madagascar

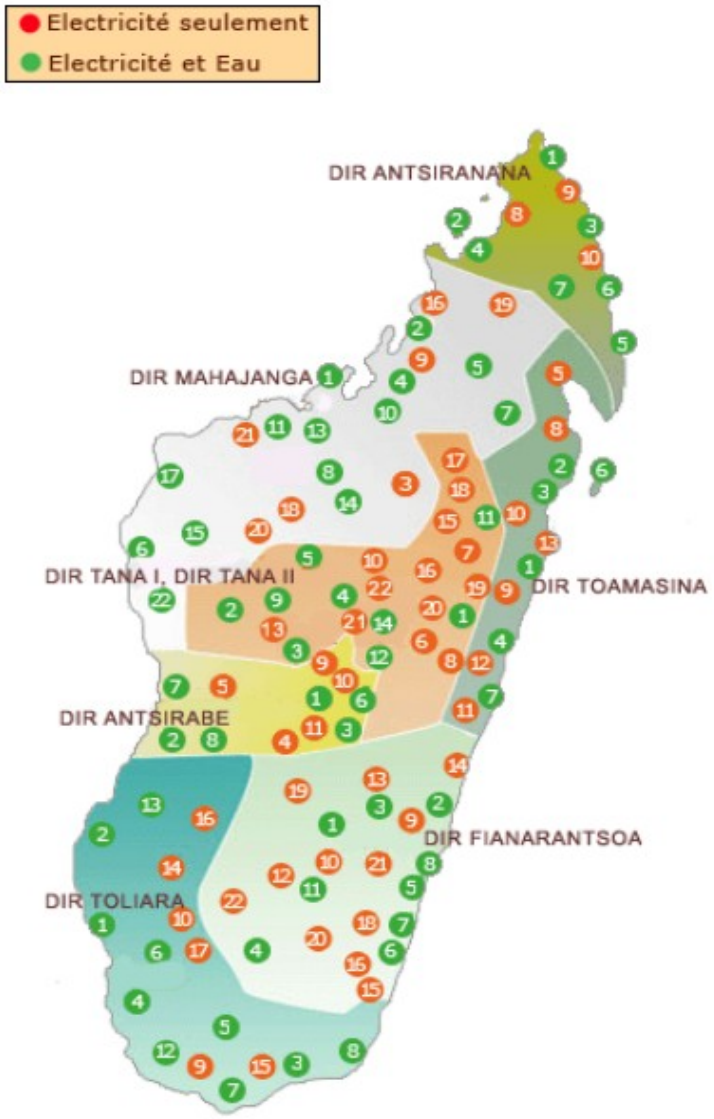



*Figure3 : Implémentation géographique du JIRAMA*

***Source : site internet du JIRAMA***

## 2.4. Activité de la société

On peut définir une activité comme l'ensemble des tâches indispensables à l'accomplissement d'une fonction. Une société

peut avoir une fonction qui la distingue des autres sociétés ou bien elle exerce une fonction similaire aux autres sociétés. La JIRAMA a comme fonction la production, le transport et la distribution de l'énergie d'une partie et se focalise également sur l'adduction et la distribution d'eau d'autre partie.

### 2.4.1. Production d'électricité

La production d'électricité est l'ensemble des procédés permettant de transformer l'énergie hydraulique, thermique,mécanique ou même nucléaire en énergie électrique. La JIRAMA produit de l'électricité dans une centrale électrique quiutilise un moteur alimenté par un carburant nommé gasoil.Alors, pour ne pas se retrouver dans un état de panique imprévu causé par le manque de

carburant ou bien pour éviter toutepanne technique, il est extrêmement essentiel de bel et bien surveiller les agents de quart d'une manière permanente.

### 2.4.2. La production d'eau

La production d'eau potable désigne l'ensemble des opérations de captage, de traitement et de conditionnement des ressources hydriques (eaux de surface ou souterraines) pour en faire un approvisionnement sûr et accessible à la population. À Ambohimahasoa, dans le secteur de la JIRAMA (rattaché à la région Haute Matsatra), la production d'eau est gérée depuis l'arrivée de la compagnie en 1994, en complément de ses activités électriques. Elle repose principalement sur des forages et pompes pour exploiter les nappes phréatiques locales, suivies d'un traitement basique (chloration et filtration) afin de respecter les normes sanitaires.

### 2.4.3. Le transport et la distribution

Le transport est l'ensemble des moyens permettant de transiter l'énergie électrique en haute tension (ITT) ou en moyenne tension (MT), soit entre les usines ou entre les différents postes de transformation.

La distribution est l'ensemble des moyens permettant de transmettre et de livrer l'énergie électrique, en moyen et basse tension, à la sortie de la centrale jusqu'aux consommateurs finaux.

## 2.5. Organigramme de la société

La figure 4 représente l' organigramme du Jirama

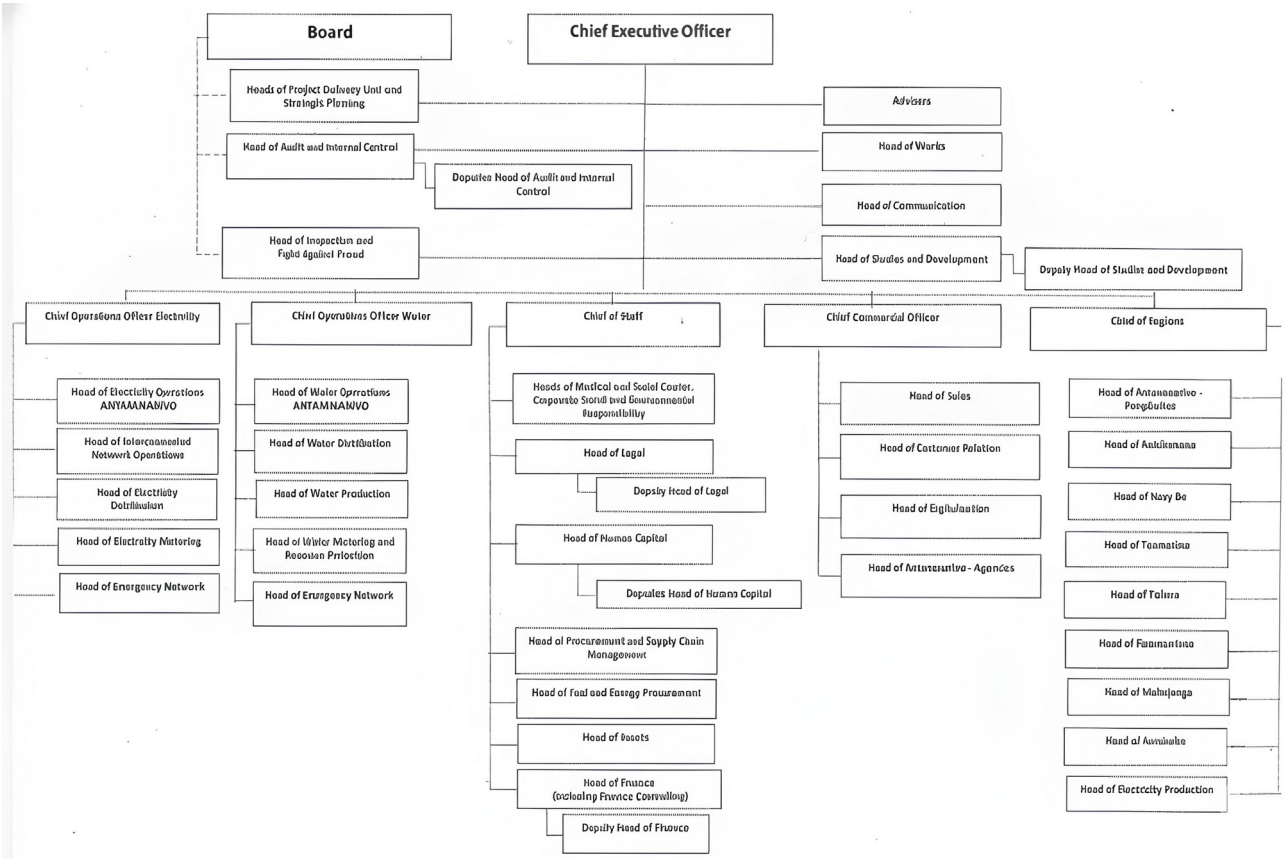



*Figure 4: organigramme du jirama*

# chapitre 3: Description du projet

## 3.1. Formulation

Le projet consiste à concevoir et mettre en place une infrastructure DevOps complète pour le déploiement, la supervision et la sécurisation d'une application web de gestion de pannes. Cette application, développée en ReactJS pour le frontend et Django pour le backend, vise à permettre aux usagers de signaler des incidents liés à la distribution d'eau et d'électricité. Le projet s'inscrit dans une démarche de professionnalisation du déploiement, en remplaçant les solutions temporaires par une architecture stable, automatisée et supervisée.

## 3.2. Objectifs

L'objectif principal est de mettre en place un pipeline CI/CD sécurisé et scalable pour le déploiement automatiser de l'application web de gestion de panne de l'entreprise JIRAMA.

## 3.3. besoins de l'entreprise

Voici les besoins de l'entreprise:

- Disposer d'une infrastructure de production stable et maîtrisée
- Automatiser les processus de livraison et de mise à jour
- Surveiller les performances et détecter les incidents rapidement
- Centraliser les logs et les métriques pour faciliter le diagnostic
- Réduire les interventions manuelles et les erreurs humaines

## 3.4. Moyens nécessaires à la réalisation du projet

### 3.4.1. Moyens matériels

*Tableau 5: moyen matériel*

| **Désignation et nombre** | **Caractéristiques** | **Système d'exploitation** |
|---|---|---|
| Ordinateur portable (1) | Processeur Intel Core i3 3ème génération, 8 Go RAM, HDD 500 Go,4 CPU(2,4Ghz) | Ubuntu 25.04 |
| PC de bureau (1) | Processeur Intel Core i5 3ème génération, 8 Go RAM, SSD 256 Go,4CPU(2,2Ghz) | Ubuntu 24,04 |
| Routeur Telma (1) | Modem 4G LTE, Wi-Fi 2.4GHz | Firmware Telma intégré |
| Connexion Internet Wi-Fi | Débit moyen 10 Mbps | — |
| VPS (1) | 4vCPU, 8 Go RAM, 150 Go | Ubuntu 24.04 |

| | | |
|---|---|---|
| | SSD, IP publique | |

### 3.4.2. Moyens logiciels

*Tableau 6: Liste des moyens logiciel*

| **Outils** | **Fonction principale** | **Version** |
|---|---|---|
| Docker | Conteneurisation | v28.2.2 |
| Docker Compose | orchestration des conteneurs | v5.0.0 |
| GitHub | Gestion du code source et collaboration | Plateforme web (pas de version locale) |
| GitHub Actions | Automatisation des pipelines CI/CD | v1.0.32 |
| Nginx | Reverse proxy et routage Blue/Green | v1.25 |
| Prometheus | Collecte des métriques système et applicatives | v3.8.1 |
| Grafana | Visualisation des métriques et alerting | v12.3.1 |
| Loki | Collecte et envoi des logs à Loki | v2.9.4 |
| Promtail | Collecte et envoi des logs à Loki | v3.5.8 |
| Trivy | Analyse de vulnérabilités | v0.68.2 |
| Semgrep | Analyse statique de sécurité | v1.65.0 |
| Ruff | Linter Python | v0.1.14 |
| ESLint | Linter JavaScript/TypeScript | v8.56.0 |
| Coverage | Mesure de couverture de tests | v4.0.7 |
| Vitest | Framework de test pour Vite/JS | v1.2.0 |
| DuckDNS | Service de DNS dynamique | Service web (pas de version) |
| Certbot | Gestion des certificats TLS | v1.7.1 |
| Visual Studio Code | Editeur de code | V1.107 |

## 3.5. Résultats attendus

- Une infrastructure DevOps multi-environnement opérationnelle sur un VPS

- Un pipeline CI/CD automatisé pour accéléré le la livraison du logiciel
- Une supervision complète via les outils de monitoring
- Une stratégie de déploiement Blue/Green avec rollback possible
- Une gestion centralisée des logs et des métriques
- Une sécurité renforcée grâce à l'intégration des outils et pratiques DevSecOps

## 3.6. Chronogramme des travaux

*Tableau 7: chronogramme de travaux*

| Activités | Septembre | | | | Octobre | | | | Novembre | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
| Prise de connaissance du projet, analyse de l'existant, définition des besoins | | | | | | | | | | | | |
| Étude des outils DevOps, choix des technologies, préparation de l'environnement local | | | | | | | | | | | | |
| Mise en place de la conteneurisation avec Docker et Docker Compose | | | | | | | | | | | | |
| Intégration de GitHub Actions, création des workflows CI/CD | | | | | | | | | | | | |
| Déploiement sur le VPS, configuration de la supervision (Prometheus, Grafana, Loki) | | | | | | | | | | | | |
| Tests finaux, documentation technique, revue de sécurité | | | | | | | | | | | | |

# PARTIE II: ANALYSE ET CONCEPTION

# chapitre 4: Analyse préalable

## 4.1. Analyse de l'existant

Toute évolution, optimisation ou transformation d'un système informatique dans une démarche DevOps nécessite une compréhension approfondie de l'environnement technique existant. L'analyse de l'existant vise à dresser un état des lieux précis de l'architecture logicielle et des processus de développement, d'intégration, de déploiement et de supervision en place. Elle permet d'évaluer le niveau d'automatisation, la performance des outils utilisés, la fiabilité des pipelines CI/CD, ainsi que la capacité de l'infrastructure à répondre aux exigences de disponibilité, de scalabilité et de sécurité. Cette étape est essentielle pour identifier les axes d'amélioration et proposer des solutions adaptées aux besoins de l'organisation dans une logique d'industrialisation et de modernisation des pratiques DevOps.

### 4.1.1. Topologie réseau

La figure 5 montre la topologie réseau de la JIRAMA.

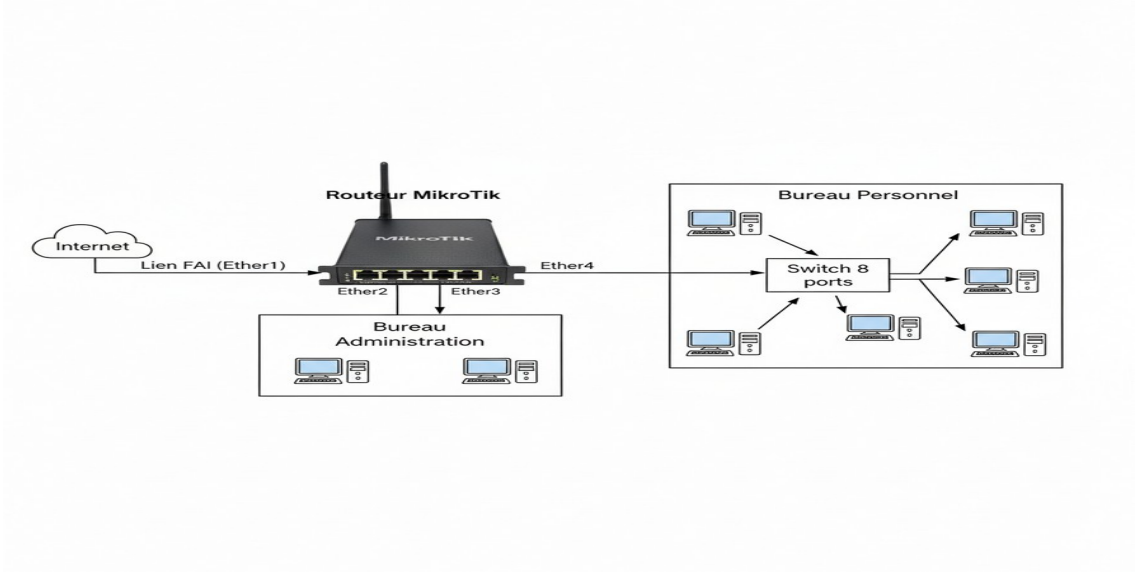



*Figure 5: topologie réseau de la JIRAMA*

### 4.1.2. Connexion internet

La JIRAMA n'a qu'une seule connexion internet. Leur fournisseur d'accès internet (FAI) est l'opérateur TELMA.

Le tableau 8 représente la liste de connexion internet.

*Tableau 8: liste de connexion internet*

| Connexion | Technologie | Vitesse | Redondance |
|---|---|---|---|
| TELMA | Fibre optique | 100 Mbps | Non |

**Remarque :**

L'Internet est très important au sein de JIRAMA , pour les raisons suivantes:

- L'existence de ses applications Web localement,
- Afin d'interagir avec les clients, les fournisseurs et l'environnement environnant

### 4.1.3. Passerelle

Le tableau 9 représente la liste de passerelle.

Tableau 9: liste de passerelle

| Désignation | Caractéristiques | Système d'exploitation | Fonctionnalités |
|---|---|---|---|
| MikroTik | 5 ports Ethernet;<br>CPU : coré2duo ;<br>RAM : 512 Mb | RouteurOS | Routage |

### 4.1.4. Postes de travails

Les ordinateurs d'utilisateur qui se répartissent dans tous les personnels, sont généralement les suivants :

Le tableau 10 représente la liste de poste de travail.

*Tableau 10: liste de poste de travail*

| **Désignations** | **Caractéristique** | **Système d'exploitation** | **Méthode d'Authentification** | **Nombre** |
|---|---|---|---|---|
| Bureau | Pentium 4, Dual Core<br>RAM : 512Mo à 4Go<br>HDD : 60Go à 500Go | Windows 7 et 10 | Local | 7 |
| Portable | Dual Core, Core i5<br>RAM : 8Go à HDD : 500Go | Windows 11 | Local | 1 |

### 4.1.5. Analyse du trafic réseau

Ces études ont été effectuées avec l'utilisation du logiciel « Wireshark version v4.4.12 »

Pour obtenir le graphe correspondant à chaque protocole il faut allez dans le menu «**Statistique** » choisir « **Graphique I/O** » et en tapant le champ « **Display Filter** » le protocole de votre choix.

Les images suivantes montrent le trafic réseau sur une machine d'un employé

La figure 6 représente la Graphe de la bande passante générale d'un employé




*Figure 6: Graphe de la bande passante générale d'un employé*

La figure 7 représente la Graphe de la protocole http d'un employé

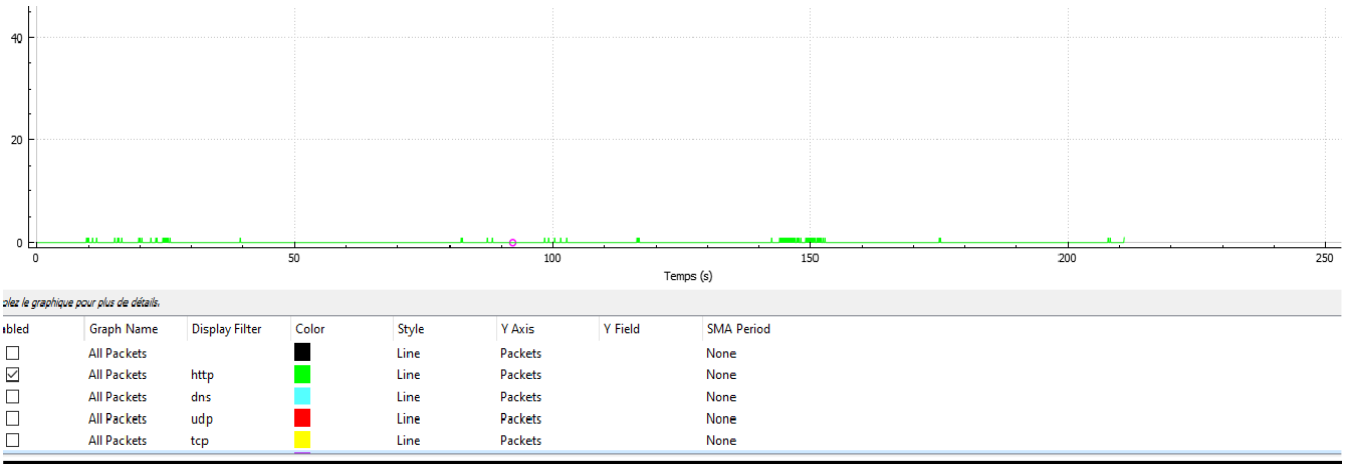



*Figure 7: Graphe de la protocole http d'un employé*

### 4.1.6. Surveillance

Aucun système de supervision des applications et des serveurs n'a été constaté au sein de l'infrastructure informatique de la JIRAMA. Aucun outil n'est actuellement en place pour assurer le suivi en temps réel de l'état des services déployés, ni pour surveiller les ressources critiques telles que l'utilisation du processeur, de la mémoire vive (RAM) ou de l'espace disque. Cette absence de supervision rend difficile la détection proactive des anomalies, la mesure des performances applicatives, ainsi que la mise en place d'un système d'alerting en cas de défaillance.

### 4.1.7. Test et sécurité

L'environnement informatique actuel de la JIRAMA ne dispose d'aucune stratégie formelle de test ni de politique de sécurité intégrée. Aucun mécanisme de validation automatisée n'est mis en œuvre pour garantir la fiabilité des applications internes, et aucun outil de test unitaire, d'intégration ou d'analyse statique du code n'est utilisé dans le cycle de développement. Cette absence de contrôle expose les applications à des erreurs non détectées et à des vulnérabilités potentielles.

Sur le plan de la sécurité réseau, l'infrastructure repose uniquement sur le pare-feu intégré au routeur, sans solution complémentaire de filtrage, de détection d'intrusion ou de journalisation centralisée. Cette approche minimaliste, bien que fonctionnelle, présente des risques importants, notamment en cas de faille non corrigée ou de mauvaise configuration du routeur.

Dans ce contexte, l'absence de dispositifs de test et de sécurité à l'échelle de l'environnement global incluant les serveurs, les applications et le réseau constitue une faiblesse majeure.

### 4.1.8. Gestion de code source

Le code source est hébergé sur GitHub, avec une organisation simple en branches (main, develop). Cependant, aucune politique de gestion des branches, de revue de code ou de tests automatisés n'était en place. Les commits étaient poussés directement, sans pipeline d'intégration continue ni vérification de la qualité ou de la sécurité du code.

### 4.1.9. Déploiement et hébergement de l' application

Les services applicatifs de la JIRAMA  ont été initialement déployés et hébergés à l'aide de plateformes PaaS gratuites, choisies par les développeurs pour leur simplicité d'utilisation et leur accessibilité.

- **Vercel** a été utilisé pour le déploiement du frontend React.js
- **Render** pour le backend Django
- **GitHub** pour l'hébergement du code source

Bien que les outils utilisés aient facilité un déploiement rapide et sans frais, leur usage s'accompagne de plusieurs limitations notables. L'environnement d'exécution reste peu maîtrisé, la supervision des services est inexistante, la gestion des secrets est rudimentaire, et les stratégies de déploiement manquent de flexibilité. De plus, ces solutions n'offrent aucune garantie en matière de performance, de disponibilité ou de personnalisation, ce qui limite leur pertinence dans un contexte de production.

## 4.2. Critique de l’existant

### 4.2.1. Point fort

L’environnement existant présente certains avantages, notamment la simplicité de mise en œuvre des plateformes PaaS (telles que Vercel et Render) et leur accessibilité gratuite, qui ont facilité un déploiement rapide lors des phases de prototypage. De même, l’infrastructure réseau repose sur une configuration minimale, suffisante pour un usage interne de base. Toutefois, une analyse approfondie révèle plusieurs limites techniques et organisationnelles qui freinent la montée en charge, la sécurité et la maintenabilité du système. Ces aspects seront détaillés dans la section suivante.

### 4.2.2. Point faible

*Tableau 11: Critique de l'existant De l’architecture actuel de la JIRAMA*

| **Problème** | **Description** | **Conséquences** |
|---|---|---|
| Contrôle limité | Dépendance aux outils PaaS | Difficulté à personnaliser l’infrastructure |
| Sécurité faible | Secrets dispersés, pas de chiffrement | Risques de fuites ou d’accès non autorisés |
| Supervision absente | Pas de métriques ni d’alertes | Détection tardive des incidents |
| Déploiement non reproductible | Pas de versionnement des environnements | Risques d’erreurs et de régressions |
| Manque d’automatisation d’action fréquent | les test et scans de sécurité sont manuels et parfois oublié par les développeurs | L’application bug vite,insatisfaction des clients |
| Pas de stratégie de déploiement | Mises à jour en place | Downtime possible, rollback difficile |
| Non-uniformité des environnements | Le développement est effectué localement sans conteneurisation. Utilisation de systèmes d'exploitation et de versions de dépendances (Python, Node.js) potentiellement différents entre les développeurs et la production | Problèmes "Ça marche sur ma machine" : Déploiement non reproductible ; Différences de comportement entre l'environnement de développement, de test et de production ; Bugs difficiles à |

| | | |
|---|---|---|
| | | diagnostiquer. |
| Coût imprévisible | Les plans gratuits de PaaS ont des limites strictes (heures de calcul, bande passante) | Risque de dépassement avec facturation surprise ou coupure de service si l'application sera fortement utilisé par le public |

### 4.3. Proposition de solution

Afin de répondre aux limites identifiées dans l'analyse critique de l'existant et de garantir une infrastructure fiable, évolutive et maîtrisée, une approche DevOps complète est proposée. Cette démarche repose sur trois piliers fondamentaux : la **maîtrise technique** (uniformisation des environnements, orchestration), l'**automatisation** (intégration et déploiement continus), et l'**observabilité** (supervision, alerting, traçabilité). L'objectif est de transformer un environnement de développement expérimental en une infrastructure de production robuste, cohérente et adaptée aux exigences d'un service critique.

Les solutions proposées sont les suivantes :

- **S1** : Mettre en place d'un mécanisme de conteneurisation afin d'uniformiser les environnements de développement, de test et de production, et d'éliminer les écarts liés aux différences de configuration.
- **S2** : Mettre en place une solution d'orchestration des services permettant de gérer efficacement les services, leurs dépendances et leur cycle de vie sur une infrastructure dédiée.
- **S3** : Intégré un pipeline d'automatisation CI/CD pour assurer le déclenchement systématique des tests, des analyses de sécurité et des déploiements à chaque mise à jour du code.
- **S4** : Mettre en œuvre un mécanisme de routage intelligent et de stratégie de déploiement progressive (type Blue/Green) afin de réduire les interruptions de service et de faciliter les retours en arrière en cas d'échec.
- **S5** : Intégré un système de supervision centralisé pour collecter, visualiser et analyser les métriques système et applicatives, et déclencher des alertes en cas d'anomalie.
- **S6** : Renforcé la sécurité à travers l'automatisation des tests de vulnérabilités, l'analyse statique du code, et la gestion centralisée des secrets.

- **S7** : Mettre en place un server proxy et mécanisme de gestion de nom de domaine et de certificats TLS afin de sécuriser les communications HTTP et d'assurer l'authenticité et la sécurité des services exposés.

## 4.4. Solution retenue

À la suite de l'analyse critique de l'existant, on a retenue toutes les solutions proposés pour répondre aux exigences techniques, pédagogiques et opérationnelles de la Jirama. Cette solution vise à remplacer l'approche actuelle basée sur des plateformes PaaS par une infrastructure maîtrisée, déployée sur un environnement dédié.

L'approche retenue repose sur les principes suivants :

- **Uniformisation des environnements** : en assurant une cohérence entre les environnements de développement, de test et de production, afin de limiter les écarts de comportement et les erreurs liées aux différences de configuration.
- **Orchestration centralisée** : pour faciliter la gestion des services, leur déploiement, leur mise à l'échelle et leur supervision, tout en assurant leur isolation et leur interconnexion.
- **Automatisation des processus** : en intégrant des mécanismes de déploiement continu, de tests automatisés et de contrôles qualité à chaque mise à jour du code.
- **Supervision et observabilité** : pour permettre un suivi en temps réel de l'état des services, des performances applicatives et des ressources système, avec des alertes en cas d'anomalie.
- **Renforcement de la sécurité** : par l'intégration de contrôles automatisés, la centralisation des secrets, et la sécurisation des communications réseau.
- **Maîtrise des coûts et de l'infrastructure** : en optant pour une solution auto-hébergée, permettant un contrôle total sur l'environnement, une meilleure prévisibilité budgétaire, et une adaptation fine aux besoins du projet.

Cette solution permet de construire une infrastructure DevOps robuste, évolutive et conforme aux standards professionnels, tout en offrant un cadre d'apprentissage concret et complet.

# chapitre 5: Choix des outils

## 5.1. Conteneurisation

La conteneurisation est une méthode de virtualisation logicielle qui regroupe une application avec ses dépendances (bibliothèques, fichiers de configuration) dans une unité portable et isolée appelée conteneur, permettant de la faire fonctionner de manière identique sur n'importe quel environnement, du poste de développement au cloud. Contrairement aux machines virtuelles, les conteneurs partagent le noyau du système d'exploitation hôte, les rendant plus légers, plus rapides et plus efficaces en ressources,ce qui les rend essentiels pour les architectures modernes comme les microservices [8]. Newman explique que la conteneurisation facilite le déploiement indépendant de chaque service, améliorant ainsi la résilience globale du système [8].

### 5.1.1. Comparaison des principaux types de conteneurs

*Tableau 12: tableau comparatif des principaux types de conteneurs*

| **Conteneur** | **Caractéristiques** | **Utilisation** |
|---|---|---|
| **Docker** | - Conteneurisation applicative légère<br>- Utilise un moteur client-serveur<br>- Large écosystème (Docker Hub, Compose, Swarm)<br>- Compatible avec la plupart des outils CI/CD<br>- Supporte les volumes, réseaux, secrets, etc. | - Développement et déploiement d'applications web et microservices<br>- Automatisation des environnements de test<br>- Intégration dans les pipelines DevOps |
| **LXC (Linux Containers)** | - Conteneurs système proches de la virtualisation<br>- Fournissent un environnement complet avec init system<br>- Moins orientés DevOps, plus proches des machines virtuelles | - Simulations de systèmes complets<br>- Environnements Linux isolés pour tests ou sandboxing<br>- Moins utilisé dans les chaînes CI/CD modernes |
| **Podman** | - Alternative à Docker sans démon (daemonless)<br>- Exécution rootless par défaut (meilleure sécurité)<br>- Compatible avec les images Docker<br>- CLI similaire à Docker | - Déploiements sécurisés sur serveurs Linux<br>- Environnements nécessitant une exécution sans privilèges<br>- Remplacement de Docker dans certains contextes |
| **rkt (Rocket)** | - Développé par CoreOS (aujourd'hui | - Anciennement utilisé dans les |

| Conteneur | Caractéristiques | Utilisation |
|---|---|---|
| | abandonné)<br>- Conçu pour la sécurité et l'intégration avec systemd<br>- Architecture orientée pods (comme Kubernetes) | environnements CoreOS<br>- Remplacé progressivement par Docker ou Podman<br>- Peu utilisé aujourd'hui |
| **Conteneurs Windows** | - Conteneurs spécifiques à l'écosystème Windows<br>- Deux modes : Windows Server Containers et Hyper-V Containers<br>- Intégration avec IIS, .NET, PowerShell | - Déploiement d'applications Windows legacy<br>- Conteneurisation de services .NET Framework<br>- Moins portable que les conteneurs Linux |

### 5.1.2. Choix et justification

Parmi les solutions comparées, Docker s'impose comme le choix le plus pertinent pour ce projet DevOps. Il se distingue par sa simplicité d'utilisation, sa compatibilité étendue avec les outils CI/CD, et son écosystème riche (Docker Hub, Docker Compose, Docker Swarm), qui en font une solution idéale pour la conteneurisation d'applications web et microservices.

Docker garantit une portabilité optimale, assurant un comportement identique de l'application sur tous les environnements (développement, test, production). Son format d'image standardisé favorise la standardisation des déploiements, tandis que son intégration fluide avec les pipelines CI/CD (notamment GitHub Actions) facilite l'automatisation des processus DevOps. De plus, Docker bénéficie d'une large communauté, d'une documentation abondante et de nombreux tutoriels, ce qui accélère la prise en main et la résolution de problèmes. Enfin, sa scalabilité permet de déployer facilement plusieurs instances d'un service pour répondre à une montée en charge, tout en restant adaptable à des architectures plus complexes si nécessaire.

### 5.1.3. Présentation de Docker

Docker est une plateforme open-source permettant de créer, déployer et exécuter des applications dans des conteneurs. Elle repose sur une architecture client-serveur, où le Docker Engine gère les conteneurs via des commandes CLI ou des fichiers de configuration (Dockerfile, docker-compose.yml). Docker permet une encapsulation complète de l'application, assurant sa portabilité, sa scalabilité, et sa réplicabilité dans tous les environnements.

Dans le cadre de ce projet, Docker est utilisé pour conteneuriser les services backend , frontend, et la base de données, facilitant ainsi le déploiement sur le VPS et l'intégration avec les outils de supervision et de sécurité.

La figure 8 montre l'architecture de docker

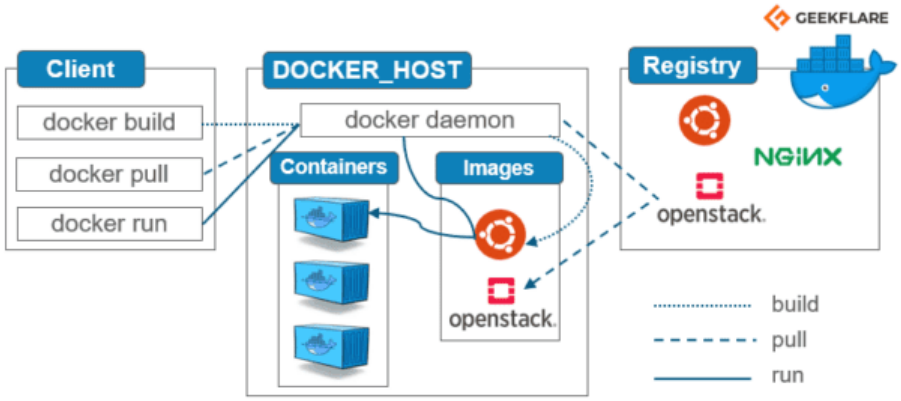



*Figure 8: architecture de docker*

## 5.2. systèmes de gestion de code source

Un système de gestion de code source est un outil permettant de suivre, organiser et contrôler les modifications apportées au code d'un projet. Il facilite la collaboration entre développeurs, conserve l'historique des versions, permet de revenir à un état antérieur en cas d'erreur, et assure la traçabilité des contributions. Dans un contexte DevOps, ces systèmes jouent un rôle central dans l'automatisation des déploiements, la gestion des branches, et l'intégration continue.

### 5.2.1. Comparaison des systèmes de gestion de code source

*Tableau 13 :Tableau comparatif des systèmes de codes sources*

| **Fonctionnalité** | **GitHub** | **GitLab** | **Bitbucket** |
|---|---|---|---|
| **Popularité** | Très élevée : plateforme la plus utilisée au monde, notamment dans les projets open source | Élevée : très utilisée dans les entreprises et les projets privés | Moyenne : principalement utilisée dans l'écosystème Atlassian |
| **Intégration CI/CD** | Native via GitHub Actions, avec une marketplace riche d'actions prêtes à l'emploi | Native via GitLab CI/CD, très puissante et intégrée dans l'interface | Intégration possible via Bitbucket Pipelines, mais moins flexible |
| **Interface utilisateur** | Intuitive, épurée, facile à prendre en main pour les débutants | Complète, avec de nombreuses fonctionnalités intégrées (CI, gestion de projet, etc.) | Moyenne, parfois jugée moins ergonomique que ses concurrents |
| **Communauté et support** | Très large communauté mondiale, documentation abondante, nombreux | Communauté active, surtout dans les milieux | Communauté plus restreinte, mais bien intégrée à l'écosystème |

| Fonctionnalité | GitHub | GitLab | Bitbucket |
|---|---|---|---|
| | tutoriels | professionnels | Jira/Confluence |
| **Fonctionnalités collaboratives** | Pull requests, discussions, gestion fine des permissions, GitHub Projects | Merge requests, gestion de tickets, tableaux de bord intégrés | Pull requests, intégration native avec Jira pour la gestion de projet |
| **Modèle d'hébergement** | SaaS (cloud GitHub), GitHub Enterprise pour auto-hébergement | SaaS ou auto-hébergé (GitLab CE/EE) | SaaS ou auto-hébergé (Bitbucket Server) |
| **Sécurité** | Gestion des secrets, scan automatique des vulnérabilités, SSO, 2FA | Contrôle d'accès avancé, audit, intégration DevSecOps | Contrôle d'accès, intégration avec les outils Atlassian pour la sécurité |

### 5.2.2. Choix et justification

Parmi les solutions comparées, GitHub s'impose comme la plateforme de gestion de code source la plus adaptée pour ce projet DevOps. Elle se distingue par sa simplicité d'utilisation, son intégration native avec GitHub Actions, et sa large communauté de développeurs. Ces atouts en font une solution complète, cohérente et parfaitement alignée avec les besoins d'un projet moderne orienté DevOps.

GitHub offre une portabilité et une accessibilité optimales, permettant aux membres de l'équipe de collaborer efficacement depuis n'importe quel environnement. Grâce à GitHub Actions, il est possible d'automatiser l'ensemble du cycle de vie du développement logiciel : tests, analyses de sécurité, intégration, et déploiement. La plateforme propose également des fonctionnalités collaboratives avancées telles que les pull requests, la gestion des issues, les revues de code, et les projets intégrés. Enfin, GitHub bénéficie d'une documentation riche, d'une communauté très active, et d'un hébergement fiable et sécurisé, aussi bien pour les projets open source que pour les projets privés.

### 5.2.3. Présentation de Github

GitHub est une plateforme complète de développement collaboratif basée sur Git [13]. Selon la documentation officielle de GitHub, cette plateforme permet d'héberger du code source, de gérer les versions, de collaborer via des pull requests, et d'automatiser les workflows grâce à GitHub Actions [13], ce qui en fait un pilier incontournable dans les environnements DevOps modernes.

Dans le cadre de ce projet, GitHub est utilisé pour :

- Centraliser le code source des différents services (backend, frontend, infrastructure)

- Gérer les branches et les contributions via des pull requests
- Automatiser les tests, les analyses de sécurité et les déploiements via GitHub Actions
- Assurer la traçabilité et la collaboration entre les membres de l'équipe

## 5.3. Système d'intégration et de déploiement continus(CI/CD)

L'intégration continue (CI) consiste à tester automatiquement chaque modification du code [1]. Fowler définit la CI comme une pratique de développement où les membres d'une équipe intègrent fréquemment leur travail, généralement plusieurs fois par jour, chaque intégration étant vérifiée par un build automatisé [1]. Le déploiement continu (CD) étend ce concept en livrant automatiquement les nouvelles versions en production après validation [2].

### 5.3.1. Comparaison des outils CI/CD

*Tableau 14:Comparaison des outils CI/CD*

| **Fonctionnalité** | **GitHub Actions** | **GitLab CI/CD** | **Jenkins** |
|---|---|---|---|
| **Intégration Git** | Native avec GitHub | Native avec GitLab | Externe, nécessite des configurations |
| **Configuration des pipelines** | Fichiers YAML simples dans .github/workflows/ | Fichiers .gitlab-ci.yml à la racine du projet | Interface graphique + scripts Groovy |
| **Déclencheurs** | Push, pull request, tags, cron, manuels | Push, merge request, tags, manuels | Très flexible, mais configuration plus complexe |
| **Marketplace d'actions** | Très riche (plusieurs milliers d'actions) | Moins fournie | Plugins nombreux mais parfois obsolètes |
| **Gestion des secrets** | GitHub Secrets, permissions granulaires | Variables CI/CD, masquage automatique | Plugins ou intégration Vault |
| **Coût** | Gratuit pour projets publics, quotas généreux | Gratuit pour projets publics, runners limités | Nécessite serveur dédié ou cloud |
| **Maintenance** | Aucune (SaaS géré par GitHub) | Faible à moyenne selon l'hébergement | Élevée (mise à jour, sécurité, plugins) |
| **Courbe d'apprentissage** | Faible, documentation claire | Moyenne | Élevée, nécessite expertise DevOps |
| **Observabilité** | Logs détaillés, visualisation des jobs, artefacts | Interface CI intégrée, pipelines visuels | Interface personnalisable, mais plus technique |

### 5.3.2. Choix et justification

Parmi les outils comparés, GitHub Actions a été retenu comme solution CI/CD principale pour ce projet. Son intégration native avec GitHub permet de déclencher automatiquement les workflows à chaque événement du dépôt (push, pull request, tag, etc.) sans configuration supplémentaire. Sa syntaxe simple et lisible en YAML facilite la prise en main, même pour les débutants, et sa marketplace riche propose des milliers d'actions prêtes à l'emploi pour automatiser les tests, les analyses de sécurité, les builds et les déploiements.

GitHub Actions offre également une gestion sécurisée des secrets via GitHub Secrets, une exécution gratuite pour les projets publics, et ne nécessite aucune infrastructure à maintenir, contrairement à Jenkins. Il s'intègre parfaitement avec les outils utilisés dans ce projet, notamment Docker, Docker Compose, Trivy, Semgrep, Ruff, ce qui en fait une solution cohérente, efficace et parfaitement adaptée à une architecture DevOps moderne.

### 5.3.3. Présentation de GitHub Actions

GitHub Actions est une plateforme d'automatisation des workflows CI/CD intégrée à GitHub. Elle permet de définir des pipelines sous forme de fichiers YAML, stockés dans le répertoire .github/workflows/ du dépôt. Chaque workflow est déclenché par un ou plusieurs événements (push, pull request, tag, cron, etc.) et peut contenir plusieurs jobs exécutés en parallèle ou en séquence.

**Fonctionnalités principales :**

- Déclenchement automatique à chaque événement Git
- Exécution de jobs sur des runners GitHub ou auto-hébergés
- Marketplace d'actions réutilisables (build, test, déploiement, sécurité…)
- Gestion des secrets via GitHub Secrets
- Support natif de Docker, Node.js, Python er de beaucoups de technologie.
- Logs détaillés, artefacts téléchargeables, visualisation claire des étapes

La figure 9 montre le fonctionnement de github actions.

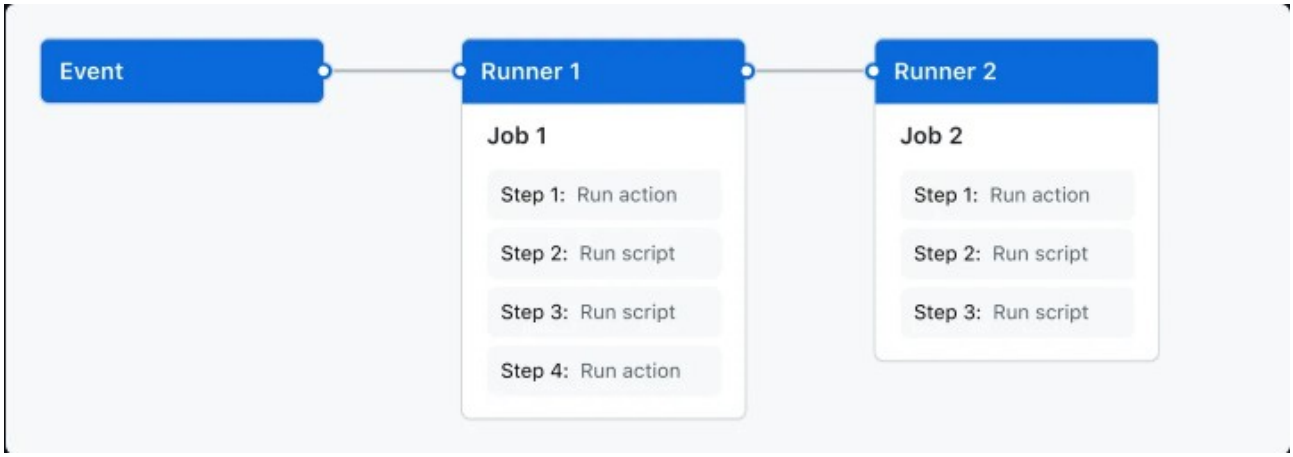



*Figure 9: fonctionnement de github actions*

## 5.4. Orchestration de conteneurs

L'orchestration de conteneurs désigne l'ensemble des mécanismes permettant de gérer le cycle de vie des conteneurs dans un environnement de production. Cela inclut le démarrage, l'arrêt, la mise à l'échelle, la mise à jour, la surveillance, la gestion réseau, la résilience, et la coordination entre plusieurs services conteneurisés. L'orchestrateur agit comme un chef d'orchestre qui garantit que chaque service fonctionne correctement, communique avec les autres, et peut être redémarré ou remplacé automatiquement en cas de défaillance.

### 5.4.1. Comparaison des principaux orchestrateurs de conteneurs

*Tableau 15: comparaison des outils d'orchestration de conteneur*

| **Caractéristique** | **Docker Compose** | **Kubernetes** | **Docker Swarm** |
|---|---|---|---|
| **Complexité** | Très faible : un seul fichier YAML, pas de composants externes | Très élevée : nécessite plusieurs composants (API server, etcd, scheduler, etc.) | Moyenne : plus simple que Kubernetes mais nécessite une configuration réseau |
| **Cible** | Mono-hôte (VPS, local) | Clusters multi-nœuds, haute disponibilité | Multi-hôte, petits clusters |
| **Haute disponibilité (HA)** | Non native, possible via des scripts ou outils externes | Native, avec redondance des pods et des nœuds | Basique, via réplication de services |
| **Mise à l'échelle** | Manuelle (via docker-compose up --scale) | Automatique (Horizontal Pod Autoscaler) | Manuelle ou semi-automatique |
| **Déploiement Blue/Green** | Possible via Nginx et deux stacks | Intégré via services et labels | Possible avec des labels et services |

| | | | |
|---|---|---|---|
| **Courbe d'apprentissage** | Rapide, adapté aux débutants | Longue, nécessite des compétences avancées | Moyenne, plus accessible que Kubernetes |
| **Maintenance** | Faible, peu de dépendances | Élevée, nécessite surveillance des composants | Moyenne |
| **Communauté et support** | Large, bien documentée | Très large, soutenue par CNCF | Moyenne, en déclin depuis l'essor de Kubernetes |

### 5.4.2. Choix et justification

Dans le cadre de ce projet, Docker Compose a été retenu comme outil d'orchestration de conteneurs. Ce choix s'explique par sa simplicité de mise en œuvre, sa courbe d'apprentissage rapide, et son adéquation parfaite avec un environnement mono-hôte tel qu'un VPS. Contrairement à Kubernetes, qui cible des architectures distribuées complexes, Compose permet de gérer efficacement plusieurs services conteneurisés à l'aide d'un seul fichier docker-compose.yml, ce qui réduit considérablement la complexité opérationnelle.

Docker Compose offre une centralisation de la configuration, une intégration fluide avec les pipelines CI/CD (notamment GitHub Actions), et une compatibilité directe avec les outils de supervision comme Prometheus, Grafana et Loki. Il permet également de simuler des stratégies de déploiement avancées telles que le Blue/Green, en combinant des stacks nommées et un reverse proxy comme Nginx. Enfin, sa légèreté, son faible besoin en maintenance, et sa capacité à évoluer vers Docker Swarm ou Kubernetes en cas de montée en charge en font une solution pragmatique, efficace et évolutive pour les besoins de ce projet.

### 5.4.3. Présentation de Docker compose

Docker Compose est un outil d'orchestration mono-hôte développé par Docker Inc. [12]. Selon la documentation officielle de Docker, Compose permet de définir, configurer et exécuter plusieurs conteneurs à l'aide d'un fichier YAML (docker-compose.yml), simplifiant ainsi la gestion d'applications multi-services [12].Chaque service (application, base de données, reverse proxy, etc.) est décrit dans ce fichier avec ses ports, volumes, variables d'environnement, dépendances et réseaux.

#### a. Fonctionnement

- Définition des services : chaque conteneur est décrit avec son image, ses ports, ses volumes, ses variables d'environnement,etc.
- Commandes simples : docker-compose up, down, logs, ps, restart, etc.
- Support des réseaux personnalisés : permet de créer des réseaux isolés entre services

- Gestion des volumes : persistance des données entre les redémarrages
- Compatibilité CI/CD : facilement intégrable dans les pipelines GitHub Actions

### b. Avantages dans un contexte mono-hôte

- Déploiement rapide sur un VPS sans configuration complexe
- Centralisation de la configuration dans un seul fichier versionné
- Facilité de rollback en cas d'échec grâce à la stratégie Blue/Green
- Adapté aux petites équipes et aux environnements à ressources limitées
- Évolutif : possibilité de migrer vers Swarm ou Kubernetes si nécessaire en cas d'évolution d'infrastructure

La figure 10 illustre l'architecture de docker compose

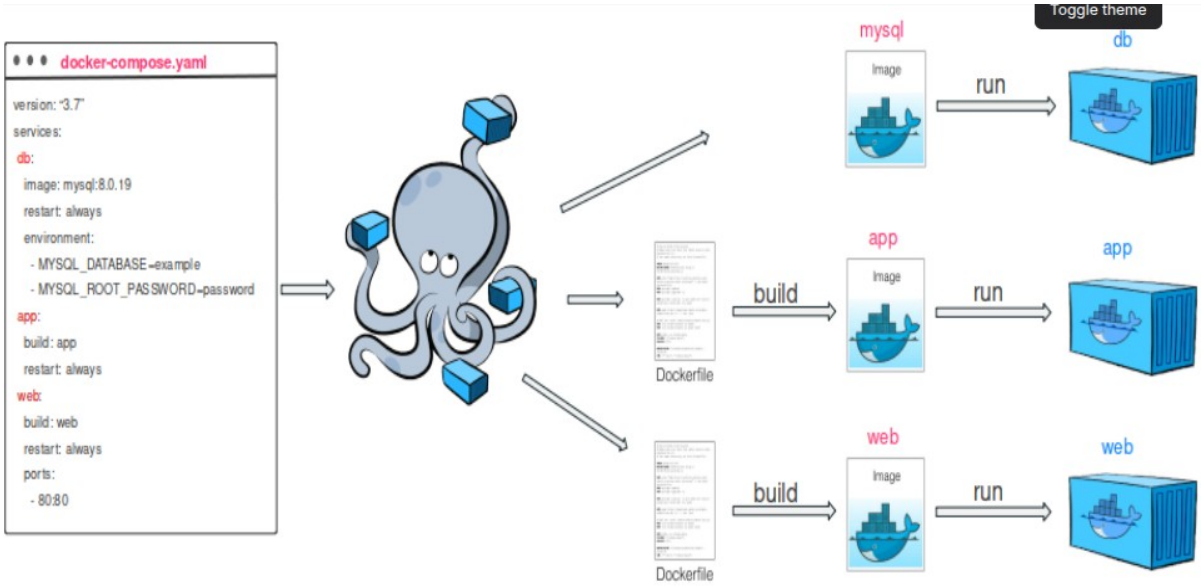



*Figure 10: Architecture de docker compose*

## 5.5. Surveillance et monitoring

La supervision consiste à observer en continu les performances, la disponibilité et les erreurs d'un système pour détecter rapidement les incidents, comprendre leurs causes et optimiser le fonctionnement global. Elle s'appuie sur trois piliers complémentaires: métriques, logs et alertes.

### 5.5.1. Comparaison des outils de supervision et d'observabilité

*Tableau 16: Tableau comparatif des outils de monitoring*

| **Fonctionnalité** | **Prometheus & Grafana (choix)** | **Datadog** | **Zabbix** | **ELK (Elasticsearch, Logstash, Kibana)** |
|---|---|---|---|---|
| Licence | Open source | SaaS (propriétaire) | Open source | Open source |
| Intégration Docker | Excellente (exporters, service discovery) | Excellente | Moyenne | Bonne (logs surtout) |
| Focus | Métriques + alertes + dashboards | Métriques + logs + traces (SaaS) | Métriques + SNMP | Logs centralisés + recherche |
| Visualisation | Grafana (dashboards, alerting) | Intégrée | Interface Zabbix | Kibana (dashboards de logs) |
| Coût | Gratuit | Payant (à l'usage) | Gratuit | Gratuit (coûts d'infra possibles) |
| Mise en œuvre | Légère, rapide sur VPS | Simple mais dépend du SaaS | Plus classique, configuration manuelle | Plus lourde en ressources |
| Cas d'usage idéal | Monitoring Docker/K8s, microservices | Entreprise avec budget SaaS | Infrastructures traditionnelles (SNMP) | Analyse de logs à grande échelle |

### 5.5.2. Choix et justification

Pour la supervision du projet, la combinaison Prometheus et Grafana a été retenue comme solution principale. Cette stack open source se distingue par sa légèreté, sa modularité et sa parfaite compatibilité avec les environnements conteneurisés, notamment sur un VPS mono-hôte. Elle permet de collecter, stocker et visualiser des métriques système et applicatives en temps réel, tout en offrant des fonctionnalités avancées de tableaux de bord personnalisés et d'alerting configurable.

Prometheus s'intègre nativement avec de nombreux exporters (Node Exporter, PostgreSQL Exporter,etc), facilitant la collecte de données sur les services, les bases de données et les conteneurs. Grafana, quant à lui, permet de visualiser ces données de manière claire et interactive, avec des dashboards adaptables aux besoins du projet.

Comparativement à d'autres solutions :

- ELK Stack est plus adapté à la gestion des logs, mais reste lourd en ressources pour un petit environnement.
- Datadog est une solution puissante et complète, mais payante, ce qui peut être un frein dans un contexte à budget limité.
- Zabbix reste pertinent pour des infrastructures classiques, mais moins adapté aux environnements conteneurisés modernes.

Ainsi, Prometheus et Grafana offrent un équilibre optimal entre performance, simplicité, extensibilité et coût, ce qui en fait une solution de choix pour la supervision dans une architecture DevOps légère et conteneurisée.

### 5.5.3. Présentation de Prometheus

Prometheus est un système de monitoring et d'alerte open source conçu pour la fiabilité [14]. Selon la documentation officielle de Prometheus, il collecte et stocke des métriques sous forme de séries temporelles avec des labels pour un filtrage multi-dimensionnel, permettant une surveillance en temps réel des applications et infrastructures [14].

La figure 11 illustre l'architecture de prometheus et certains de ses composants d'écosystème

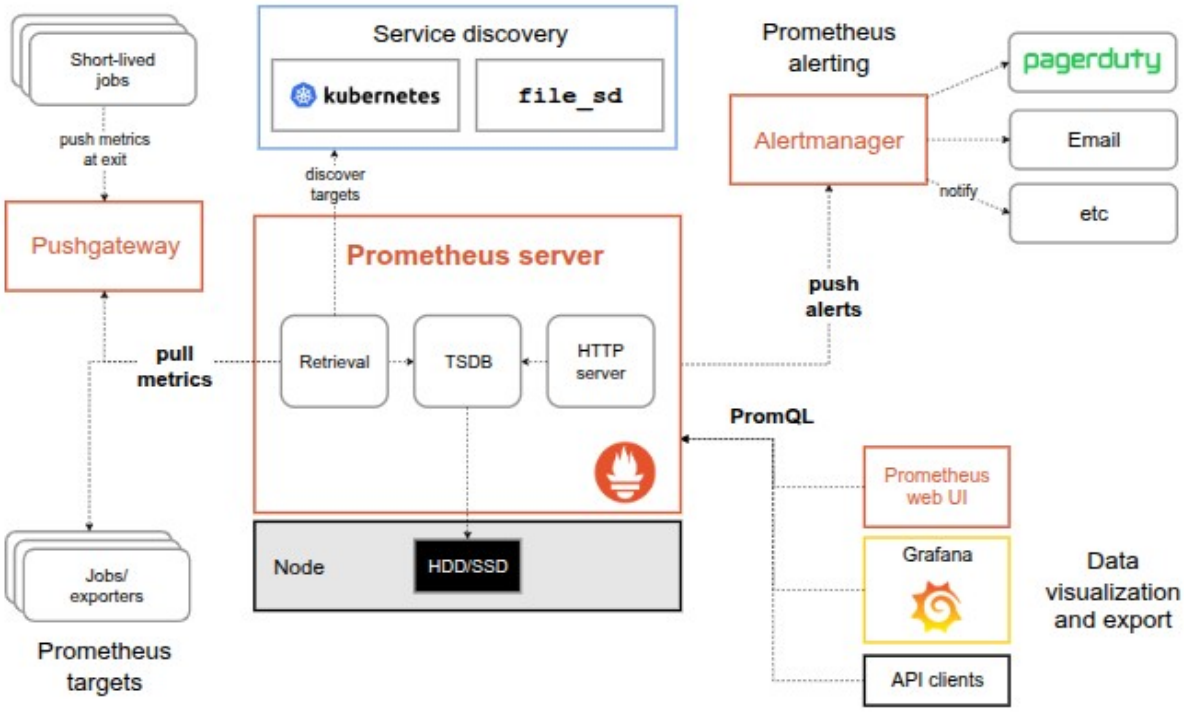



*Figure 11: architecture de prometheus*

#### a. Fonctionnement

Prometheus scrape (récupère) périodiquement les métriques exposées par les applications via HTTP. Il les stocke dans sa base de données TSDB optimisée, puis les rend disponibles pour interrogation via PromQL. En cas d'anomalie détectée, Alertmanager envoie des notifications.

#### b. Caractéristiques Principales

- **Modèle PULL** : Récupération active des métriques
- **PromQL** : Langage de requête puissant pour les séries temporelles
- **Multi-dimensionnel** : Métriques avec labels pour un filtrage précis
- **Intégration Docker** : Découverte automatique des services
- **Alertmanager** : Système d'alerte intégré avec regroupement intelligent
- **Exporters ecosystem** : +100 intégrations disponibles
- **Stockage local efficace** : TSDB compressée et performante

### 5.5.4. Présentation de Grafana

Grafana est une plateforme de visualisation et d'alerting multi-sources [15]. D'après la documentation Grafana Labs, cette solution se connecte à plus de 40 sources de données différentes, dont Prometheus et Loki, pour afficher des métriques en temps réel et définir des alertes personnalisées [15].

#### a. Fonctionnement

Grafana collabore avec des outils comme :

- Loki: Système de logs inspiré de Prometheus; indexe par labels, stocke les flux de logs à coût réduit.
  → Travail avec Grafana: Grafana interroge Loki via LogQL pour requêter et visualiser les logs; corrélation métriques/logs dans un même dashboard.
- Promtail: Agent de collecte de logs pour Loki; lit les logs des conteneurs Docker (via journald/fichiers), ajoute des labels, et envoie à Loki.
  → Travail avec Grafana/Loki: Permet la centralisation des logs applicatifs et Nginx, facilitant le troubleshooting ciblé.
- Grafana Alerting: Engine d'alertes intégré.
  → Travail avec Prometheus: On peut soit utiliser l'alerting de Grafana basé sur métriques, soit continuer à déléguer l'alerting à Prometheus/Alertmanager; les deux coexistent.

### b. Points forts

- **Visualisation avancée:** nombreux panels (time series, gauges, heatmaps), variables pour dashboards dynamiques.
- **Corrélation:** croise métriques (Prometheus) et logs (Loki) pour diagnostiquer rapidement.
- **Ouverture:** se connecte à MySQL, Postgres, Elasticsearch, et bien d'autres si nécessaire.

## 5.6. Serveur proxy

Dans le cadre de ce projet, un composant proxy était nécessaire pour assurer la gestion des requêtes entrantes, la distribution des ressources web, la redirection vers les services backend, ainsi que la sécurisation et l'optimisation du trafic. Plusieurs solutions ont été étudiées pour remplir ces fonctions, notamment en tant que serveur web, reverse proxy, load balancer et point d'entrée réseau.

### 5.6.1. Tableau comparatif des solutions de proxy

| **Critère** | **NGINX** | **Apache HTTP Server** | **HAProxy** | **Traefik** |
|---|---|---|---|---|
| **Serveur web** | Excellente prise en charge des fichiers statiques | Historique et robuste | Non prévu pour ça | Limité |
| **Reverse proxy** | Très performant | Fonctionnalité basique | Spécialisé | Moderne et dynamique |
| **Load balancing** | Round-robin, IP hash | Configuration complexe | Avancé | Intégré |
| **Point d'entrée réseau** | Partiel (HTTP/HTTPS uniquement) | Non adapté | Possible mais limité | Conçu pour ça |
| **Performance** | Excellente | Moyenne | Excellente | Très bonne |
| **Configuration** | Flexible mais verbeux | Simple | Technique | Déclaratif (YAML) |
| **Communauté & support** | Très large | Très large | Spécialisée | En croissance |
| **Intégration DevOps/CI/CD** | Compatible Docker, SSL, etc. | Moins adaptée | Utilisé en production | Très bien intégré |

### 5.6.2. Choix et justification

Pour assurer la gestion des requêtes entrantes, la distribution du trafic, la redirection vers les services backend et la sécurisation des échanges, NGINX a été retenu comme serveur proxy principal dans ce projet. Ce choix repose sur sa polyvalence, sa robustesse et sa compatibilité avec les environnements conteneurisés.

NGINX combine efficacement les rôles de serveur web, de reverse proxy et de load balancer dans un outil unique, léger et performant. Il est capable de gérer un grand nombre de connexions simultanées avec une faible consommation de ressources, ce qui le rend particulièrement adapté aux environnements à forte charge ou aux infrastructures mono-hôte comme un VPS. Sa configuration flexible, bien que parfois verbeuse, permet une personnalisation fine du comportement réseau, tout en restant accessible grâce à une documentation abondante et une communauté très active.

NGINX s'intègre naturellement avec des outils modernes tels que Docker, Let's Encrypt (pour la gestion automatique des certificats HTTPS), et les pipelines CI/CD. Il peut être positionné en frontal pour jouer le rôle de point d'entrée réseau, assurant la sécurisation des échanges, la redirection intelligente du trafic HTTP/HTTPS, et la distribution des ressources web. Ces caractéristiques en font une solution fiable, éprouvée et parfaitement adaptée aux besoins de ce projet DevOps.

# PARTIE 3: REALISATION

# chapitre 6: Installation et configuration

## 6.1. Configuration de github

### 6.1.1. Organisation des environnements

Le projet est structuré autour de trois environnements distincts, chacun avec sa branche Git dédiée et son workflow CI/CD :

*Tableau 17: environnement cible*

| Environnement | Branche GitHub | Workflow CI/CD | Objectif principal |
|---|---|---|---|
| **Développement** | develop | development.yml | Tester le code, tolérer les erreurs, expérimenter librement |
| **Staging** | staging | staging.yml | Simuler la production en local, valider les intégrations, ne tolère pas les erreurs |
| **Production** | main | cy.yml + cd.yml | Déploiement final, blue/green, haute disponibilité |

### 6.1.2. *Configuration du repository GitHub*

La configuration du repository GitHub constitue une étape cruciale dans la mise en place d'un projet collaboratif et structuré. Elle permet de centraliser le code source, de gérer les versions, et d'organiser les environnements de développement, de test (staging) et de production. Voici les étapes détaillées de cette configuration, accompagnées d'explications sur leur utilité.

#### a. Création du repository

La première étape consiste à créer un nouveau repository sur GitHub. Cette opération se fait généralement via l'interface web de GitHub :

- Nom du repository : il doit être explicite et refléter le nom ou l'objectif du projet alors on l'a nommé gestion-de-panne-devops puisque c'est une application de gestion de panne avec la quelle on applique les méthodologie devOps moderne
- Description : une brève description permet aux collaborateurs de comprendre rapidement le but du projet.
- Visibilité : le choix entre un repository public ou privé dépend du niveau de confidentialité souhaité et on a choisit le public pour sa gratuité

#### b. Création des branches principales

Afin de structurer le développement et de séparer les différents environnements, trois branches principales sont créées :

- **develop :** cette branche est dédiée au développement actif. C'est ici que les nouvelles fonctionnalités sont intégrées et testées localement.
- **staging :** cette branche intermédiaire permet de tester les fonctionnalités dans un environnement de préproduction. Elle sert à valider les changements avant leur mise en production.
- **main :** cette branche représente la version stable du projet, déployée en production. Seuls les changements validés et testés y sont fusionnés.

### 6.1.3. Stratégie de fusion

Chaque branche suit une stratégie de fusion bien définie :

- Une fois que les développeurs ont finit leurs travail, ils push les codes sur la branche develop qui est dédié pour la développement
- Lorsque plusieurs fonctionnalités sont prêtes, develop est fusionnée dans staging pour les tests d'intégration.
- Enfin, après validation, staging est fusionnée dans main pour le déploiement final en production.

## 6.2. Mise en place d'un environnement de développement conteneurisé et automatisé

### 6.2.1. Objectif

L'objectif de cette phase est de construire un environnement de développement isolé, reproductible et collaboratif, permettant à chaque développeur de travailler localement avec les mêmes conditions que celles de la production. Cette approche s'inscrit dans une logique DevOps, visant à réduire les conflits de dépendances, à accélérer le cycle de développement, et à préparer le projet pour les phases de staging et de production.

### 6.2.2. Architecture technique de l'environnement de développement

L'environnement repose sur une architecture conteneurisée multi-service, orchestrée par Docker Compose, et enrichie par un Makefile pour automatiser les tâches courantes.

Il comprend :

*Tableau 18: rôle principale des outils de développement*

| **Composant** | **Rôle principal** |
|---|---|
| **Docker-compose,dev,yml** | Définition et orchestration des services |
| **MySQL** | Base de données relationnelle pour Django |
| **Django** | Backend de l'application, exposé via runserver |
| **React** | Frontend de l'application, compilé et servi en local |
| **Makefile** | Automatisation des commandes Docker et DevOps |

La figure 12 illustre l'architecture de l'environnement de développement

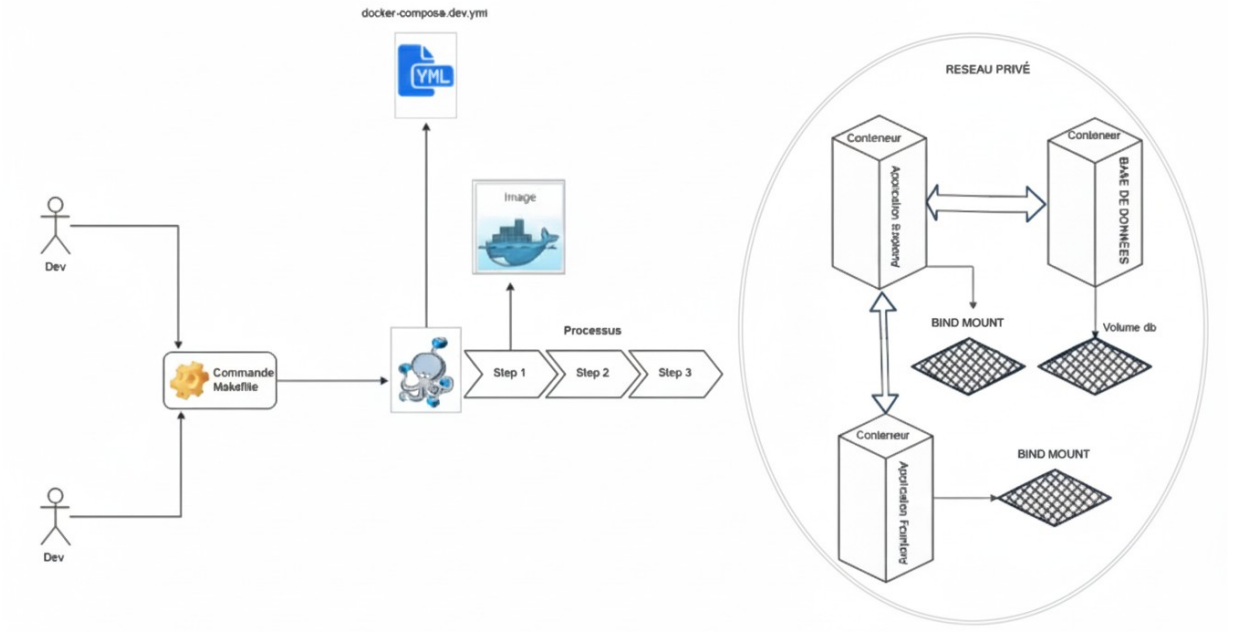



*Figure 12: architecture de l'environnement de développement*

### 6.2.3. Conteneurisation des service

#### a. Backend Django

Le backend de l'application est conteneurisé à l'aide d'un Dockerfile multi-étape dynamique, conçu pour s'adapter automatiquement à l'environnement cible (développement ou production) grâce à la variable APP_ENV.

Voici le contenue de la configuration du Dockerfile Django

```
FROM python:3.13-slim AS base
LABEL maintainer="Zo <zomahefaranaivo@gmail.com>" \
description="Backend Django pour l'application gestion de panne avec React"
ENV PYTHONDONTWRITEBYTECODE=1 \
PYTHONUNBUFFERED=1 \
DJANGO_SETTINGS_MODULE=backend.settings \
PATH="/opt/venv/bin:$PATH"
WORKDIR /back
RUN apt-get update && \
apt-get install -y --no-install-recommends curl netcat-openbsd && \
python3 -m venv /opt/venv && \
rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --upgrade pip && \
pip install --no-cache-dir -r requirements.txt
FROM base
ARG APP_ENV=prod
ENV DJANGO_ENV=${APP_ENV}
WORKDIR /back
RUN addgroup --system zoappgroup && \
adduser --system --ingroup zoappgroup --no-create-home zoappuser
COPY --chown=zoappuser:zoappgroup . .
RUN if [ "$DJANGO_ENV" = "prod" ]; then \
python manage.py collectstatic --noinput; \
fi
COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh
USER zoappuser
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=10s --start-period=60s --retries=5 \
CMD curl -f http://localhost:8000/health/ || exit 1
STOPSIGNAL SIGTERM
ENTRYPOINT ["/entrypoint.sh"]
CMD ["sh", "-c", "if [ \"$DJANGO_ENV\" = \"prod\" ]; then \
gunicorn backend.wsgi:application --bind 0.0.0.0:8000 --workers=3; \
else \
python manage.py runserver 0.0.0.0:8000; fi"]
```

La construction du conteneur repose sur une architecture dite multi-stage, qui consiste à séparer les étapes d'installation des dépendances et de préparation de l'environnement de celles de l'exécution proprement dite. Cette méthode permet de produire une image finale plus légère, plus rapide à déployer et plus sécurisée, car elle n'embarque que les éléments strictement nécessaires à l'exécution de l'application.

### *a.1. Script entrypoint.sh*

Ce script entrypoint.sh est utilisé comme point d'entrée du conteneur. Ce script joue un rôle crucial dans l'automatisation de l'initialisation du backend : il attend que la base de données soit disponible, applique les migrations, et crée automatiquement les comptes administratifs nécessaires si ceux-ci n'existent pas encore. Cela garantit que l'application est toujours prête à fonctionner dès son lancement, sans intervention manuelle.

Voici la configuration de l'entrypoint mis en place :

```
#!/bin/bash
echo "Attente de la base de données..."
until nc -z db 3306; do
  echo "En attente de la base à db:3306..."
  sleep 5
done
echo "Migrations Django..."
python manage.py migrate --noinput
echo "Vérification du superadmin..."
python manage.py shell <<EOF
from django.contrib.auth import get_user_model
User = get_user_model()
if not User.objects.filter(is_superuser=True).exists():
    User.objects.create_superuser('admin', 'admin@example.com',
'adminpass123')
EOF
echo "Vérification de l'admin applicatif..."
python manage.py shell <<EOF
from todo.models import Admin
from django.contrib.auth.hashers import make_password
if not Admin.objects.exists():
    Admin.objects.create(
        full_name="Admin Principal",
        matricule="CH-01",
        role="aadmin&chef jirama",
        contact="0331234567",
        email="mahefa@gmail.com",
        username="mahefa",
```

```
        password=make_password("jirama+/2025")
    )
else:
    print("Admin déjà présent en base, aucune création.")
EOF
echo "Initialisation terminée"
exec "$@"
```

Cette stratégie de conteneurisation présente plusieurs avantages majeurs. Tout d'abord, elle élimine toute intervention manuelle lors du déploiement : dès que le conteneur démarre, l'application est automatiquement initialisée, configurée et prête à l'emploi. Cela réduit considérablement les risques d'erreurs humaines et accélère les cycles de mise en production.

Ensuite, elle renforce la robustesse du système. Les migrations de base de données sont systématiquement appliquées, ce qui garantit que le schéma de données est toujours à jour avec le code source. De plus, les comptes critiques comme le super utilisateur Django ou l'administrateur applicatif sont créés automatiquement s'ils sont absents, assurant ainsi un accès immédiat à l'interface d'administration. Sur le plan de la sécurité, l'exécution sous un utilisateur non-root et l'isolation des dépendances dans un environnement virtuel réduisent la surface d'attaque et les risques liés à l'exécution de code tiers.

Enfin, cette approche est hautement adaptable. Le même Dockerfile peut être utilisé pour tous les environnements du projet : développement, staging et production simplement en modifiant la variable APP_ENV. Cela garantit une cohérence parfaite entre les environnements et facilite les tests, les déploiements et la maintenance du projet.

### b. Frontend React

Le frontend de l'application est développé avec React et conteneurisé à l'aide d'un Dockerfile spécifique à l'environnement de développement. Contrairement au backend Django, il n'est pas possible d'utiliser un Dockerfile dynamique unique pour gérer à la fois les environnements de développement et de production. En effet, les workflows React diffèrent fondamentalement selon le contexte :

- En développement, l'application est exécutée avec **npm start**, ce qui lance un serveur de développement avec hot reload, utile pour un cycle de développement rapide.
- En production, l'application doit d'abord être compilée avec **npm run build**, puis servie par un serveur statique (comme Nginx ou serve).

Cette distinction impose la création de deux Dockerfiles séparés : un pour le développement (Dockerfile.dev) et un autre pour la production (Dockerfile.prod)

Voici le contenue du Dockerfile react pour le développement

```
FROM node:20-alpine AS dev
WORKDIR /app
ENV NODE_ENV=development
COPY package*.json ./
RUN npm install --legacy-peer-deps
COPY . .
EXPOSE 3000
CMD [ "npm", "start" ]
```

Ce Dockerfile utilise une image légère basée sur Node.js 20 en version Alpine, optimisée pour la rapidité et la compacité. Il définit le répertoire de travail /app, copie les fichiers *package.json* et *package-lock.json*, puis installe les dépendances avec l'option *--legacy-peer-deps*. Cette option permet de contourner certains conflits de dépendances fréquents dans les projets React complexes.

Une fois les dépendances installées, l'ensemble du code source est copié dans le conteneur. Le port 3000 est exposé, car c'est celui utilisé par le serveur de développement React. Enfin, la commande npm start est exécutée au démarrage du conteneur, ce qui lance l'application avec hot reload.

### c. Mise en place des fichiers .dockerignore : optimisation et sécurité

Les fichiers .dockerignore jouent un rôle essentiel dans la construction des images Docker. Ils permettent d'exclure les fichiers inutiles ou sensibles du contexte de build, ce qui :

- Réduit la taille des images Docker, en évitant d'inclure des fichiers comme *node_modules*, *__pycache__*, ou les fichiers de logs.
- Améliore la sécurité, en excluant les fichiers *.env*, les clés secrètes, ou les configurations locales.
- Accélère les temps de build, en limitant le nombre de fichiers transférés au démon Docker.

Chaque service Django et React dispose de son propre fichier .dockerignore, adapté à sa stack technologique.

Le backend exclut les fichiers Python compilés, les environnements virtuels, les fichiers statiques générés, et les secrets. Le frontend exclut les dossiers build/, node_modules/, les fichiers de logs, et les configurations d'éditeur.

Celle de react exclut les fichier de dépendance node_modules les fichier de build et les fichiers liés à l'IDE.

#### d. Base de données Mysql

Le service de base de données est conteneurisé directement via le fichier *docker-compose.dev.yml* sans nécessiter d'un Dockerfile personnalisé. Il repose sur l'image officielle mariadb:10.11, reconnue pour sa stabilité et sa compatibilité avec les projets Django utilisant MySQL comme moteur de base de données.

### 6.2.4. Orchestration avec Docker Compose

L'orchestration des services de l'application est assurée par Docker Compose, à travers un fichier de configuration nommé ***docker-compose.dev.yml.*** Ce fichier centralise la définition des services nécessaires au bon fonctionnement de l'environnement de développement : la base de données, le backend Django et le frontend React. Chaque service y est configuré avec ses ports, ses volumes, ses variables d'environnement, ses dépendances et son réseau.

L'un des principaux avantages de cette approche est la simplicité de lancement, une seule commande permet d'instancier l'ensemble de l'environnement local, prêt à l'emploi :

```
docker compose -f docker-compose.dev.yml --env-file .env up -d
```

Cette commande lit les variables d'environnement depuis un fichier .env, ce qui permet de modifier les paramètres sensibles (comme les mots de passe ou les ports) sans toucher au code source ni au fichier Compose lui-même. Cela favorise la portabilité, la sécurité et la maintenabilité du projet.
Voici les services db et backend dans le fichier docker-compose.dev.yml

```
services:
  db:
    image: mariadb:10.11
    container_name: db
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${DB_NAME}
      MYSQL_USER: ${DB_USER}
      MYSQL_PASSWORD: ${DB_PASSWORD}
    ports:
      - "${DB_PORT}:3306"
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-uroot", "-p$
{MYSQL_ROOT_PASSWORD}"]
      interval: 30s
      timeout: 10s
      retries: 10
      start_period: 40s
```

```
  volumes:
    - mysql_data:/var/lib/mysql
    - ./my.cnf:/etc/mysql/conf.d/my.cnf
  networks:
    - panne_jirama
backend:
  build:
    context: ./backend
    args:
      APP_ENV: ${APP_ENV}
  container_name: django-dev-panne
  restart: unless-stopped
  environment:
    APP_ENV: ${APP_ENV}
    DEBUG: ${DEBUG}
    DJANGO_SECRET_KEY: ${DJANGO_SECRET_KEY}
    DJANGO_ALLOWED_HOSTS: ${DJANGO_ALLOWED_HOSTS}
    DB_HOST: ${DB_HOST}
    DB_NAME: ${DB_NAME}
    DB_USER: ${DB_USER}
    DB_PASSWORD: ${DB_PASSWORD}
    DB_PORT: ${DB_PORT}
  ports:
    - "${BACK_PORT}:${BACK_PORT}"
  volumes:
    - ./backend:/back
    - ./backend/media:/back/media
  depends_on:
    db:
      condition: service_healthy
  networks:
    - panne_jirama
frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile.dev
  container_name: react-dev-panne
  restart: unless-stopped
  env_file:
    - ./frontend/.env
  ports:
    - "${FRONT_PORT}:${FRONT_PORT}"
```

```
    volumes:
      - ./frontend:/app
      - /app/node_modules
    depends_on:
      backend:
        condition: service_healthy
    networks:
      - panne_jirama
volumes:
  mysql_data:
networks:
  panne_jirama:
  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile.dev
    container_name: react-dev-panne
    restart: unless-stopped
    env_file:
      - ./frontend/.env
    ports:
      - "${FRONT_PORT}:${FRONT_PORT}"
    volumes:
      - ./frontend:/app
      - /app/node_modules
    depends_on:
      backend:
        condition: service_healthy
    networks:
      - panne_jirama
volumes:
  mysql_data:
networks:
  panne_jirama:
```

Voici le fichier **.env** associé à docker-compose.dev.yml

```
APP_ENV=dev
DEBUG=1
DJANGO_SECRET_KEY=SuperSecretKeyForDev
DJANGO_ALLOWED_HOSTS=localhost,127.0.0.
1
DB_HOST=db
DB_NAME=jirama_db
DB_USER=django_user
DB_PASSWORD=Django@2025!
MYSQL_ROOT_PASSWORD=rootpass123
DB_PORT=3306
FRONT_PORT=3000
BACK_PORT=8000
```

Ce fichier .env permet de centraliser la configuration de l'environnement de développement.Cette orchestration avec Docker Compose constitue une base solide pour un environnement de développement cohérent, reproductible et facile à déployer. Elle favorise la collaboration entre développeurs et simplifie la12 pt gestion des dépendances entre services.Cette orchestration avec Docker Compose constitue une base solide pour un environnement de développement cohérent, reproductible et facile à déployer. Elle favorise la collaboration entre développeurs et simplifie la gestion des dépendances entre services.

### 6.2.5. Automatisation des commandes avec Makefile

Pour simplifier l'utilisation de Docker,Docker compose et accélérer les tâches quotidiennes, un Makefile complet a été mis en place. Il regroupe l'ensemble des commandes nécessaires à la gestion des environnements de développement, de staging et de production, ainsi qu'aux services de monitoring et de proxy.

Voici le contenue du makefile qu'on a mis en place pour le développement

```
COMPOSE_DEV=docker compose -f docker-compose.dev.yml --env-file .env -p panne-dev
COMPOSE_BLUE=docker compose -f docker-compose.blue.yml --env-file .env.global -p
panne-blue
COMPOSE_GREEN=docker compose -f docker-compose.green.yml --env-file .env.global -p
panne-green
COMPOSE_DB=docker compose -f docker-compose.db.yml --env-file .env.global
COMPOSE_MONITORING=docker compose -f docker-compose.monitoring.yml
EDGE=docker compose -f docker-compose.edge.yml
DB_NAME=jirama_db
DB_USER=django_user
DB_PASSWORD=Django@2025!
CONTAINER_DB_DEV=db
CONTAINER_BACKEND_DEV=django-dev-panne
CONTAINER_FRONTEND_DEV=django-dev-panne
CONTAINER_BACKEND_BLUE=backendBlue
CONTAINER_FRONTEND_BLUE=frontendBlue
CONTAINER_DB=db-global
CONTAINER_BACKEND_GREEN=backendGreen
CONTAINER_FRONTEND_GREEN=frontendGreen
CONTAINER_PROMETHEUS=prometheus-monitoring
CONTAINER_GRAFANA=grafana-monitoring
CONTAINER_LOKI=loki-monitoring
CONTAINER_PROMTAIL=promtail-monitoring
CONTAINER_MYSQLEXPORTER=mysql-exporter-monitoring
dev-up:
        $(COMPOSE_DEV) up -d
dev-down:
        $(COMPOSE_DEV) down
dev-down-all:
        $(COMPOSE_DEV) down -v
dev-build:
        $(COMPOSE_DEV) up -d --build
dev-start:
        $(COMPOSE_DEV) start
dev-stop:
        $(COMPOSE_DEV) stop
dev-restart:
        $(COMPOSE_DEV) restart
dev-logs:
        $(COMPOSE_DEV) logs -f
backend-dev-logs:
        docker container logs $(CONTAINER_BACKEND_DEV)
frontend-dev-logs:
        docker container logs $(CONTAINER_FRONTEND_DEV)
```

Grace à la collaboration avec les développeurs on a automatisé tous les commandes nécessaires dans le « Makefile » pour « debuger », analyser, corriger, tester et tous ce qui est important qui pourrait minimiser l'erreur humaine et accélérez le développement de l'application dans un environnement conteneurisé. Cette automatisation réduit considérablement les erreurs humaines, améliore la productivité quotidienne et facilite l'intégration de nouveaux collaborateurs en leur offrant une interface de commande unifiée et documentée.

### 6.2.6. Mise en place du workflow develop.yml pour la branche develop

#### a. Objectif du workflow

Le workflow develop.yml est un pipeline d'intégration continue conçu pour automatiser toutes les vérifications techniques sur la branche develop. Il garantit que chaque modification du code respecte les standards de qualité, de sécurité et de maintenabilité avant d'être fusionnée vers la branche staging. Ce workflow est déclenché automatiquement à chaque push sur la branche develop.

#### b. Structure générale du fichier develop.yml

Il est structuré en trois jobs parallèles :

- lint : vérification du style et de la qualité du code (Python et React)
- test : exécution des tests unitaires et mesure de la couverture
- security : analyse statique du code et scan de sécurité des dépendances et fichiers Docker
- Le workflow est nommé Develop CI pour être facilement identifiable dans l'interface GitHub.

Le workflow est configuré à se déclenché automatiquement à chaque push sur la branche develop.

Voici la configuration du condition de déclenchement automatique du workflow.

```
name: Develop CI

on:

  push:

    branches: [ "develop" ]
```

#### c. Job 1 : Lint Python & React

Ce job vérifie la qualité du code source en analysant le style et les erreurs potentielles dans les fichiers Python (backend) et JavaScript/TypeScript (frontend).

**Étapes principales :**

- **Checkout du code** avec actions/checkout@v5

- **Setup Python 3.13** avec cache pip
- **Installation et exécution de Ruff** pour analyser le code Python (ruff check) et générer un rapport JSON
- **Formatage automatique** avec ruff --fix et ruff format, sans bloquer le pipeline
- **Setup Node.js 20.19.0** avec cache npm
- **Installation des dépendances React** avec npm install --legacy-peer-deps
- **Linting React** avec npx eslint, export des résultats en JSON et upload des artefacts

Voici la configuration du job lint dans le workflow develop.yml

```
jobs:
  lint:
    name: Lint Python & React
    runs-on: ubuntu-latest
    steps:
    - name: Checkout du code
      uses: actions/checkout@v5
    - name: Setup Python
      uses: actions/setup-python@v6
      with:
        python-version: '3.13'
        cache: 'pip'
    - name: Installer Ruff
      run: pip install ruff
    - name: Lint Python avec Ruff
      continue-on-error: true
      run: |
        cd backend
        ruff check . --output-format=json > ruff-report.json
    - name: Upload Ruff report
      uses: actions/upload-artifact@v4
      with:
        name: ruff-report
        path: backend/ruff-report.json
    - name: Format Python avec Ruff
      continue-on-error: true
      run: |
        cd backend
        ruff check . --fix
        ruff format .
```

```
- name: Setup Node.js
    uses: actions/setup-node@v6
    with:
      node-version: 20.19.0
      cache: 'npm'
      cache-dependency-path: frontend/package-lock.json
  - name: Installer les dépendances React
    run: |
      cd frontend
      npm install --legacy-peer-deps
  - name: Lint React avec ESLint
    run: |
      cd frontend
      npx eslint src --ext .js,.jsx,.ts,.tsx -f json -o eslint-report.json
  - name: Upload ESLint report
    uses: actions/upload-artifact@v4
    with:
      name: eslint-report
      path: frontend/eslint-report.json
```

### d. JobTests Django & React

Ce job exécute les tests unitaires du backend et du frontend, et mesure la couverture du code.

**Backend Django :**

- Setup Python et installation des dépendances via requirements.txt
- Installation de coverage.py
- Exécution des tests avec coverage run manage.py test
- Génération et upload du rapport HTML (htmlcov)

**Frontend React :**

- Setup Node.js et installation des dépendances
- Exécution des tests avec npx vitest run --coverage
- Upload du rapport de couverture (frontend/coverage)

Voici la configuration du job test dans le workflow develop.yml

```
test:
  name: Tests Django & React + Coverage
  runs-on: ubuntu-latest
  steps:
  - name: Checkout du code
    uses: actions/checkout@v5
  - name: Setup Python
    uses: actions/setup-python@v6
    with:
      python-version: '3.13'
      cache: 'pip'
  - name: Installer les dépendances Python
    run: |
      cd backend
      python -m pip install --upgrade pip
      pip install -r requirements.txt
  - name: Installer coverage.py
    run: pip install coverage
  - name: Tests Django avec coverage
    continue-on-error: true
    run: |
      cd backend
      coverage run manage.py test --settings=backend.settings_ci
      coverage report
      coverage html
  - name: Upload rapport coverage Django
    uses: actions/upload-artifact@v4
    with:
      name: django-coverage-report
      path: backend/htmlcov
  - name: Setup Node.js
    uses: actions/setup-node@v6
    with:
      node-version: 20.19.0
      cache: 'npm'
      cache-dependency-path: frontend/package-lock.json
  - name: Installer les dépendances React
    run: |
      cd frontend
      npm install --legacy-peer-deps
```

```
 - name: Tests React avec Vitest + coverage
    continue-on-error: true
    run: |
        cd frontend
        npx vitest run --coverage
  - name: Upload rapport coverage React
    uses: actions/upload-artifact@v4
    with:
      name: react-coverage-report
      path: frontend/coverage
```

### e. Job 3 : Analyse de sécurité (Semgrep + Trivy)

Ce job vérifie la sécurité du code et des fichiers Docker.

- **Audit des dépendances :**
- **Analyse statique avec Semgrep**
- **Scan des fichiers Docker et compose avec Trivy**

Voici la configuration du job security dans le workflow develop.yml

```
security:
   name: Analyse de sécurité (Semgrep + Trivy)
   runs-on: ubuntu-latest
   steps:
   - name: Checkout du code
     uses: actions/checkout@v5
   - name: Setup Node.js
     uses: actions/setup-node@v6
     with:
       node-version: 20.19.0
       cache: 'npm'
       cache-dependency-path: frontend/package-lock.json
   - name: Audit sécurité npm
     run: |
       cd frontend
       npm audit --audit-level=high || echo "Vulnérabilités connues, documentées
dans SECURITY.md"
```

```
    - name: Setup Python
      uses: actions/setup-python@v6
      with:
        python-version: '3.13'
        cache: 'pip'
    - name: Audit sécurité Python
      run: |
        pip install pip-audit
        pip-audit || echo "Vulnérabilités connues, documentées dans SECURITY.md"
    - name: Installer Semgrep
      run: pip install semgrep
    - name: Scan de sécurité avec Semgrep
      continue-on-error: true
      run: semgrep --config=auto --json > semgrep-report.json
    - name: Upload rapport Semgrep
      uses: actions/upload-artifact@v4
      with:
        name: semgrep-report
        path: semgrep-report.json
  - name: Scan Dockerfile backend
      continue-on-error: true
      uses: aquasecurity/trivy-action@master
      with:
        scan-type: 'fs'
        scan-ref: './backend/Dockerfile'
    - name: Scan Dockerfile frontend
      continue-on-error: true
      uses: aquasecurity/trivy-action@master
      with:
        scan-type: 'fs'
        scan-ref: './frontend/Dockerfile.dev'
    - name: Scan docker-compose.dev.yml
      continue-on-error: true
      uses: aquasecurity/trivy-action@master
      with:
        scan-type: 'fs'
        scan-ref: './docker-compose.dev.yml'
```

#### f. Rapport générés

Le tableau 19 présente l'ensemble des rapports générés automatiquement au cours du processus DevOps. Ces rapports, produits par différents outils d'analyse et de test, sont utilisés pour assurer la qualité, la sécurité et la couverture du code source. Ils sont générés à chaque étape clé du pipeline CI/CD via GitHub Actions et télechargeable via l'artifact de github.

*Tableau 19: listes des rapports générés dans github*

| **Rapport** | **Outil** | **Format** | **Utilité principale** |
|---|---|---|---|
| ruff-report.json | Ruff | JSON | Linting Python |
| eslint-report.json | ESLint | JSON | Linting React |
| django-coverage-report | coverage.py | HTML | Couverture des tests backend |
| react-coverage-report | Vitest | HTML | Couverture des tests frontend |
| semgrep-report.json | Semgrep | JSON | Analyse statique et sécurité du code |

## 6.3. Phase de staging : validation locale avant mise en production

### 6.3.1. Objectif

L'objectif de cette phase est de valider une version candidate de l'application dans un environnement de staging local, identique à la production, mais isolé. Cette approche permet de tester les images Docker, les interactions entre services, les métriques et les logs, tout en garantissant une transition fluide vers la production. Elle s'inscrit dans une logique DevOps, avec une architecture blue/green, une orchestration multi-compose, et un pipeline CI/CD strict.

### 6.3.2. Conteneurisation des services

#### a. Conteneurisation du backend Django

Le backend de l'application est conteneurisé à l'aide d'un Dockerfile multi-étape dynamique, identique à celui utilisé en développement. La différence réside dans la valeur de la variable APP_ENV, qui est fixée à prod dans le fichier .env.global. Cela permet de simuler un environnement de production tout en restant en local.

Cette approche garantit que le backend est lancé dans des conditions proches de la production, avec une image Docker stable, sécurisée et prête à être déployée.

### b. Conteneurisation du frontend React

Le frontend React utilise un Dockerfile conçu spécifiquement pour la production qui est basé sur la méthode multi-étape avec la première étape l'installation des dépendances et le npm run build pour la production et dans la seconde étape on copie juste le résultat du première étape dans une image nginx légère pour alleger l'image finale. Contrairement au développement, il ne repose pas sur un serveur de développement (npm start) mais sur un serveur statique Nginx comme à la production.

Voici le contenue du Dockerfile,prod du frontend react

```
FROM node:20-alpine AS builder
WORKDIR /app
ARG REACT_APP_API_URL
ENV REACT_APP_API_URL=$REACT_APP_API_URL
ENV NODE_ENV=production
COPY package*.json ./
RUN npm install --legacy-peer-deps
COPY . .
RUN npm run build
# ---- Étape 2: La "Production" ----
FROM nginx:1.29.1-alpine-slim
LABEL maintainer="Zo <zomahefaranaivo@gmail.com>" \
    description="frontend react pour l'appli de gestion de panne avec django comme back"
COPY --from=builder /app/build /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
RUN apk --no-cache add curl
HEALTHCHECK --interval=15s --timeout=3s --start-period=5s --retries=2 \
  CMD curl -f http://localhost/ || exit 1
```

Le fichier nginx.conf présenté ci-dessous est utilisé dans l'image dédié à la production du frontend React. Il configure le serveur Nginx pour servir l'application React compilée de manière optimale et compatible avec le routage côté client.

Voici la configuration du fichier  nginx.conf associé à la Dockerfile.prod du frontend react

```
server {
  listen 80;
  server_name localhost;
  root /usr/share/nginx/html;
  index index.html;
  location / {
    try_files $uri /index.html;
  }
}
```

#### c. Conteneurisation de la Base de données

Le service de base de données est défini dans docker-compose.db.yml_et partagé entre les versions blue et green,en général c'est un service qui ne dois jamais être arrêté car la base de données ne change jamais mais il s'adapte juste à la version active.

### 6.3.3. Mise en place de docker-compose pour l'orchestration des services

#### a. Mise en place d'une architecture multi-compose en staging

*Tableau 20: explication des rôles des fichiers docker-compose en staging*

| **Fichier Compose** | **Rôle principal** |
|---|---|
| docker-compose.blue.yml | Déploie la version candidate (backendBlue, frontendBlue) |
| docker-compose.db.yml | Fournit une base MySQL partagée avec volume persistant et healthcheck |
| docker-compose.edge.yml | Configure un proxy Nginx pour router les requêtes vers les services actifs |
| docker-compose.monitoring.yml | Active les outils de monitoring (Prometheus, Grafana, Loki, etc.) |
| .env.global | Centralise les variables d'environnement pour tous les services |

### b. Configuration de docker-compose.blue.yml

C’est la Version candidate à déployer en production après la phase du staging, dans le cas contraire si blue est l’active en production alors on utilise la nouvelle version docker-compose.green.yml dans la phase de staging, On utilise toujours la version inactive de la production en staging pour bien tester la nouvelle version à déployer afin de garantir que l’application est près pour la production.

Ce fichier définit les services backendBlue et frontendBlue, chacun basé sur une image Docker versionnée et poussée automatiquement vers DockerHub par le workflow staging.yml.

Voici la configuration de Docker-compose.blue.yml.

```
services:
  backend-blue:
    image: zomahefaranaivo/django-backend-panne-staging:latest
    container_name: backendBlue
    restart: always
    environment:
      APP_ENV: ${APP_ENV}
      DEBUG: ${DEBUG}
      DJANGO_SECRET_KEY: ${DJANGO_SECRET_KEY}
      DJANGO_ALLOWED_HOSTS: ${DJANGO_ALLOWED_HOSTS}
      DB_HOST: ${DB_HOST}
      DB_NAME: ${DB_NAME}
      DB_USER: ${DB_USER}
      DB_PASSWORD: ${DB_PASSWORD}
      DB_PORT: ${DB_PORT}
    volumes:
      - django_static:/back/staticfiles
      - django_media:/back/media
    networks:
      - panne-jirama-blue
  frontend-blue:
    image: zomahefaranaivo/react-frontend-nginx-panne-staging:latest
    container_name: frontendBlue
    restart: always
    expose:
      - "80"
    depends_on:
      backend-blue:
        condition: service_healthy
    networks:
      - panne-jirama-blue
```

```
volumes:
  django_static:
    external: true
    name: jirama-django-static
  django_media:
    external: true
    name: jirama-django-media
networks:
  panne-jirama-blue :
    external: true
    name: panne-jirama-blue
```

### c. Configuration docker-compose.green.yml

Il a exactement la même configuration que celle de blue sauf pour les nom de services (frontend-green et backend-green) et le réseau(panne-jirama-green) pour bien identifié la couleur active qui est très utile pour la méthode de déploiement blue/green en production.

### d. Configuration docker-compose.db.yml

C’est la Base de données partagée lié à une volume persistante qui est configuré pour s’adapter à n’importe version active(blue ou green) car il appartient aux deux réseaux docker-compose panne-jirama-blue et panne-jirama-green.

Voici la configuration du fichier docker-compose.db.yml

```
services:
  db:
    image: mariadb:10.11
    container_name: db-global
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${DB_NAME}
      MYSQL_USER: ${DB_USER}
      MYSQL_PASSWORD: ${DB_PASSWORD}
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-uroot", "-p$
{MYSQL_ROOT_PASSWORD}"]
      interval: 30s
      timeout: 10s
      retries: 10
      start_period: 40s
    volumes:
      - mysql-staging-jirama:/var/lib/mysql
    networks:
      - panne-jirama-blue
      - panne-jirama-green
volumes:
  mysql-staging-jirama:
    external: true
    name: mysql-staging-jirama
networks:
  panne-jirama-blue:
    external: true
    name: panne-jirama-blue
  panne-jirama-green:
    external: true
    name: panne-jirama-green
```

### e. Configuration docker-compose.edge.yml

Le service *edge* constitue le point d'entrée principal de l'architecture de staging. Il repose sur un preverse proxy basé sur Nginx, chargé de recevoir les requêtes HTTP entrantes et de les rediriger vers les services internes appropriés, notamment le frontend, le backend et les composants de supervision. Cette approche permet de centraliser la gestion des accès et de simplifier l'architecture réseau de l'application. Le conteneur est basé sur l'image officielle *nginx:1.25-alpine* et expose le port HTTP afin d'assurer l'accessibilité du système. Sa configuration repose sur le montage de volumes dédiés permettant de gérer dynamiquement le routage des requêtes, de sécuriser l'accès aux interfaces sensibles et de servir efficacement les fichiers statiques et médias de l'application backend. Cette organisation améliore les performances globales et facilite la maintenance de l'infrastructure.

Voici la configuration du fichier docker-compose.edge.yml

```
services:
  edge:
    image: nginx:1.25-alpine
    container_name: edge
    restart: always
    ports:
      - "80:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
      - ./conf.d:/etc/nginx/conf.d:rw
      - django_static:/back/staticfiles
      - django_media:/back/media
      - ./nginx-secure:/etc/nginx/secure:ro
    networks:
      - panne-jirama-blue
      - panne-jirama-green
volumes:
  django_media:
    name: jirama-django-media
  django_static:
    name: jirama-django-static
networks:
  panne-jirama-blue:
    external: true
    name: panne-jirama-blue
  panne-jirama-green:
    external: true
    name: panne-jirama-green
```

### *e.1. Configuration du fichier nginx.conf*

Ce fichier configure le comportement global du serveur nginx(edge). Il :

- Configure un serveur écoutant sur le port 80
- Route les requêtes vers :
    - / vers la frontend react via proxy_pass http://frontend
    - /api,backend Django via proxy_pass http://backend
    - /django-static/ et /media/,fichiers statiques et médias
    - /grafana et /prometheus,interfaces de monitoring protégée

Voici la configuration mis en place du fichier nginx.conf

```
events {}
http {
 include       mime.types;
 default_type  application/octet-stream;
 # Définir les services actifs en dur
 upstream frontend {
  include /etc/nginx/conf.d/frontend_active.conf;
 }
 upstream backend {
  include /etc/nginx/conf.d/backend_active.conf;
 }
 server {
  listen 80;
  server_name localhost;
  # Résolution DNS Docker
  resolver 127.0.0.11 ipv6=off;
  location / {
   proxy_pass http://frontend;
   proxy_set_header Host $host;
   proxy_set_header X-Real-IP $remote_addr;
   proxy_redirect off;
   proxy_intercept_errors on;
   error_page 404 = /index.html;
  }
  location ~ ^/api(/|$) {
   proxy_pass http://backend;
   proxy_set_header Host $host;
   proxy_set_header X-Real-IP $remote_addr;
   proxy_redirect off;
  }
  # Fichiers statiques Django
  location /django-static/ {
   alias /back/staticfiles/;
   access_log off;
   expires 30d;
   add_header Cache-Control "public";
   add_header Content-Type text/css;
  }
  # Fichiers media Django (images des pannes)
  location /media/ {
   alias /back/media/;
   access_log off;
   expires 30d;
   add_header Cache-Control "public";
  }
```

```
 # Grafana
   location ~ ^/grafana(/|$) {
     # DOUBLE PROTECTION: Auth + Restriction IP
     auth_basic "Accès Monitoring Réservé";
     auth_basic_user_file /etc/nginx/secure/.htpasswd;
     allow 129.222.109.160;
     allow 172.22.0.1;
     deny all;
     proxy_pass http://grafana:3000;
     proxy_set_header Host $host;
     proxy_set_header X-Real-IP $remote_addr;
     proxy_redirect off;
   }
   # Prometheus
   location ~ ^/prometheus(/|$) {
     # DOUBLE PROTECTION: Auth + Restriction IP
     auth_basic "Accès Monitoring Réservé";
     auth_basic_user_file /etc/nginx/secure/.htpasswd;
     allow 129.222.109.160;
     allow 172.22.0.1;
     deny all;
     proxy_pass http://prometheus:9090;
     proxy_set_header Host $host;
     proxy_set_header X-Real-IP $remote_addr;
     proxy_redirect off;
   }
   location = /metrics/ {
     proxy_pass http://backend;
     proxy_set_header Host localhost;
     proxy_set_header X-Real-IP $remote_addr;
     proxy_redirect off;
   }
  }
}
```

### *e.2. Configuration des upstream*

Les fichiers conf.d contient deux fichiers frontend_active.conf et backend_active.conf qui contiennent une seule ligne chacun:

- *server frontend-blue:80;*
- *server backend-blue:8000;*

Les blocs upstream permettent de définir des alias logiques (frontend, backend) qui pointent vers des services Docker internes. Cela facilite :

- Le changement dynamique de version (blue ou green) sans modifier tout le fichier Nginx
- La réutilisation des blocs de configuration (proxy_pass http://frontend) dans plusieurs routes
- La gestion centralisée des points d'accès aux services

Ces fichiers sont modifiés automatiquement lors d'un déploiement pour basculer vers frontend-green ou backend-green, facilitant ainsi le déploiement blue/green.

### *e.3. Sécurisation d'accès des services de monitoring*

Ce fichier contient les identifiants chiffrés pour protéger l'accès aux interfaces sensibles comme Grafana et Prometheus. Il est utilisé avec la directive auth_basic dans les blocs /grafana et /prometheus et monté en volume dans le docker-compose.edge.yml.

Avec un ajout de restriction IP en plus, cela garantit une forte sécurité que seuls les utilisateurs autorisés peuvent accéder aux dashboard de supervision.

Voici le contenue du fichier qui contient les identifiants pour accéder au service de monitoring avec le mot de passe chiffré :

*mahefa:$2y$05$KAuNxC0Lzj7XKtCPLhpBRuEhIozgFo75JRpKH7Vvp4NATvINVhsA2*

Avantage du service edge mise en place:

- **Centralisation** : toutes les requêtes passent par un seul point, simplifiant la gestion des accès et des logs
- **Sécurité** : les interfaces critiques sont protégées par mot de passe et restriction IP
- **Flexibilité** : le routage peut être modifié dynamiquement en changeant les fichiers conf.d sans redémarrer tous les services
- **Alignement avec la production** : ce proxy simule exactement le comportement attendu en environnement réel

Ce service est donc un composant stratégique de l'environnement de staging. Il permet de tester l'intégration complète des services dans des conditions proches de la production, tout en assurant un contrôle précis sur le routage, la sécurité et la supervision.

### f. docker-compose.monitoring.yml

Objectif du monitoring en staging

L'intégration du monitoring dès la phase de staging permet de simuler fidèlement la production et de détecter les anomalies avant la mise en ligne. Cela permet de :

- Vérifier que les métriques sont bien exposées par les services (backend, base de données, système)
- Tester les dashboard Grafana dans un environnement réaliste
- Identifier les régressions de performance ou les erreurs silencieuses
- Valider la configuration des outils de supervision avant leur déploiement en production

Le staging étant une simulation isolée de la production, il constitue l'environnement idéal pour tester la chaîne de supervision sans impacter les utilisateurs finaux.

Voici la configuration des services prometheus, grafana, loki dans docker-compose.monitoring.yml

```
services:
  prometheus:
    image: prom/prometheus
    container_name: prometheus-monitoring
    volumes:
      - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml
    command:
      - '--config.file=/etc/prometheus/prometheus.yml'
      - '--web.external-url=http://localhost/prometheus/'
      - '--web.route-prefix=/prometheus'
    depends_on:
      - mysql-exporter
    networks:
      - panne-jirama-blue
      - panne-jirama-green
  grafana:
    image: grafana/grafana
    container_name: grafana-monitoring
    environment:
      - GF_SERVER_ROOT_URL=http://localhost/grafana/
      - GF_SERVER_SERVE_FROM_SUB_PATH=true
    volumes:
      - grafana_data:/var/lib/grafana
    depends_on:
      - prometheus
      - loki
```

```
    command:
      - '--path.procfs=/host/proc'
      - '--path.sysfs=/host/sys'
      - '--collector.filesystem.ignored-mount-points="^/(sys|proc|host|etc|dev|tmp|var/
lib/docker/.+)($|/)"'
    networks:
      - panne-jirama-blue
      - panne-jirama-green
volumes:
  grafana_data:
    name: grafana_data
networks:
  panne-jirama-blue:
    external: true
    name: panne-jirama-blue
  panne-jirama-green:
    external: true
    name: panne-jirama-green
```

### *f.1. Fichier prometheus.yml*

Ce fichier configure les cibles de Prometheus. Il définit plusieurs job_name.

Voici la configuration mise en place de prometheus.yml

```
global:
  scrape_interval: 15s  # Prometheus interroge les services toutes les 15 secondes
scrape_configs:
  - job_name: 'django-backend'
    metrics_path: /metrics/
    static_configs:
      - targets: ['edge:80']
    relabel_configs:
      - target_label: instance
        replacement: backend
      - target_label: color
        replacement: active
  - job_name: 'node'
    static_configs:
      - targets: ['node-exporter:9100']
    relabel_configs:
      - target_label: instance
        replacement: serveur
      - target_label: color
        replacement: host
  - job_name: 'mysql'
    metrics_path: /metrics
    static_configs:
      - targets: ['mysql-exporter:9104']
    relabel_configs:
      - target_label: instance
        replacement: mysql
  - job_name: 'prometheus'
    metrics_path: prometheus/metrics
    static_configs:
      - targets: ['prometheus:9090']
    relabel_configs:
      - target_label: instance
        replacement: prometheus
```

### *f.2. Fichier promtail-config.yml*

Ce fichier configure Promtail pour :

- Lire les logs des conteneurs Docker dans /var/lib/docker/containers/
- Associer des labels aux logs (nom du job, chemin du fichier)
- Envoyer les logs à Loki via l'URL http://loki:3100/loki/api/v1/push

Cela permet d'agréger tous les logs applicatifs consultable via Grafana.

Voici la configuration du fichier promtail-config.yml

```
server:
  http_listen_port: 9080
  grpc_listen_port: 0
positions:
  filename: /tmp/positions.yaml
clients:
  - url: http://loki:3100/loki/api/v1/push
scrape_configs:
  - job_name: docker
    static_configs:
      - targets:
          - localhost
        labels:
          job: docker
          __path__: /var/lib/docker/containers/*/*.log
```

### *f.3.Configuration de mysql-exporter*

Ce fichier contient les identifiants d'un utilisateur MySQL dédié à la supervision. Il est monté dans le conteneur mysql-exporter et utilisé pour se connecter à la base de données sans exposer les mots de passe dans les variables d'environnement.

Voici le contenue du fichier :

```
[client]
user=mysqld_exporter
password=exporter_pass
host=db
```

Pour que ça fonctionne il faut créer cette utilisateur dédié dans la base en lui donnant que les privilèges nécessaires pour que mysqld-exporter puisse accéder aux statistiques internes de MySQL sans compromettre la sécurité de la base. Cette méthode respecte les bonnes pratiques de sécurité en séparant les privilèges et en limitant l'accès seulement aux métriques.

### g. Configuration des variable d'environnement

Le fichier env.global regroupe toutes les variables d'environnement nécessaire pour une configuration centralisée.
Voici le contenue du fichier .env.global

```
# Environnement d'exécution
APP_ENV=prod
DEBUG=0
# Clé secrète Django (différente de dev)
DJANGO_SECRET_KEY=SuperSecretKeyForStaging
# Hôtes autorisés (staging local)
DJANGO_ALLOWED_HOSTS=backend,localhost,127.0.0.1,192.168.119.84
# Base de données
DB_HOST=db
DB_NAME=jirama_db
DB_USER=django_user
DB_PASSWORD=Django@2025!
MYSQL_ROOT_PASSWORD=rootpass123
DB_PORT=3306
```

Il est utilisé par tous les fichiers Compose pour garantir la cohérence et la sécurité de la configuration.

### h. Avantages de cette orchestration modulaire

- Séparation des responsabilités : chaque fichier gère un domaine précis (app, DB, proxy, monitoring)
- Flexibilité : on peut démarrer uniquement la version blue ou green selon le scénario
- Cohérence avec la production : même structure, mêmes images, mêmes variables
- Sécurité : pas de bind mount, pas de hot reload, pas de debug
- Traçabilité : chaque version est figée dans une image Docker taggée et testée

Cette orchestration multi-compose constitue une base robuste pour un environnement de staging professionnel, reproductible et aligné avec les exigences de la production. Elle permet de tester chaque version dans des conditions réalistes, tout en conservant une grande souplesse opérationnelle.

### 6.3.4. Mise en place du workflow GitHub Actions : staging.yml

#### a. Objectif

Le workflow staging.yml est un pipeline d'intégration continu conçu pour valider automatiquement chaque version candidate avant sa mise en production. Il s'appuie sur la même structure que le workflow de développement (develop.yml), avec des ajustements spécifiques à l'environnement de staging.

Ce workflow est déclenché automatiquement à chaque push ou pull request vers la branche staging, ce qui permet de tester les contributions fusionnées depuis develop dans un environnement simulant la production.

Voici le code de configuration du condition de déclenchement du workflow staging.yml

```
name: Staging CI

on:

  push:

    branches: [ "staging" ]

  pull_request:

    branches: [ "staging"]
```

#### b. Les jobs du staging.yml

Les trois premiers jobs sont identiques à ceux du workflow develop.yml, avec une petite modification pour l'adaptés à la phase de staging. Ils permettent de :

- Vérifier la qualité du code (Ruff, ESLint)
- Exécuter les tests unitaires et mesurer la couverture (coverage.py Vitest)
- Auditer les dépendances et analyser le code statiquement (pip-audit, npm audit, Semgrep)
- Scanner les fichiers Docker avec Trivy
- **Job Dockerhub** : ce job est exécuté uniquement après la réussite des trois premiers jobs, grâce à la directive **needs**

Ces jobs sont configurés pour être bloquants : aucune tolérance aux erreur.

**Fonctionnement du job Dockerhub**

- Connexion à DockerHub via docker/login-action@v3
- Activation du builder avancé avec docker/setup-buildx-action@v3
- Construction et push des images avec docker/build-push-action@v6
- Les images sont taggées deux fois pour une meilleur gestion de version

Voici la configuration du job dockerhub dans le workflow staging.yml

```
Dockerhub:
  name: Build & push des images vers dockerhub
  runs-on: ubuntu-latest
  needs: [Lint, Test, Security]
  if: success()
  steps:
   - name: Checkout du code
     uses: actions/checkout@v5
   - name: Login to Docker Hub
     uses: docker/login-action@v3
     with:
       username: ${{ vars.DOCKERHUB_USERNAME }}
       password: ${{ secrets.DOCKERHUB_TOKEN }}
   - name: Set up QEMU
     uses: docker/setup-qemu-action@v3
   - name: Set up Docker Buildx
     uses: docker/setup-buildx-action@v3
   - name: Build & Push Frontend
     uses: docker/build-push-action@v6
     with:
       context: ./frontend
       file: ./frontend/Dockerfile.prod
       push: true
       build-args: |
         REACT_APP_API_URL=${{ vars.REACT_APP_API_URL }}
       tags: ${{ vars.DOCKERHUB_USERNAME }}/react-frontend-nginx-panne-staging:v$
{{ github.run_number }}
```

```
    - name: Build & Push Backend
      uses: docker/build-push-action@v6
      with:
        context: ./backend
        file: ./backend/Dockerfile
        push: true
        tags: ${{ vars.DOCKERHUB_USERNAME }}/django-backend-panne-staging:v${{ github.run_number }}
```

### c. Configuration des variables d'environnement

Les secrets DockerHub (DOCKERHUB_USERNAME, DOCKERHUB_TOKEN) sont stockés dans GitHub Actions, dans la section "Secrets and variables".On va dans la paramètre du compte « account settings »,puis on rentre sur la section « jeton d'accès personnel » et une fois dedans on clique juste sur générer un nouveau jeton.On configure la date d'expiration et le droit du token, puisque c'est un token utilisé pour faire le push et le pull on a choisit l'option Lire et écrire.

Une fois que le token dockerhub est créer,il faut le mettre dans les secrets de github pourque le workflows l'utilise comme variable d'environnement et aussi la variable REACT_APP_API_URL qui est nécessaire à la build de react pour la connexion backend.

La figure 13 montre les secrets et variables dans github pour le workflow staging.yml

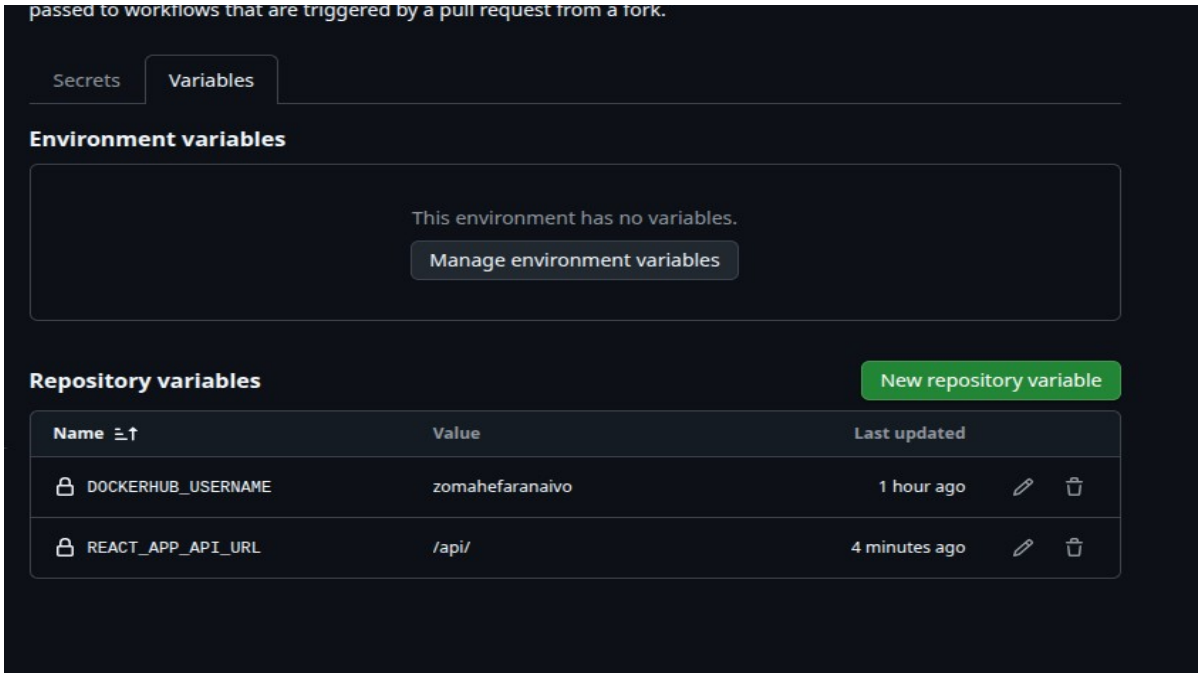



*Figure 13: Liste des variables et secrets github en staging*

## 6.4. Mise en place de l'environnement de production

### 6.4.1. Introduction à la mise en production

La mise en production constitue l'aboutissement technique du projet. Elle vise à garantir que l'application soit disponible, sécurisée, scalable et observable dans un environnement réel. Cette section détaille l'ensemble des étapes techniques mises en œuvre pour déployer l'application "Gestion de panne" sur un serveur distant, avec un pipeline CI/CD complet, un déploiement blue/green, un monitoring avancé, et une architecture « Dockerisée" modulaire.

### 6.4.2. Choix et configuration du serveur VPS

#### a. Choix du fournisseur

Le serveur VPS a été provisionné chez LWS (Ligne Web Services), un hébergeur français offrant des VPS à bas coût. Le choix s'est porté sur une formule VPS M, avec 8 Go de RAM, 150 Go de disque, et Ubuntu 24.04 LTS comme système d'exploitation. Ce choix répondait aux critères suivants :

- Hébergement en France (RGPD)
- Support de Docker
- Accès root SSH sur ubuntu reconnue par sa stabilité et compatibilité avec les outils devOps
- système basé
- Prix compétitif

Voici l'interface de configuration du serveur dans lws.

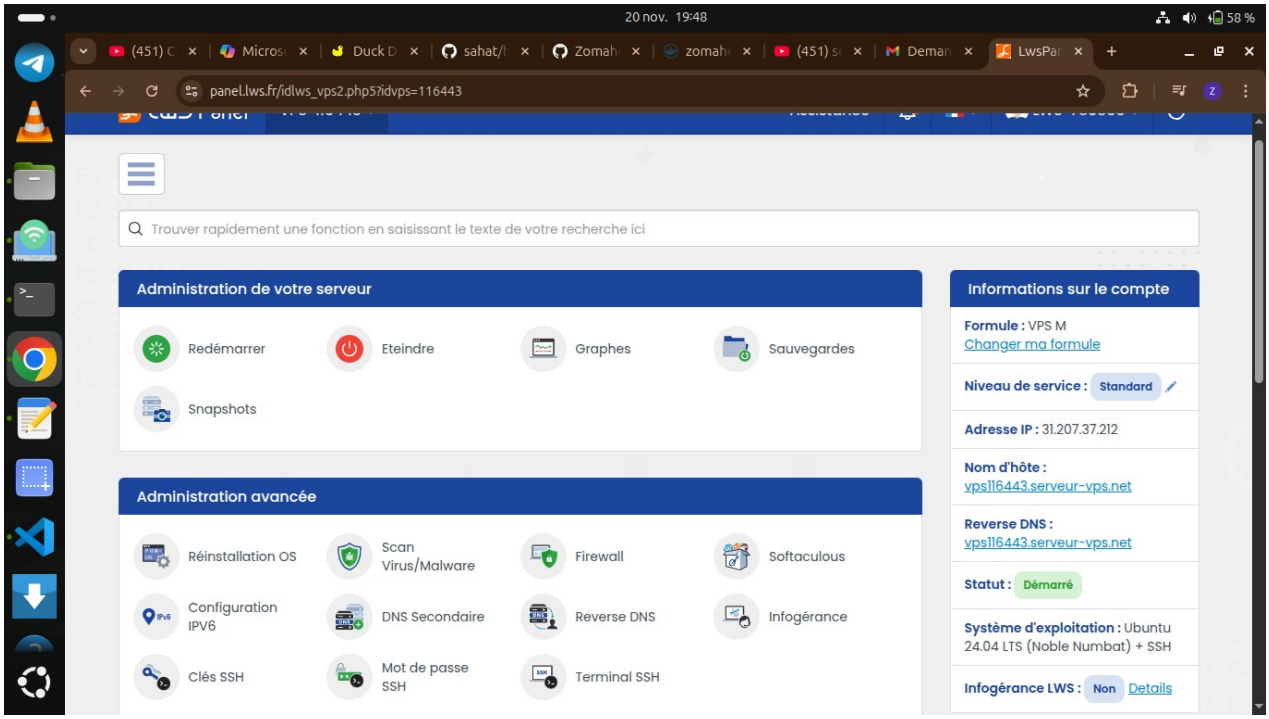



*Figure 14: interface de configuration du VPS*

### b. Connexion SSH et sécurité

#### *b.1. Configuration ssh*

Une **clé SSH ED25519** a été générée localement avec la commande suivante :

```
ssh-keygen -t ed25519 -C "zo@lws"
```

La clé publique a été ajoutée au panneau LWS dans la section “Clés SSH”, permettant une connexion sécurisée sans mot de passe

Des mesures de sécurité ont été appliquées :

- Désactivation du login root par mot de passe (PermitRootLogin no)
- Désactivation de l’authentification par mot de passe (PasswordAuthentication no)
- Création d’un utilisateur non-root mahefa, membre des groupes sudo et docker
- Configuration des permissions sur .ssh et authorized_keys

#### *b.2. Création et configuration de l’utilisateur dédié*

Un utilisateur non-root nommé **mahefa** a été créée pour effectuer les opérations de déploiement. Il est membre des groupes sudo et docker, ce qui lui permet d’exécuter des commandes administratives et de manipuler les conteneurs sans privilèges root.

La figure 15 montre la création et configuration de l’utilisateur non root dans le VPS

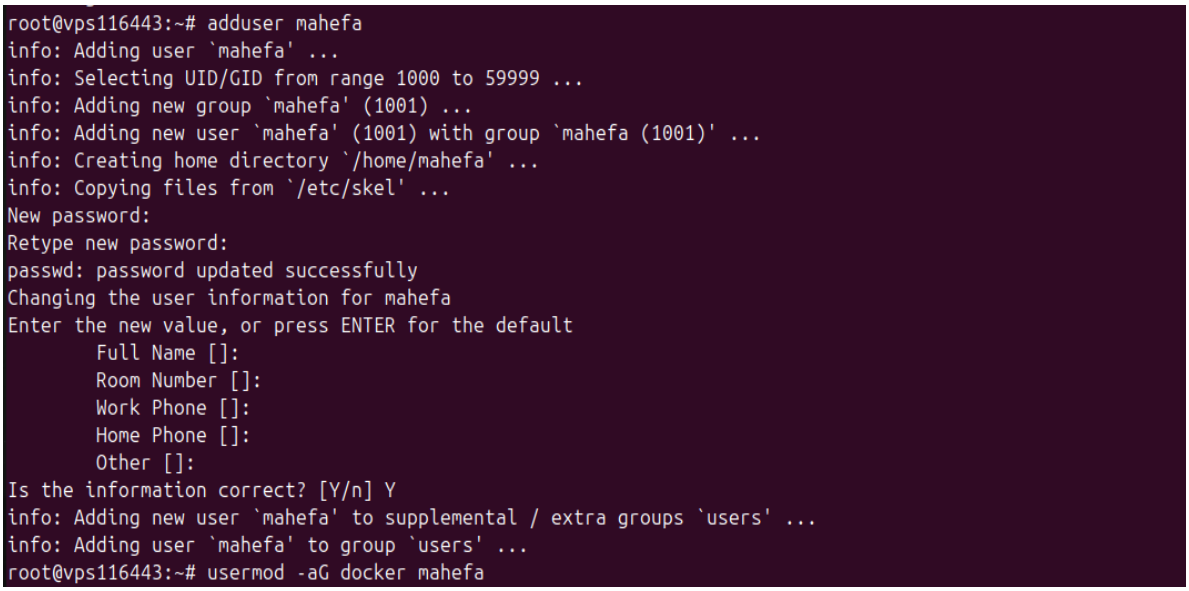



*Figure 15: création de l'utilisateur non root*

Les clés SSH ont été copiées depuis le compte root vers le dossier .ssh de l'utilisateur mahefa, et on lui a attribué les permissions suivantes :

- *chmod 700 ~/.ssh*
- *chmod 600 ~/.ssh/authorized_keys*
- *chown -R mahefa:mahefa ~/.ssh*

On décommente la ligne **PasswordAuthentification no** et le **PermitRootLogin no** dans le fichier **/etc/ssh/ssd_config** pour désactiver l'authentification par mot de passe ce qui garantit la sécurité du serveur que seul ce qui a la clé privée peut s'y connecter avec l'utilisateur non root.

### c. Installation de Docker et Docker Compose

Docker a été installé sur le serveur via le dépôt officiel :

Avant l'installation, tous les paquets en conflit ont été supprimés avec la commande

*sudo apt remove $(dpkg --get-selections docker.io docker-compose docker-compose-v2 docker-doc podman-docker containerd runc | cut -f1)*

Ensuite, le dépôt officiel Docker a été ajouté, puis les paquets installés

La figure 16 montre l'ajout du dépôt officiel de docker

```
# Add Docker's official GPG key:
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VER
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
```

*Figure 16: commande de configuration du dépôt docker*

Voiçi la commande d'installation de la dernière version de docker et de docker-compose

*sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin*

### d. Configuration du pare-feu (UFW)

Le pare-feu UFW a été configuré pour autoriser uniquement les ports nécessaires, le port 22 pour la connexion ssh, le port 80 pour la requête HTTP et le port 443 pour HTTPS avec ces commandes :

*sudo ufw allow 80*

*sudo ufw allow 443*

*sudo ufw enable*

### e. Configuration du répertoire de projet

Le projet est déployé dans le dossier /opt/panne-2025, choisi pour sa vocation système :

- /opt est réservé aux applications tierces installées manuellement
- Cela évite de polluer /home ou /var/www et facilite la maintenance
- Ce choix est cohérent avec les standards Linux pour les déploiements applicatifs

Voici les commandes nécessaires à la création et configuration du dossier de projet :

*sudo mkdir -p /opt/panne-2025*

On donne le droit à l'utilisateur **mahefa** pour qu'il puisse accédez au dossier

*sudo chown mahefa:mahefa /opt/panne-2025*

### f. Gestion des secrets et fichiers sensibles

On créer ces fichiers manuellement dans le répertoire du projet sur le vps parce que ce sont des fichiers sensible à ne jamais versionné dans le « repositories github ».

### *f.1. Mise en place du fichier.env.global*

Ce fichier contient toutes les variables d'environnement partagées entre les services.

La figure 17 montre la configuration du fichier .env.global pour la production

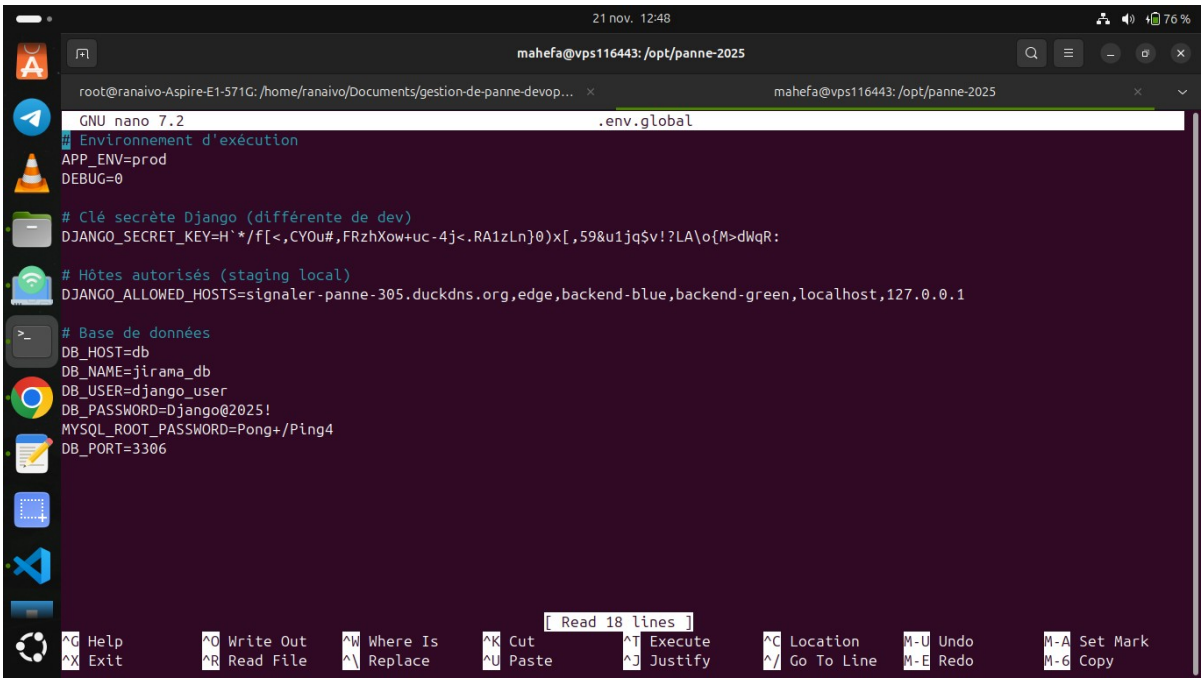



*Figure 17: fichier .env.global pour la production*

**Bonnes pratiques appliquées :**

- Fichier exclu du versioning via .gitignore
- Permissions restreintes : chmod 600 .env.global
- Centralisation des variables pour éviter la duplication

### *f.2. Mise en place du fichier.my-exporter.cnf*

Ce fichier est utilisé par mysql-exporter pour accéder à la base de données avec l'utilisateur spécifique.

La figure 18 montre le contenue du fichier my-exporter.cnf

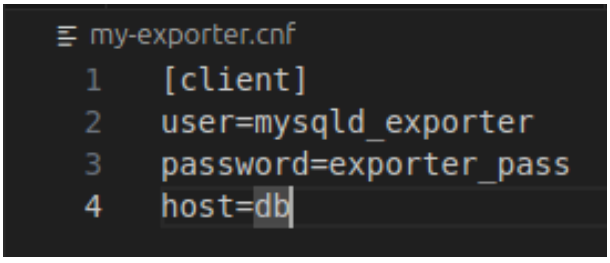



*Figure 18: contenue du fichier my-exporter.cnf*

Après la configuration du fichier, on a créé l'utilisateur dédié qui est décrivez dans le fichier à la base de données mysql avec des droits limités pour qu'ils ne peut pas faire autre choses que la

collecte des métriques. C'est une bonne pratique de sécurité pour bien protéger la base de données en limitant l'accès multi-utilisateurs dans la base.

La figure 19 montre la création et configuration de l'utilisateur mysql dédié pour la collecte des métriques.

```
root@ranaivo-Aspire-E1-571G:/home/ranaivo/stage-L3-update-farany# docker exec -it db-global mysql -uroot -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 4
Server version: 10.11.15-MariaDB-ubu2204 mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE USER 'mysqld_exporter'@'%' IDENTIFIED BY 'exporter_pass';
Query OK, 0 rows affected (0.304 sec)

MariaDB [(none)]> GRANT PROCESS, REPLICATION CLIENT, SELECT ON *.* TO 'mysqld_exporter'@'%';
Query OK, 0 rows affected (0.039 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.001 sec)

MariaDB [(none)]>
```

*Figure 19: création de l'utilisateur mysql-exporter*

#### *f.3. Sécurisation de l'accès à prometheus et grafana avec .htpasswd*

Ce Fichier est généré avec l'outil **htpasswd** pour créer un utilisateur avec un mot de passe fortement hashé pour sécuriser l'accès à Grafana et Prometheus.

On le créer avec cette commande

```
htpasswd -c .htpasswd mahefa
```

### g. Mise en place du certificat SSL avec DuckDNS et Certbot

Pour garantir la confidentialité des échanges entre les utilisateurs et l'application, il est essentiel d'activer le protocole HTTPS. Cette section décrit la mise en place d'un certificat SSL gratuit via **Let's Encrypt**, en utilisant **Certbot** et un domaine dynamique fourni par DuckDNS.

#### *g.1. Configuration du domaine DuckDNS*

DuckDNS est un service gratuit de DNS dynamique, idéal pour les serveurs auto-hébergés ou les VPS avec IP fixe. Le domaine choisi pour ce projet est ***signaler-panne-305.duckdns.org***

La figure 20 illustre la configuration de la liaison de l'IPV4 publique du serveur avec le nom de domaine dans duckDns facilitant aisi la visibilité de l'application dans le web.

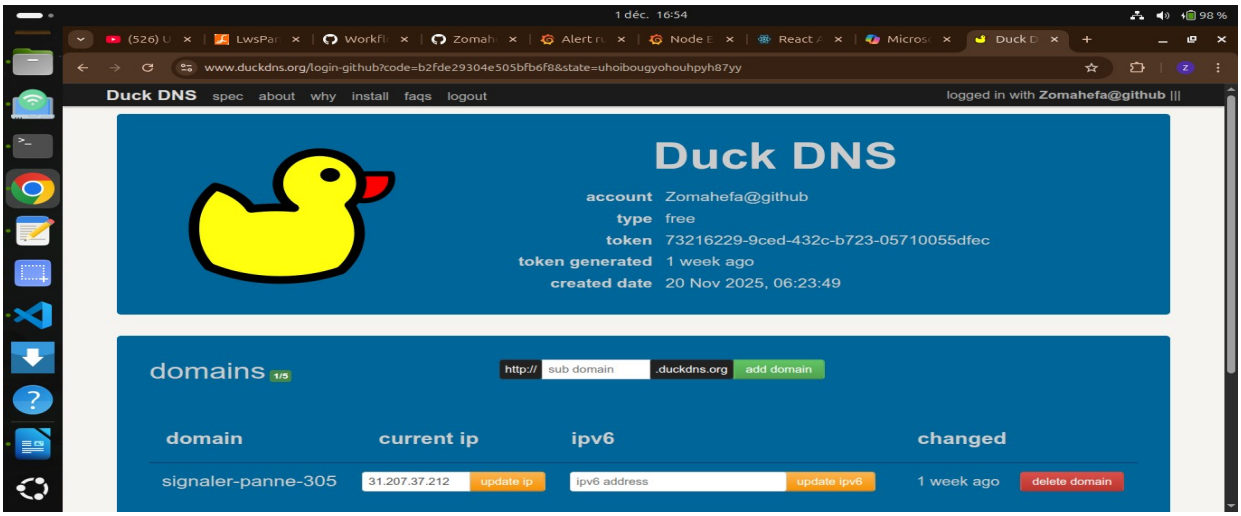


*Figure 20: Configuration dns du serveur*

### *g.2. Installation de Certbot sur le serveur*

Certbot est l'outil officiel recommandé par Let's Encrypt pour obtenir et renouveler automatiquement les certificats SSL [19]. Selon l'Electronic Frontier Foundation, Certbot automatise entièrement le processus d'obtention et de renouvellement des certificats, garantissant une sécurité HTTPS sans intervention manuelle [19].

Étapes d'installation :

- Création d'un environnement virtuel Python dédié :

  sudo python3 -m venv /opt/certbot/

- Mise à jour de pip :

  sudo /opt/certbot/bin/pip install --upgrade pip

- Installation de Certbot et du plugin Nginx :

  sudo /opt/certbot/bin/pip install certbot certbot-nginx

- Création d'un lien symbolique pour l'exécutable :

  sudo ln -s /opt/certbot/bin/certbot /usr/bin/certbot

### *g.3. Obtention du certificat SSL*

Le certificat est généré avec la méthode standalone, qui lance un serveur temporaire sur le port 80 pour valider le domaine avec la commande :

sudo certbot certonly --standalone -d signaler-panne-305.duckdns.org

Pour vérifiez le certificat,on vérifie le repertoire de letsenscrypt avec la commande :

ls -l /etc/letsencrypt/live/signaler-panne-305.duckdns.org

La figure 21 montre la génération de certificat et le contenu du repertoire letsenscrypt

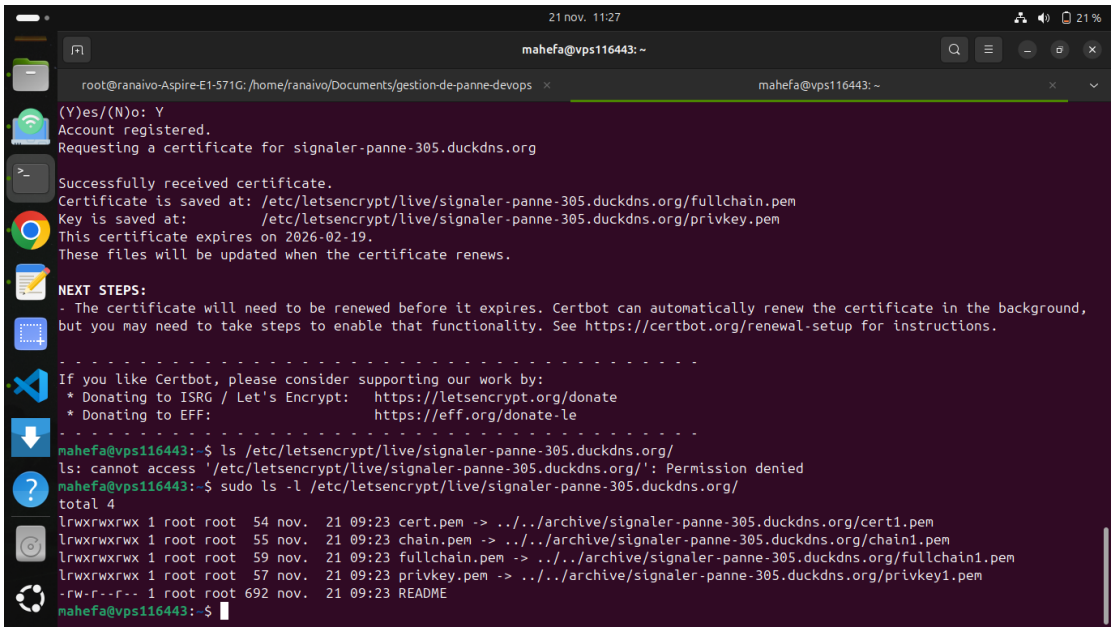


*Figure 21: vérification du répertoire letsenscrypt*

### *g.4. Renouvellement automatique du certificat*

Les certificats Let's Encrypt sont valides 90 jours. Pour éviter toute expiration, une tâche cron est ajoutée avec la commande :

```
sudo crontab -e
```

Pour permettre l'éxecution de la commande de renouvellement tous les 3 heures du matin on ajoute ce ligne à la fin du fichier qui s'ouvre automatiquement après la création du tache cron.

```
0 3 * * * /usr/bin/certbot renew –quiet
```

### 6.4.3. Mise en place de la conteneurisation

#### a. Objectif de la conteneurisation en production

L'objectif de cette section est de présenter l'architecture conteneurisée mise en place pour l'environnement de production. Elle repose sur Docker Compose multi-fichiers, avec une séparation claire des services : backend, frontend, base de données, proxy edge, monitoring. Chaque composant est encapsulé dans un conteneur dédié, configuré pour la performance, la sécurité et la scalabilité.

#### b. Création manuelle des volumes externes

Avant de lancer les services, les volumes Docker déclarés comme external dans les fichiers Compose doivent être créés manuellement sur le serveur. Cela garantit la persistance des données et évite les erreurs de démarrage.

La figure 22 montre la création des volumes sur le serveur

```
mahefa@vps116443:~$ docker network create panne-jirama-blue
e83edf1d4045b4113c6aa71c50397914a8634b16d87fbed3367be616ba57eea6
mahefa@vps116443:~$ docker network create panne-jirama-green
640829c906ac44ef12493a074d7b04d2e67c6449abf13de0223b337fc6080346
mahefa@vps116443:~$ docker volume create jirama-django-media
jirama-django-media
mahefa@vps116443:~$ docker volume create jirama-django-static
jirama-django-static
mahefa@vps116443:~$ docker volume create mysql-panne-jirama
mysql-panne-jirama
mahefa@vps116443:~$ docker volume create grafana-data-jirama
grafana-data-jirama
mahefa@vps116443:~$ cd /opt/panne-2025/
mahefa@vps116443:/opt/panne-2025$ nano .env.global
mahefa@vps116443:/opt/panne-2025$ nano .my.cnf
mahefa@vps116443:/opt/panne-2025$ chmod 600 /opt/panne-2025/.env.global
mahefa@vps116443:/opt/panne-2025$ chmod 600 /opt/panne-2025/.my.cnf
```

*Figure 22: création des volumes dans le VPS*

### c. Conteneurisation

La phase de production repose sur la même base de conteneurisation que celle décrite dans la phase de staging. Les Dockerfile utilisés pour le backend (Django) et le frontend (React) sont strictement identiques, incluant les fichiers associés tels que :

- entrypoint.sh : script d'initialisation du conteneur backend
- .dockerignore : exclusion des fichiers inutiles lors du build

Cette approche garantit une cohérence totale entre les environnements de staging et de production, réduisant les risques de divergence de comportement entre les phases de test et de mise en production.

### d. Mise en place des fichiers docker-compose pour l'orchestration

On a structuré le déploiement en plusieurs fichiers Docker Compose distincts pour garantir une isolation claire, une maintenabilité accrue et une alignement précis avec la stratégie de déploiement blue/green. Chaque fichier correspond à un domaine fonctionnel spécifique

#### *d.1. Mise en place de docker-compose.blue.yml*

Voici les configurations qu'on a mis en place :

- Le service frontend dépend du backend via `depends_on` avec `condition service_healthy` celà garantit que les services démarre en ordre et sont toujours saines ;
- Les volumes statiques et médias sont montés pour être partagé aux serveur nginx edge pour qu'il sert les fichier media et static de django pour une meilleur performance

Voici le contenue du fichier docker-compose.blue.yml

```
services:
  backend-blue:
    image: zomahefaranaivo/django-backend-panne-prod:latest
    container_name: backendBlue
    restart: always
    environment:
      APP_ENV: ${APP_ENV}
      DEBUG: ${DEBUG}
      DJANGO_SECRET_KEY: ${DJANGO_SECRET_KEY}
      DJANGO_ALLOWED_HOSTS: ${DJANGO_ALLOWED_HOSTS}
      DB_HOST: ${DB_HOST}
      DB_NAME: ${DB_NAME}
      DB_USER: ${DB_USER}
      DB_PASSWORD: ${DB_PASSWORD}
      DB_PORT: ${DB_PORT}
    volumes:
      - django_static:/back/staticfiles
      - django_media:/back/media
    networks:
      - panne-jirama-blue
  frontend-blue:
    image: zomahefaranaivo/react-frontend-panne-prod:latest
    container_name: frontendBlue
    restart: always
    expose:
      - "80"
    depends_on:
      backend-blue:
        condition: service_healthy
    networks:
      - panne-jirama-blue
```

```
volumes:
  django_static:
    external: true
    name: jirama-django-static
  django_media:
    external: true
    name: jirama-django-media
networks:
  panne-jirama-blue :
    external: true
    name: panne-jirama-blue
```

### *d.2. Mise en place de docker-compose.green.yml*

Elle contient la même configuration que blue mais seuls les noms de services, du volume et des réseaux sont différent avec frontend-green et backend-green comme service et panne-jirama-green comme réseau.

### *d.3. Mise en place de docker-compose.db.yml*

Ce fichier démarre MariaDB avec un healthcheck pour garantir sa disponibilité. La base est partagée entre les deux environnements (blue/green) via les réseaux Docker. Le volume *mysql_data_production* assure la persistance.

Voici la configuration de docker-compose.db.yml

```
services:
  db:
    image: mariadb:10.11
    container_name: db-global
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${DB_NAME}
      MYSQL_USER: ${DB_USER}
      MYSQL_PASSWORD: ${DB_PASSWORD}
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-uroot", "-p$
{MYSQL_ROOT_PASSWORD}"]
      interval: 30s
      timeout: 10s
      retries: 10
      start_period: 40s
    volumes:
      - mysql_data_production:/var/lib/mysql
    networks:
      - panne-jirama-blue
      - panne-jirama-green
volumes:
  mysql_data_production:
    name: mysql_data_production
networks:
  panne-jirama-blue:
    external: true
    name: panne-jirama-blue
  panne-jirama-green:
    external: true
    name: panne-jirama-green
```

### *d.4. Mise en place du docker-compose.edge.yml*

Ce fichier lance le proxy Nginx, qui agit comme point d'entrée unique. Il redirige les requêtes vers les services actifs (blue ou green) via les fichiers ***frontend_active.conf*** **et** ***backend_active.conf*** dans le dossier *conf.d* qui est modifiés dynamiquement par le pipeline CD.

```
services:
  edge:
    image: nginx:1.25-alpine
    container_name: edge
    restart: always
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
      - ./conf.d:/etc/nginx/conf.d:rw
      - /etc/letsencrypt:/etc/letsencrypt:ro
      - django_static:/back/staticfiles
      - django_media:/back/media
      - ./nginx-secure:/etc/nginx/secure:ro
    networks:
      - panne-jirama-blue
      - panne-jirama-green
volumes:
  django_media:
    external: true
    name: jirama-django-media
  django_static:
    external: true
    name: jirama-django-static
networks:
  panne-jirama-blue:
    name: panne-jirama-blue
  panne-jirama-green:
    name: panne-jirama-green
```

- **Configuration du fichier nginx.conf**

Le fichier nginx.conf utilisé en production est monté en volume dans le conteneur edge, défini dans le fichier docker-compose.edge.yml. Ce conteneur joue le rôle de **reverse proxy sécurisé**, exposant les ports 80 et 443, et intégrant les certificats TLS générés par Certbot.

Voici la configuration du fichier nginx.conf pour la production

```
events {}
http {
 include       mime.types;
 default_type  application/octet-stream;
 upstream frontend {
  include /etc/nginx/conf.d/frontend_active.conf;  #Fichier généré dynamiquement
 }
 upstream backend {
  include /etc/nginx/conf.d/backend_active.conf;   #Fichier généré dynamiquement
 }
 server {
  listen 80;
  server_name signaler-panne-305.duckdns.org;
  return 301 https://$host$request_uri;  # Redirection permanente
 }
 server {
  add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
  listen 443 ssl;
  server_name signaler-panne-305.duckdns.org;
  ssl_certificate /etc/letsencrypt/live/signaler-panne-305.duckdns.org/fullchain.pem;
  ssl_certificate_key /etc/letsencrypt/live/signaler-panne-305.duckdns.org/privkey.pem;
  resolver 127.0.0.11 ipv6=off;  # Résolution DNS interne Docker
  location / {
   proxy_pass http://frontend;
   proxy_set_header Host $host;
   proxy_set_header X-Real-IP $remote_addr;
   proxy_redirect off;
   proxy_intercept_errors on;
   error_page 404 = /index.html;  #SPA routing
  }
```

```
location ~ ^/api(/|$) {
  proxy_pass http://backend;
  proxy_set_header Host $host;
  proxy_set_header X-Real-IP $remote_addr;
  proxy_redirect off;
}
location /django-static/ {
  alias /back/staticfiles/;
  access_log off;
  expires 30d;
  add_header Cache-Control "public";
}
location /media/ {
  alias /back/media/;
  access_log off;
  expires 30d;
  add_header Cache-Control "public";
}
location ~ ^/grafana(/|$) {
  auth_basic "Accès Monitoring Réservé";
  auth_basic_user_file /etc/nginx/secure/.htpasswd;
  allow 129.222.109.160;    # IP AUTORISÉ
  deny all;
  proxy_pass http://grafana:3000;
  proxy_set_header Host $host;
  proxy_set_header X-Real-IP $remote_addr;
  proxy_redirect off;
}
```

```
 location ~ ^/prometheus(/|$) {
     auth_basic "Accès Monitoring Réservé";
     auth_basic_user_file /etc/nginx/secure/.htpasswd;  # CHEMOD CORRIGÉ
     allow 129.222.109.160;
     deny all;            # Bloquer toutes les autres IPs
     proxy_pass http://prometheus:9090;
     proxy_set_header Host $host;
     proxy_set_header X-Real-IP $remote_addr;
     proxy_redirect off;
    }
    location = /metrics/ {
     proxy_pass http://backend;
     proxy_set_header Host localhost;
     proxy_set_header X-Real-IP $remote_addr;
     proxy_redirect off;
    }
  }
}
```

#### 6.4.4. Mise en place du CI/CD avec GitHub Actions : automatisation complète

Afin de garantir une livraison continue fiable et reproductible, le projet intègre une chaîne CI/CD automatisée à l'aide de GitHub Actions. Cette automatisation permet de valider le code, de construire les images Docker, puis de déployer les services dans l'environnement de production sans intervention manuelle. Elle s'inscrit dans une logique DevOps visant à réduire les erreurs humaines, accélérer les cycles de livraison et renforcer la qualité logicielle.

Pour structurer cette automatisation, deux workflows distincts ont été mis en place.

Les fichiers ci.yml (Continuous Integration) et cd.yml (Continuous Deployment) ont été délibérément séparés afin de clarifier les responsabilités de chaque étape du pipeline :

- ci.yml est dédié à la vérification de la qualité du code : exécution des tests, analyse statique, linting, et build des images Docker.
- cd.yml est déclenché uniquement après validation de la CI, et prend en charge le déploiement automatisé dans l'environnement de production.

Cette séparation permet une meilleure lisibilité, une maintenance facilitée, et offre la possibilité de déclencher manuellement ou conditionnellement le déploiement, ce qui est essentiel pour un contrôle précis en production

La figure 23 montre l'architecture générale d'un CI/CD

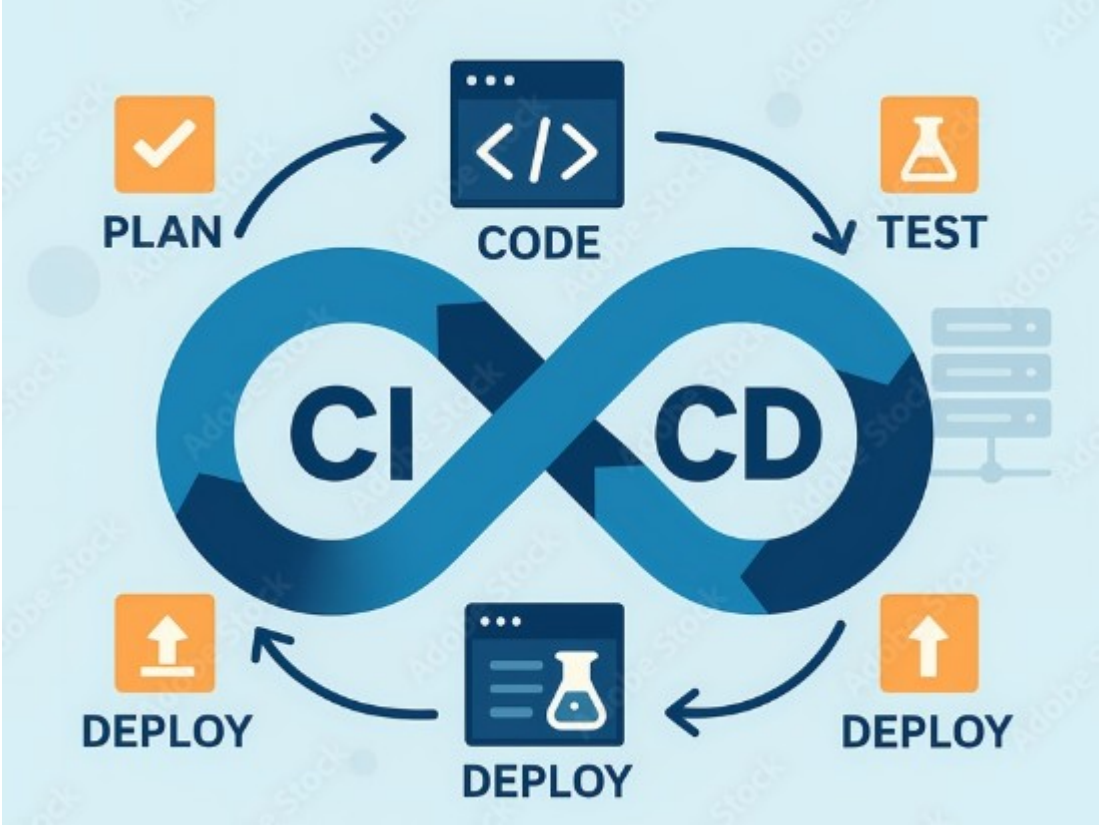



*Figure 23: Architecture d'un CI/CD*

### a. Configuration des secrets GitHub

Avant de mettre en place les workflows CI/CD, il est essentiel de configurer les secrets et variables dans GitHub pour sécuriser les accès et rendre le pipeline dynamique.

Secrets configurés :

- DOCKERHUB_TOKEN : jeton d'accès DockerHub
- VPS_SSH_KEY : clé privée SSH pour se connecter au serveur
- VPS_HOST : IP publique du serveur VPS
- VPS_USER : utilisateur SSH du VPS
- VPS_PROJECT_DIR : chemin du projet sur le serveur (ex. /opt/panne-2025)
- DUCKDNS_DOMAIN : domaine public utilisé pour le déploiement

La figure 24 montre l'ensemble des secrets stocker dans github :

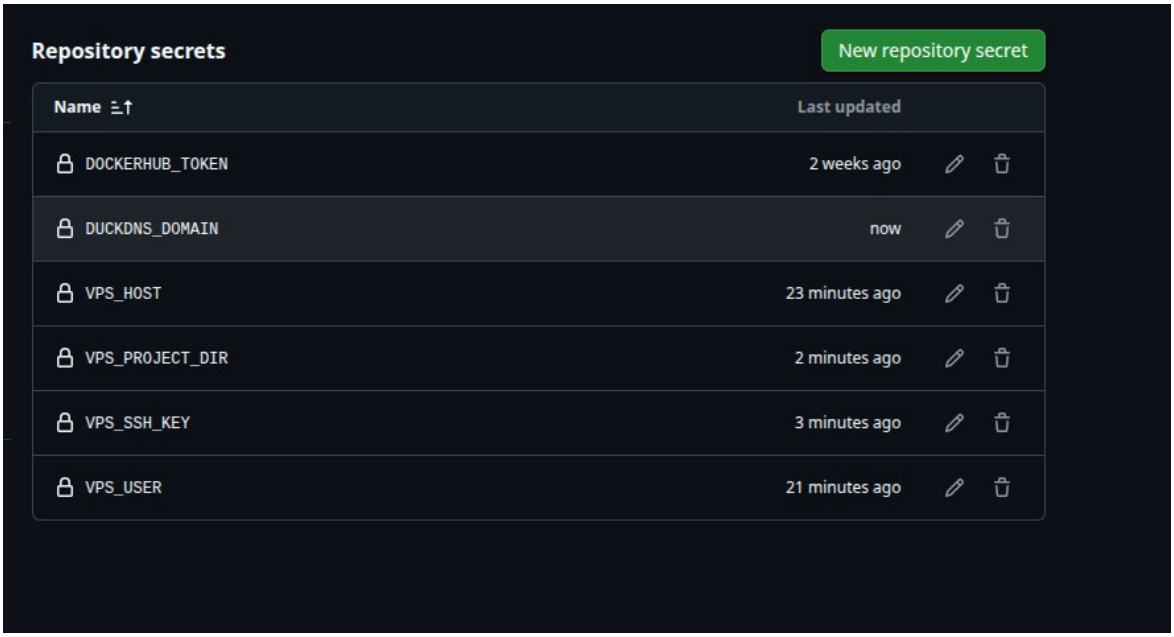



*Figure 24: Liste de secrets mise en place dans github*

### b. Mise en place du Ci Pipeline : qualité, sécurité, fiabilité

Le fichier ci.yml est déclenché à chaque *push* ou *pull_request* sur la branche. Il contient quatre jobs : **Lint**, **Test**, **Security**, **Dockerhub**. Toute sa configuration est équivalent à celle du fichier staging.yml sauf sur le condition de déclenchement.

Voici configuration du condition de déclenchement du ci.yml

```
name: CI Pipeline

on:

  push:

    branches: [ "main" ]

  pull_request:

    branches: [ "main"]
```

**Stratégie de tag : version incrémentale + latest**

Pour assurer à la fois la traçabilité des versions et la fluidité du déploiement, deux tags sont appliqués à chaque image.

*Tableau 21: les tags des images docker*

| Tag | Rôle | Exemple |
|---|---|---|
| v${{ github.run_number }} | Tag unique et incrémental à chaque exécution du workflow CI | v42 |
| latest | Tag constant utilisé par les fichiers docker-compose.blue.yml et docker-compose.green.yml pour toujours récupérer la dernière version | latest |

- Le tag v${{ github.run_number }} permet de retrouver précisément quelle version a été générée à quel moment, utile pour les audits, les rollback manuels ou les tests spécifiques.
- Le tag latest est utilisé dans les fichiers Compose pour garantir que le déploiement en production utilise toujours la dernière image à jour validée par le pipeline CI.

### c. Mise en place du CD Pipeline : déploiement blue/green

Le fichier cd.yml est déclenché automatiquement si il y a succès du CI Pipeline. Il déploie la nouvelle version sur le serveur VPS avec une stratégie blue/green, incluant tests santé et rollback.

```
name: CD blue/green

on:

  workflow_run:

    workflows: ["CI Pipeline"]

    types: [completed]
```

Il contient un seul job nommé **deploy** qui est responsable du déploiement sur le VPS.

#### *c.1. Connexion SSH sécurisée*

Utilisation de **webfactory/ssh-agent** pour injecter la clé privée SSH depuis GitHub Secrets.

Cela évite d'exposer le mot de passe et permet une connexion sécurisée.

#### *c.2. Synchronisation des fichiers*

Utilisation de **rsync** pour transférer uniquement les fichiers nécessaires pour démarrer l'application. Cela optimise la bande passante et évite les erreurs de configuration.

### *c.3. Mise en place du script de déploiement intelligent*

Le script bash dans cd.yml :

- Lit la couleur active depuis le fichier active_color.txt
- Déploie la couleur inactive avec docker compose up
- Exécute des tests santé avec docker exec curl
- Modifie les fichiers frontend_active.conf et backend_active.conf
- Recharge Nginx avec nginx -s reload pour appliquer les modifications
- Vérifie plusieurs fois dans un boucle que le site répond en 200 sinon ça revérifie 6 fois
- Si échec  lors des six tentatives de vérification, il y a rollback automatique vers l'ancienne version, celà garantit la haute disponibilité même en cas d'erreur
- Utilise la commande echo pour la clarté et traçabilité des étapes exécuté

### *c.4. Nettoyage de l'ancienne version*

Le workflow utilise cette commande pour arrêter la version ancienne :

```
docker compose -f docker-compose.${ACTIVE_COLOR}.yml --env-file .env.global down
```

Cela permet de libérer les ressources de la couleur précédente, tout en gardant la base et le monitoring actifs.

Voici le contenue du job deploy

```
 - name: Déployer couleur inactive, tester, basculer
     run: |
       ssh -o StrictHostKeyChecking=no ${{ secrets.VPS_USER }}@$
{{ secrets.VPS_HOST }} << 'EOF'
       set -e
       cd ${{ secrets.VPS_PROJECT_DIR }}
       # --- Définition des variables ---
       ACTIVE_COLOR=$(cat active_color.txt 2>/dev/null || echo "blue")
       if [ "$ACTIVE_COLOR" = "blue" ]; then
         TARGET_COLOR="green"
         TARGET_COMPOSE="docker-compose.green.yml"
         INACTIVE_FE="frontendGreen"
         INACTIVE_BE="backendGreen"
         PREVIOUS_FE="frontendBlue"
         PREVIOUS_BE="backendBlue"
         SERVICE_INACTIVE_FE="frontend-green"
         SERVICE_INACTIVE_BE="backend-green"
         SERVICE_ACTIVE_FE="frontend-blue"
         SERVICE_ACTIVE_BE="backend-blue"
       else
         TARGET_COLOR="blue"
         TARGET_COMPOSE="docker-compose.blue.yml"
         INACTIVE_FE="frontendBlue"
         INACTIVE_BE="backendBlue"
         PREVIOUS_FE="frontendGreen"
         PREVIOUS_BE="backendGreen"
         SERVICE_INACTIVE_FE="frontend-blue"
         SERVICE_INACTIVE_BE="backend-blue"
         SERVICE_ACTIVE_FE="frontend-green"
         SERVICE_ACTIVE_BE="backend-green"
       fi
       echo "Couleur active: $ACTIVE_COLOR ; Inactive à déployer:
$TARGET_COLOR"
       echo "Démarrage des services statiques ---"
       docker compose -f docker-compose.db.yml --env-file .env.global up -d –
no-recreate



       echo "Nettoyage:Arrêt et suppression de l'ancienne couleur
($ACTIVE_COLOR)..."
       docker compose -f docker-compose.${ACTIVE_COLOR}.yml --env-
file .env.global down
       EOF
```

```
 docker compose -f docker-compose.monitoring.yml up -d --no-recreate
 echo "Démarrage de la nouvelle couleur ---"
 docker compose -f $TARGET_COMPOSE --env-file .env.global pull
 docker compose -f $TARGET_COMPOSE --env-file .env.global up -d --wait
 docker compose -f docker-compose.edge.yml up -d --no-recreate
 # --- TESTS DE SANTÉ AVEC DOCKER EXEC ---
 echo "Tests santé avec docker exec..."
 # Installation curl dans edge
 docker exec edge sh -c "apk add --no-cache curl >/dev/null 2>&1 || true"
 echo "Test backend ($INACTIVE_BE):"
 docker exec $INACTIVE_BE curl -sSf http://localhost:8000/health/ || exit 1
 echo "Test frontend ($INACTIVE_FE):"
 docker exec $INACTIVE_FE curl -sSf http://localhost/ | head -n 1 || exit 1
echo "Bascule: modification des fichiers upstream---"
       echo "server $SERVICE_INACTIVE_FE:80;" > conf.d/frontend_active.conf
       echo "server $SERVICE_INACTIVE_BE:8000;" > conf.d/backend_active.conf
echo "Vérification des fichiers modifiés"
       cat conf.d/frontend_active.conf
       cat conf.d/backend_active.conf
       echo "Rechargement de Nginx ---"
       sleep 5
       docker exec edge nginx -t
       docker exec edge nginx -s reload
       echo "$TARGET_COLOR" > active_color.txt
       echo "Vérification post-bascule ---"
       MAX_RETRIES=6
       RETRY_COUNT=0
       while [ $RETRY_COUNT -lt $MAX_RETRIES ]; do
         if curl -sSf -I https://signaler-panne-305.duckdns.org | grep -q '200';
then
           break
         fi
         RETRY_COUNT=$((RETRY_COUNT + 1 ))
         sleep 5
       done
       echo "tentative de roolback"
       if [ $RETRY_COUNT -eq $MAX_RETRIES ]; then
         echo "server $SERVICE_ACTIVE_FE:80;" > conf.d/frontend_active.conf
         echo "server $SERVICE_ACTIVE_BE:8000;" > conf.d/backend_active.conf
         docker exec edge nginx -s reload
         #echo "$ACTIVE_COLOR" > active_color.txt
         echo "Rollback vers $ACTIVE_COLOR effectué. Le déploiement a échoué."
         exit 1
```

```
    else
      echo "Toute est OK,roolback annulé"
    fi
    echo "Déploiement vers $TARGET_COLOR réussi."
    echo "Nettoyage:Arrêt et suppression de l'ancienne couleur
($ACTIVE_COLOR)..."
    docker compose -f docker-compose.${ACTIVE_COLOR}.yml --env-
file .env.global down
    EOF
```

### 6.4.5. Mise en place du monitoring et de la supervision

Cette section présente la configuration complète du système de monitoring et de supervision de l'application. L'objectif est de garantir une visibilité en temps réel sur l'état de santé de l'infrastructure, de détecter rapidement les anomalies, et de faciliter le diagnostic en cas d'incident. La stack de monitoring repose sur Prometheus, Grafana, Loki, Promtail, Node Exporter et MySQL Exporter.

#### a. Architecture de supervision

Tous les services de monitoring sont définis dans le fichier docker-compose.monitoring.yml. Ils sont connectés aux réseaux panne-jirama-blue et panne-jirama-green pour accéder à l'ensemble des conteneurs, quelle que soit la couleur active.

Services inclus :

- **Prometheus** : collecte des métriques
- **Grafana** : visualisation des métriques via des dashboards
- **Loki** : agrégation des logs
- **Promtail** : collecte et envoi des logs vers Loki
- **MySQL Exporter** : exposition des métriques de la base de données
- **Node Exporter** : exposition des métriques système (CPU, RAM, disque)

Voici la configuration mis en place pour le docker-compose.monitoring.yml

```
services:
  prometheus:
    image: prom/prometheus
    container_name: prometheus-monitoring
    volumes:
      - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml
    command:
      - '--config.file=/etc/prometheus/prometheus.yml'
      - '--web.external-url=https://signaler-panne-305.duckdns.org/prometheus/'
      - '--web.route-prefix=/prometheus'  #  Préfixe pour le routing Nginx
      - '--web.enable-lifecycle'  # Permet de recharger la config sans redémarrer
    depends_on:
      - mysql-exporter
    networks:
      - panne-jirama-blue
      - panne-jirama-green
  grafana:
    image: grafana/grafana
    container_name: grafana-monitoring
    ports:
      - "3000:3000"  #  ACCÈS DIRECT: Permet l'accès via IP:3000 si Nginx est down
    environment:
      - GF_SERVER_ROOT_URL=https://signaler-panne-305.duckdns.org/grafana/
      - GF_SERVER_SERVE_FROM_SUB_PATH=true  #  Support des sous-chemins
      - GF_SECURITY_ADMIN_PASSWORD=Admin123!
      - GF_USERS_ALLOW_SIGN_UP=false  #  Désactive l'inscription publique
      - GF_AUTH_ANONYMOUS_ENABLED=false  #  Désactive l'accès anonyme
    volumes:
      - grafana_data:/var/lib/grafana
    depends_on:
      - prometheus
      - loki
    networks:
      - panne-jirama-blue
      - panne-jirama-green
```

```
loki:
    image: grafana/loki:2.9.4
    container_name: loki-monitoring
    command: -config.file=/etc/loki/local-config.yaml
    networks:
      - panne-jirama-blue
      - panne-jirama-green
  promtail:
    image: grafana/promtail:3.5.8
    container_name: promtail-monitoring
    volumes:
      - /var/lib/docker/vfs/containers:/var/lib/docker/vfs/containers:ro
      - /var/log:/var/log:ro  # Lecture seule des logs système
      - ./monitoring/promtail-config.yml:/etc/promtail/config.yml
      - /var/run/docker.sock:/var/run/docker.sock:ro
    command: -config.file=/etc/promtail/config.yml
    depends_on:
      - loki
    networks:
      - panne-jirama-blue
      - panne-jirama-green
  mysql-exporter:
    image: prom/mysqld-exporter
    container_name: mysql-exporter-monitoring
    restart: always
    command:
      - '--config.my-cnf=/etc/my-exporter.cnf'
    volumes:
      - ./my-exporter.cnf:/etc/my-exporter.cnf:ro
    networks:
      - panne-jirama-blue
      - panne-jirama-green
```

```
 node-exporter:
    image: prom/node-exporter
    container_name: node-exporter-monitoring
    privileged: true
    volumes:
      - /proc:/host/proc:ro
      - /sys:/host/sys:ro
      - /:/rootfs:ro
    command:
      - '--path.procfs=/host/proc'
      - '--path.sysfs=/host/sys'
      - '--collector.filesystem.ignored-mount-points="^/(sys|proc|host|etc|dev|tmp|var/lib/docker/.
+)($|/)"'
    networks:
      - panne-jirama-blue
      - panne-jirama-green
volumes:
  grafana_data:  #Stockage persistant pour Grafana
networks:
  panne-jirama-blue:
    external: true
    name: panne-jirama-blue
  panne-jirama-green:
    external: true
    name: panne-jirama-green node-exporter:
    image: prom/node-exporter
    container_name: node-exporter-monitoring
    privileged: true
    volumes:
      - /proc:/host/proc:ro
      - /sys:/host/sys:ro
      - /:/rootfs:ro
    command:
      - '--path.procfs=/host/proc'
      - '--path.sysfs=/host/sys'
      - '--collector.filesystem.ignored-mount-points="^/(sys|proc|host|etc|dev|tmp|var/lib/docker/.
+)($|/)"'
    networks:
      - panne-jirama-blue
      - panne-jirama-green
volumes:
  grafana_data:  #Stockage persistant pour Grafana
networks:
  panne-jirama-blue:
    external: true
    name: panne-jirama-blue
```

```
  panne-jirama-green:
  external: true
  name: panne-jirama-green
```

### b. Configuration de Prometheus

Le fichier prometheus.yml dans le dossier monitoring définit les jobs de scraping :

- django-backend : via /metrics/ exposé par Django
- mysql : via mysql-exporter:9104
- node : via node-exporter:9100
- prometheus : auto-surveillance

Chaque job est enrichi avec des labels personnalisés (instance, color, service) pour faciliter le filtrage dans Grafana.

Voici la configuration de prometheus

```
global:
  scrape_interval: 15s
scrape_configs:
  - job_name: 'django-backend'
    metrics_path: /metrics/
    scheme: https
    static_configs:
      - targets: ['signaler-panne-305.duckdns.org']
    relabel_configs:
      - target_label: instance
        replacement: backend   #Remplace l'instance (qui serait le nom de domaine) par
"backend"
      - target_label: color
        replacement: active
  - job_name: 'mysql'
    metrics_path: /metrics
    static_configs:
      - targets: ['mysql-exporter:9104']
    relabel_configs:
      - target_label: instance
        replacement: mysql
  - job_name: 'node'
    static_configs:
      - targets: ['node-exporter:9100']
    relabel_configs:
      - target_label: instance
```

```
      replacement: serveur
    - target_label: color
      replacement: host
- job_name: 'prometheus'
  metrics_path: prometheus/metrics
  static_configs:
    - targets: ['prometheus:9090']
  relabel_configs:
    - target_label: instance
      replacement: prometheus
```

### c. Configuration de Promtail

Le fichier *promtail-config.yml* dans le dossier monitoring est configurée pour collecté des logs Docker et des logs système pour être envoyé à Loki et visualisez dans grafana.

Voici la configuration du fichier promtail-config.yml

```
clients:
  - url: http://loki:3100/loki/api/v1/push
server:
  http_listen_port: 9080
  grpc_listen_port: 0
positions:
  filename: /tmp/positions.yaml
scrape_configs:
  - job_name: docker-all
    static_configs:
      - targets: ['localhost']
        labels:
          job: 'docker'
          __path__: '/var/lib/docker/vfs/containers/*/*.log'
    pipeline_stages:
      - json:
          expressions:
            log: log
            stream: stream
            time: time
  - job_name: docker-dynamic
    docker_sd_configs:
      - host: unix:///var/run/docker.sock
    relabel_configs:
      - source_labels: [__meta_docker_container_name]
        target_label: container_name
      - source_labels: [__meta_docker_container_name]
        regex: .*Blue
        target_label: color
        replacement: blue
```

```
- source_labels: [__meta_docker_container_name]
      regex: .*Green
      target_label: color
      replacement: green
    - source_labels: [__meta_docker_container_name]
      regex: edge
      target_label: service
      replacement: nginx
    - source_labels: [__meta_docker_container_name]
      regex: (frontend.*)
      target_label: service
      replacement: react-frontend
    - source_labels: [__meta_docker_container_name]
      regex: (backend.*)
      target_label: service
      replacement: django-backend
    - source_labels: [__meta_docker_container_id]
      target_label: __path__
      replacement: /var/lib/docker/vfs/containers/${1}/*.log
  pipeline_stages:
    - json:
        expressions:
          log: log
          stream: stream
          time: time
```

### d. Mise en place de Grafana

#### *d.1. Configuration des dashboards*

Des dashboard personnalisés ont été créés dans Grafana pour visualiser :

- Les métriques système (via Node Exporter)
- Les performances de la base de données (via MySQL Exporter)
- Les métriques applicatives Django (requêtes, erreurs, temps de réponse)

- Les logs agrégés (via Loki)

Pour créer ces dashboard on a configuré deux source de données dans grafana qui sont prometheus et loki via les services prometheus-monitoring et loki-monitoring. Prometheus collecte les métriques système, base de données et du backend django grâce au fichier prometheus.yml. Tansdisque Loki c'est l'agrégeur les logs reçus venant de promtail grace à la fichier promtail-config.yml.

La figure 25 montre les sources de données créer

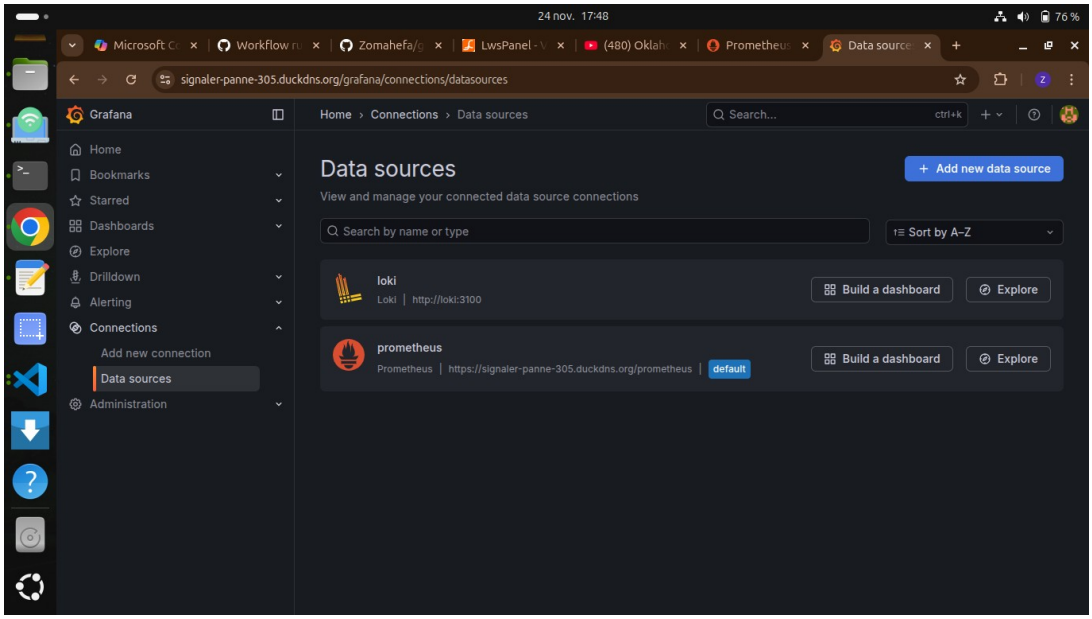



*Figure 25: source de données dans grafana*

Voici tous les dashboard utilisé pour la supervision complète de l'infrastructure

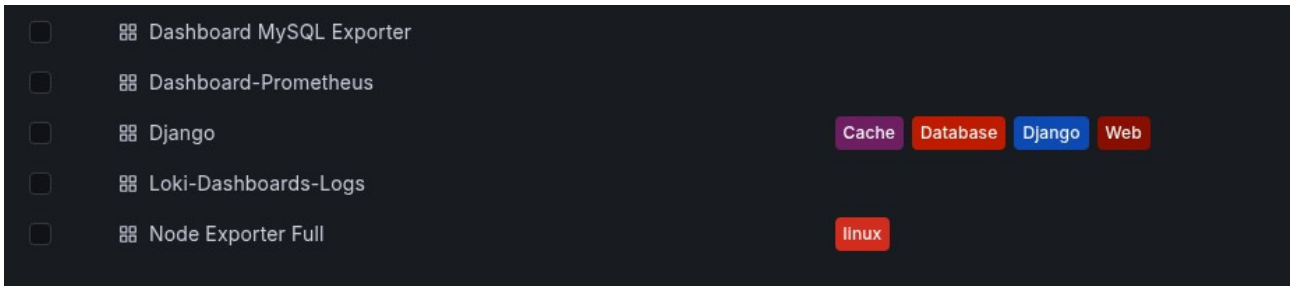



*Figure 26: dashboard grafana*

### *d.2. Configuration du contact point*

Dans Grafana, des alertes ont été configurées pour détecter :

- Un service en down dans Prometheus
- Une montée anormale de la charge CPU ou de la RAM
- Une absence de logs ou de métriques pendant un intervalle donné

Les alertes sont associées à un **Contact Point** configuré pour envoyer des notifications par email ou webhook,dans notre cas on a utilisé un webhook via slack une application multiplateforme avec une très belle interface pour mieux visualisez les alertes.

***Configuration du contact points slack***

Slack est une plateforme de communication collaborative largement utilisée dans les environnements DevOps. Elle permet de centraliser les notifications, les échanges techniques et les alertes en temps réel. Son intégration avec Grafana offre une solution simple, rapide et efficace pour recevoir les alertes critiques directement dans un canal dédié.

Slack a été choisi pour sa simplicité d'intégration, sa compatibilité avec les webhooks, et sa capacité à notifier instantanément les membres de l'équipe.

### *d.3. Création de l'application Slack*

Pour permettre à Grafana d'envoyer des alertes vers Slack, une application personnalisée a été créée via l'interface Slack API.

Étapes :

- Création de l'application "Grafana Alerts"
- Activation des **Incoming Webhooks**
- Génération d'un **Webhook URL** associé au canal #tous-grafana-alerts

La figure 27 montre la création de l'application « grafana alerts » dans slack

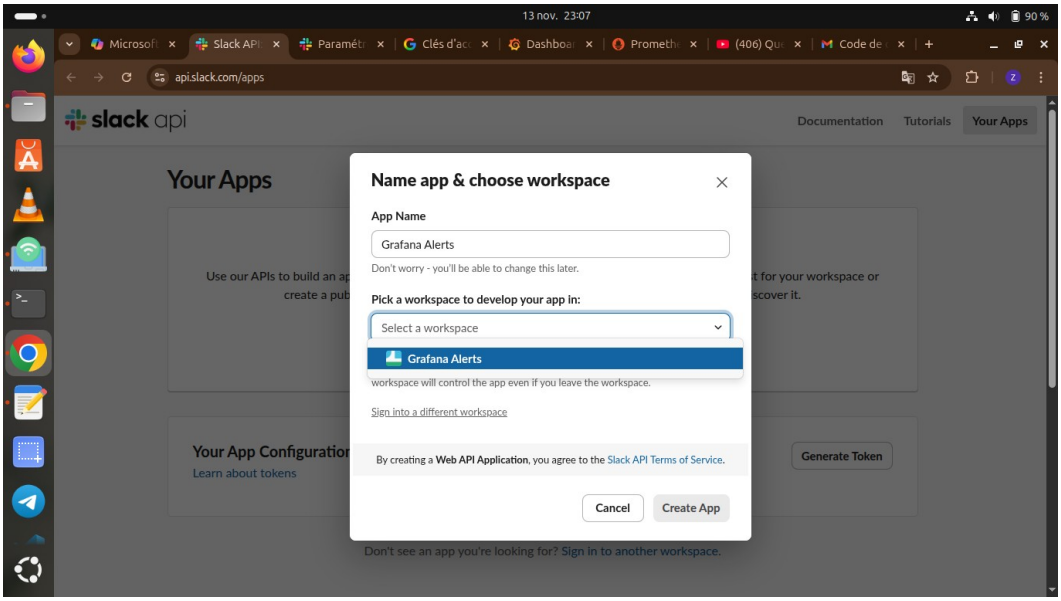



*Figure 27: création de l'application grafana alerts dans slack*

Le contact point dans grafana est configuré via webhook alors on a besoin d'activé et créer un webhook dans slack.

La figure 28 montre la création d'un webhook dans slack

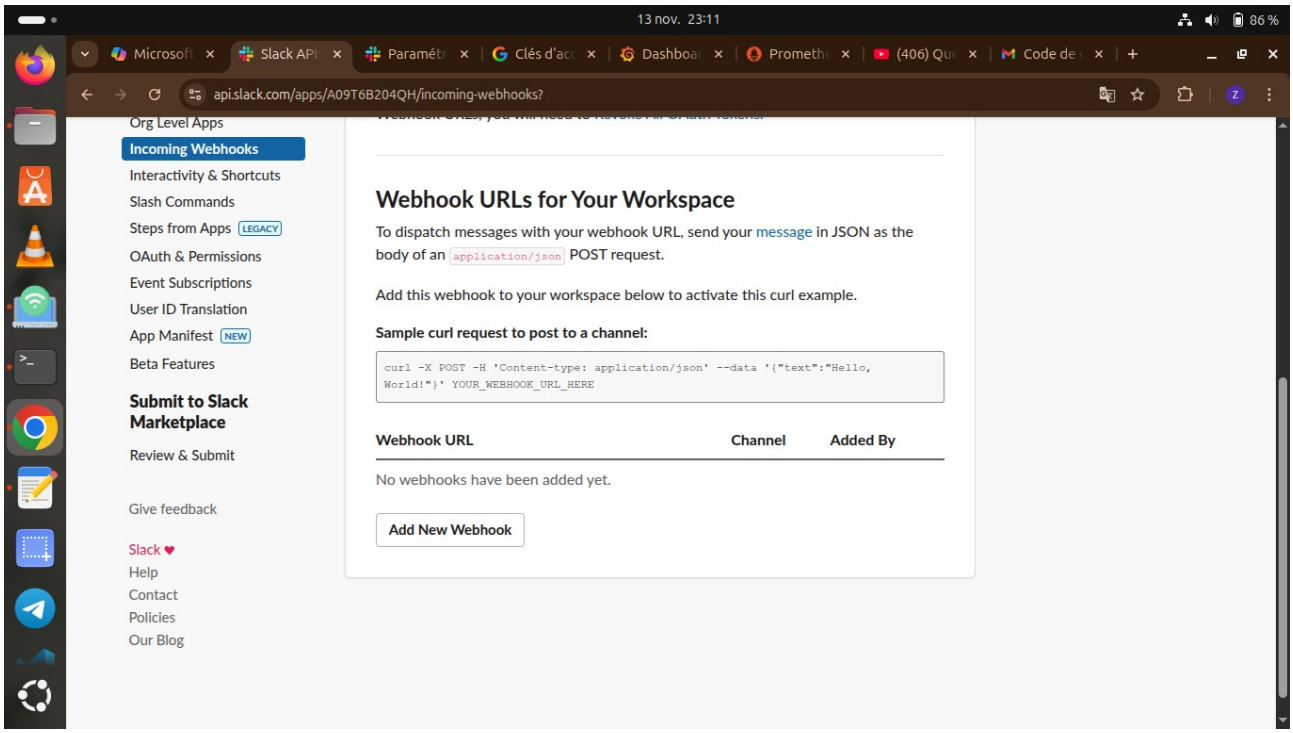



*Figure 28: création d'un webhook dans slack*

Puis installez l'application et le canal pour terminer la configuration de slack

La figure 29 montre l'installation de l'application « grafana alerts » dans slack

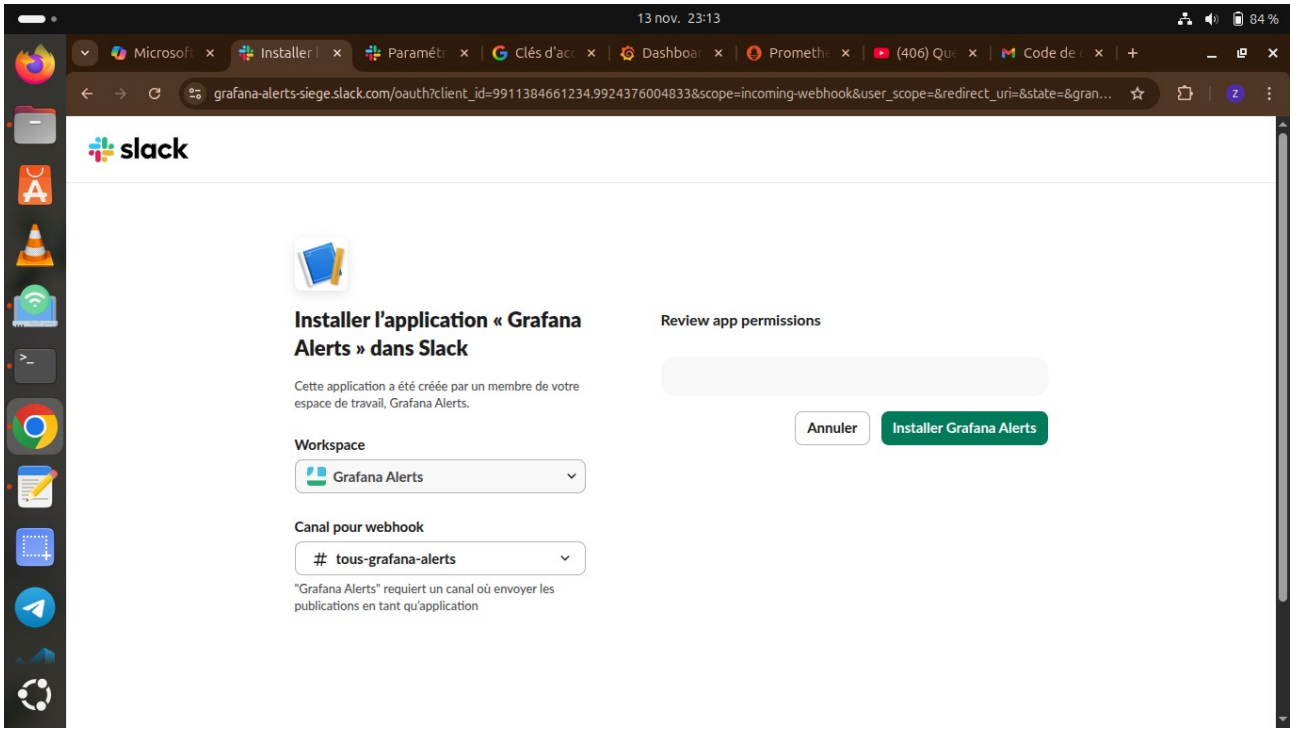



*Figure 29: installation de l'application grafana-alerts dans slack*

### d.4. Configuration du contact point dans Grafana

Dans Grafana, un contact point a été configuré pour envoyer les alertes vers Slack via le webhook précédemment généré.

Paramètres :

- **Nom** : zomahefa-slack
- **Intégration** : Slack

**Résultat attendu :**

Grâce à cette configuration :

- Les alertes critiques  sont envoyées directement dans Slack
- L'équipe est informée en temps réel sans avoir à consulter Grafana
- La supervision devient proactive et collaborative
- L'équipe reçoit les alertes à n'importe quel moment car slack a une version web, android, desktop  (linux ,windows et macOS) alors c'est peu de chance que les alertes ne sont pas visualisés

La figure 30 montre la création du contact point dans grafana

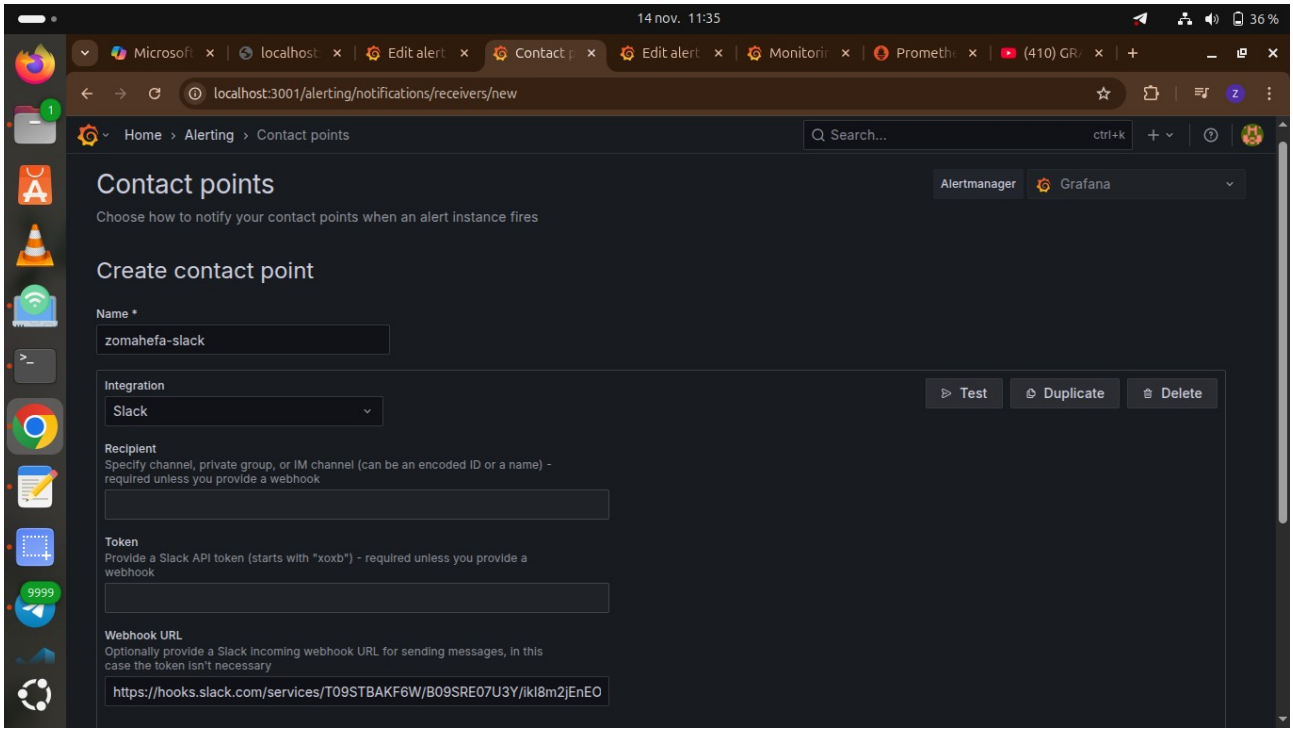



*Figure 30: création du contact point dans grafana*

### *d.5. Configuration des alertes*

Les alertes permettent de détecter automatiquement les anomalies dans le système et de prévenir l'équipe en temps réel. Dans ce projet, elles sont configurées dans Grafana à partir des métriques Prometheus et des logs agrégés par Loki. L'objectif est de garantir une supervision proactive, réactive et contextualisée.

Dans Grafana, les règles d'alerte peuvent être créées à partir de n'importe quelle source de données compatible comme Prometheus et Loki. Chaque règle suit une structure en plusieurs étapes :

- **Définition du nom et de la source**
    - Choix du nom de la règle
    - Sélection de la source (Prometheus ou Loki)
    - Définition de la requête selon le dashboard qu'on veut alerter par exemple logs contenant status=500 ou métriques dépassant un seuil spécifique
- **Organisation et évaluation**
    - Attribution à un dossier pour une meilleur organisation
    - Ajout de labels pour faciliter le regroupement et la gestion des alertes par exemple service=backend, severity=critical
    - Définition du comportement d'évaluation : fréquence, période de pending
- **Configuration des notifications**
    - Sélection du contact point SLACK.
    - Définition des timings :
        - Group wait : délai avant envoi
        - Group interval : intervalle entre alertes similaires
        - Repeat interval : fréquence de répétition
        - Mute timings : plages horaires où les alertes sont suspendues
- **Personnalisation du message d'alerte**
    - Résumé : description courte de l'alerte
    - Description : explication détaillée de la règle
    - Runbook URL : lien vers une documentation ou procédure de résolution
    - Annotation du dashboard et du panel associé

**Résultat attendu :**

Grâce à cette configuration :

- Les anomalies critiques sont détectées automatiquement
- Les alertes sont envoyées dans Slack ou tout autre canal configuré
- Les messages sont clairs, contextualisés et liés aux dashboards
- L'équipe peut réagir rapidement et suivre les incidents dans Grafana

La mise en place de cette architecture de supervision permet à l'équipe de disposer d'une visibilité complète et en temps réel sur l'état de l'infrastructure. Chaque service déployé est surveillé en continu, ce qui facilite la détection précoce des anomalies et permet une réaction rapide en cas d'incident.

Les métriques collectées par Prometheus, les logs agrégés par Loki, et les dashboards dynamiques de Grafana offrent une base solide pour le diagnostic, l'analyse de performance et le suivi de la santé des composants critiques. Les alertes configurées assurent une notification immédiate en cas de défaillance, renforçant ainsi la résilience du système.

Cette supervision ne se limite pas à un simple outil de contrôle : elle incarne un véritable levier stratégique dans l'approche DevOps du projet. En passant d'un mode réactif à une posture proactive, elle permet d'anticiper les dysfonctionnements, de réduire les temps d'indisponibilité, et de garantir une haute disponibilité du service en production.

# chapitre 7: Test et validation

Cette section vise à démontrer, preuves à l'appui, que l'ensemble des configurations mises en place dans les phases précédentes ont été testées et validées avec succès. Elle s'appuie sur des captures d'écran, des logs, et des résultats concrets issus du serveur VPS, de GitHub Actions, de Grafana, et de l'application elle-même.

## 7.1. Architecture finale du projet

Après avoir installez et configurez tous les outils techniques retenus pour le projet. On a une nouvelle architecture. Cette nouvelle architecture représente la structure logique et technique mise en place pour assurer le bon fonctionnement du système, depuis le développement de l'application jusqu'à son déploiement en production.

Elle a été conçue pour répondre aux exigences de **modularité**, **automatisation**, **sécurité** et **continuité de service**. Elle intègre les outils choisis dans une chaîne cohérente, en s'appuyant sur des pratiques DevOps modernes telles que l'intégration continue (CI), le déploiement continu (CD), l'utilisation de conteneurs Docker, et une stratégie de déploiement blue-green.

L'architecture complète est présentée dans la Figure 31 qui montre l' Architecture finale du projet

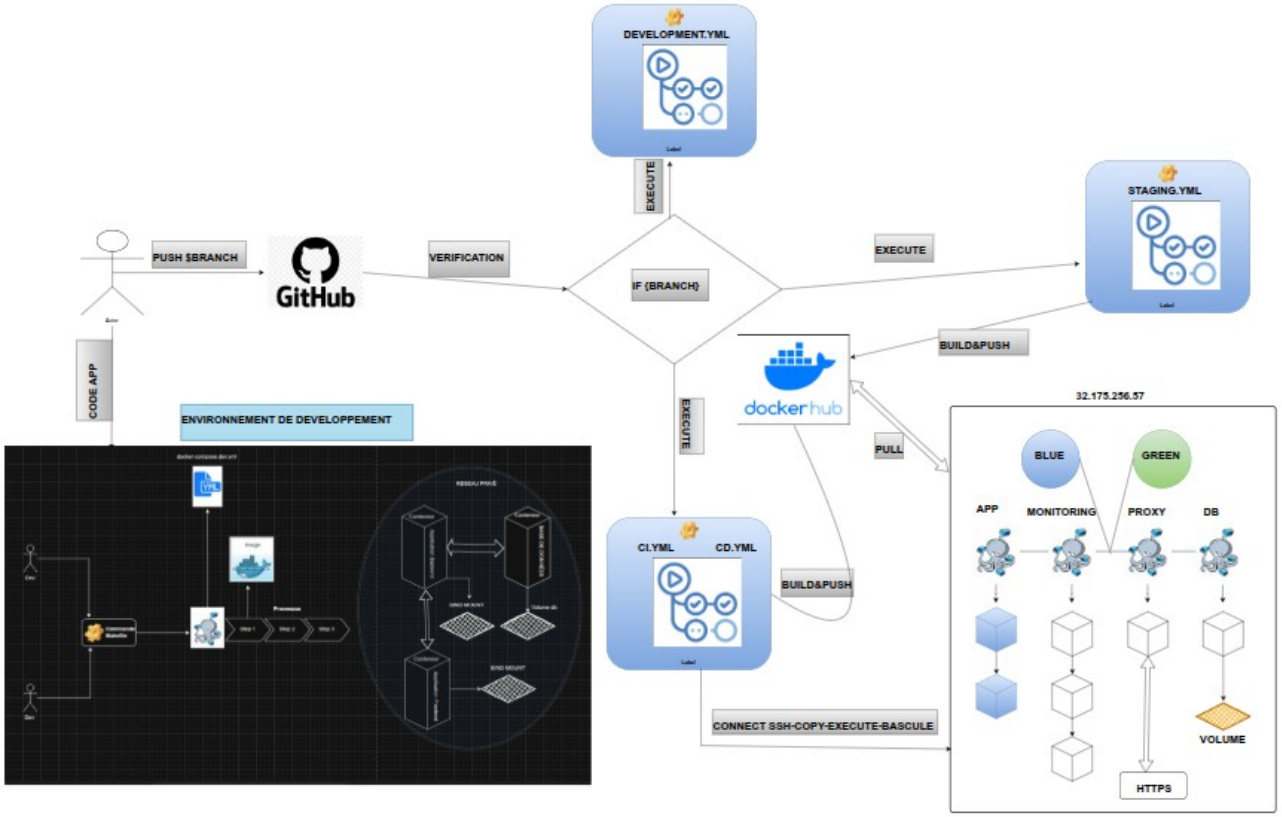



*Figure 31: architecture final du projet*

### 7.1.1. Description du fonctionnement de l'architecture

**Push dans une branche sur GitHub**

Le développeur pousse son code (`git push`) vers une branche du dépôt GitHub.

**Déclenchement automatique du pipeline GitHub Actions**

Trois workflows principaux sont définis :

- `development.yml`: déclenché uniquement après un évènement push sur la branche `develop` dédié à la développement
- `staging.yml` : déclenché uniquement après un évènement push ou pull request sur la branche `staging`
- `ci.yml` : déclenché uniquement après un évènement push ou pull request sur la branche `main` dédié à la production pour faire l'intégration continue
- `cd.yml` : déclenché uniquement que si ci.yml est en résussite, ce fichier est responsable du déploiement continue de l'application sur le VPS.

**Étape de vérification**

Le pipeline exécute obligatoirement :

- Tests unitaires et d'intégration
- Lint et analyse statique du code
- Scan de sécurité de l'image Docker (Trivy)
- Toutes les étapes sont importantet en cas d'échec d'une étape en production le pipeline s'arrête

**Condition sur la branche**

GitHub Actions choisit le bon fichier YAML en fonction de la branche poussée.

**Construction et publication de l'image Docker**

- Un Dockerfile multi-stage est présent à la racine du projet.
- L'image est construite avec deux tags :
    - monrepo/monapp:${{ github.sha }}`
    - monrepo/monapp:latest` (ou `staging-latest`)
- L'image est immédiatement poussée vers Docker Hub .

**Déploiement automatisé sur le serveur de staging/production**

Le workflow GitHub Actions :

- Se connecte en SSH au serveur cible
- copie seulement les fichiers docker-compose nécessaire pour démarrer l'application
- Récupère les images sur docker hub via docker pull
- Détermine quel environnement est actuellement actif (Blue ou Green)
- Déploie la nouvelle version sur l'environnement inactif et grade la version active pendant un certain temps, il fait le test sur la version inactive puis arrêtez la verion active,au cas contraire si les tests échoue, il y a rollback automatique sur la version active, celà garantit la haute disponibilité de l'application.

**Les 4 fichiers Docker Compose gérés sur le serveur**

Sur le serveur, nous utilisons quatre fichiers docker-compose distincts qui tournent en parallèle sur le même réseau:

- docker-compose.db.yml : dédié à la base de données, il est relié avec un volume persistant docker-compose.monitoring.yml: dédié à la monitoring, il assure la supervision de l'application et de tous le système
- docker-compose.edge.yml :Reverse proxy, serveur web et routeur edge utilisant nginx avec certificats TLS pour la sécurité
- docker-compose.blue.yml ou docker-compose.green.yml : L'application elle-même (version courante ou nouvelle) |

**Stratégie de déploiement Blue-Green sans interruption**

- À chaque nouveau déploiement, GitHub Actions lance le stack nactif
- Une fois les conteneurs de la version inactive à déployer démarrés et les tests de santé validés (healthcheck + tests via script), on bascule le reverse proxy nginx: on change les upstream pour rediriger 100 % du trafic vers la version active
- L'ancien environnement est arrêté proprement
- En cas de problème rollback rapide en moins d'une seconde en basculant simplement sur l'ancienne version

### 7.1.2. Avantages de cette architecture

- Zéro downtime grâce à la mise en place du deploiment bluegreen
- Rollback ultra-rapide (simple commande `docker compose -f docker-compose.blue.yml up -d`)
- Infrastructure très légère (un seul serveur suffit pour staging/production)
- Coût quasi nul (Docker Hub gratuit + serveur VPS ou machine personnelle)
- Traçabilité parfaite (chaque déploiement = un commit GitHub + une image taguée)
- Monitoring et alertes intégrés dès le départ
- Évolutive : Facilité d'intégration des outils comme K8S,Terraform,AWS,etc en cas d'évolution de l'infrastructure

Cette architecture constitue la réalisation concrète de tous les objectifs fixés en début de projet et sera utilisée pour tous les tests de performance et de charge présentés dans le chapitre suivant.

## 7.2. Validation des workflows CI/CD

Pour chaque branche (develop, staging, main), les workflows GitHub Actions ont été déclenchés automatiquement à chaque push ou pull request. Les résultats suivants ont été observés :

- Branche develop : le workflow development.yml s'exécute avec tolérance aux erreurs. Il permet de tester rapidement les nouvelles fonctionnalités sans bloquer les développeurs.

La figure 32 montre la réussite du workflow dans github actions après un push sur develop

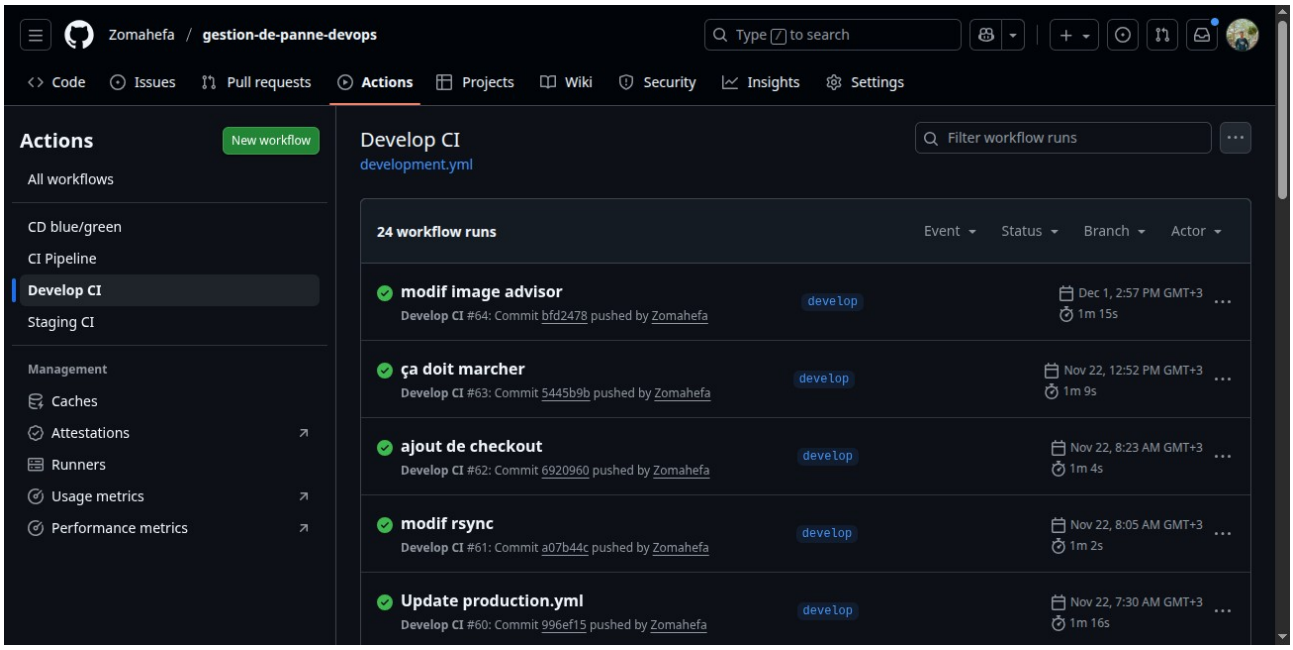



*Figure 32: validation du workflow development.yml*

- Branche staging : le workflow staging.yml exécute les tests, les audits de sécurité, et construit les images Docker puis les push sur dockerhub.

La figure 33 illustre la réussite du workflow dans Github après un push sur staging

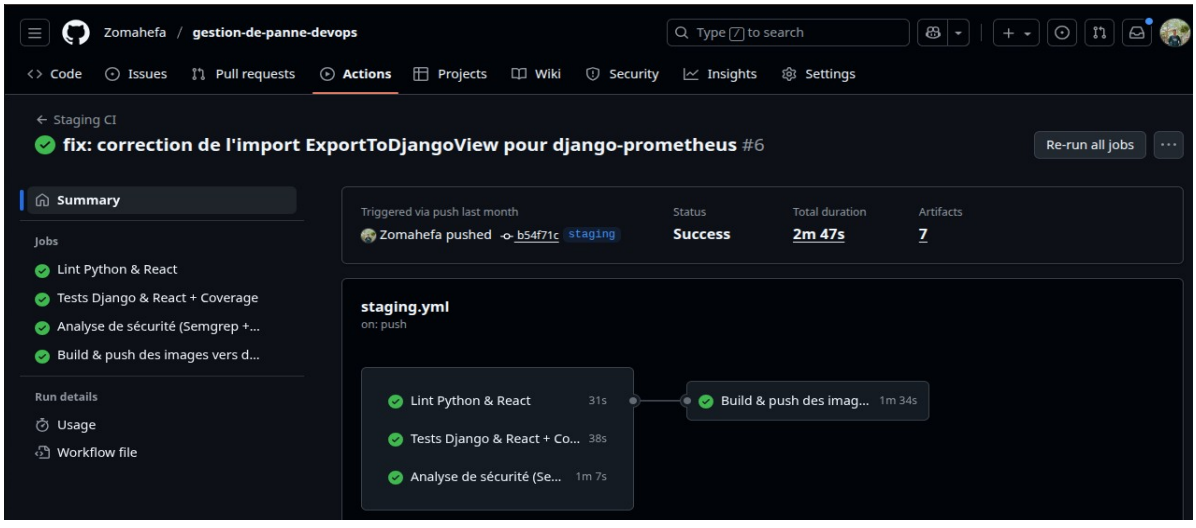



*Figure 33: validation du workflow staging.yml*

Voici les images backend et frontend sur dockerhub qui contient le tag latest et les tag qui s'incrémente à chaque push sur github.

La figure 34 montre les images backend pour la phase de staging sur dockerhub

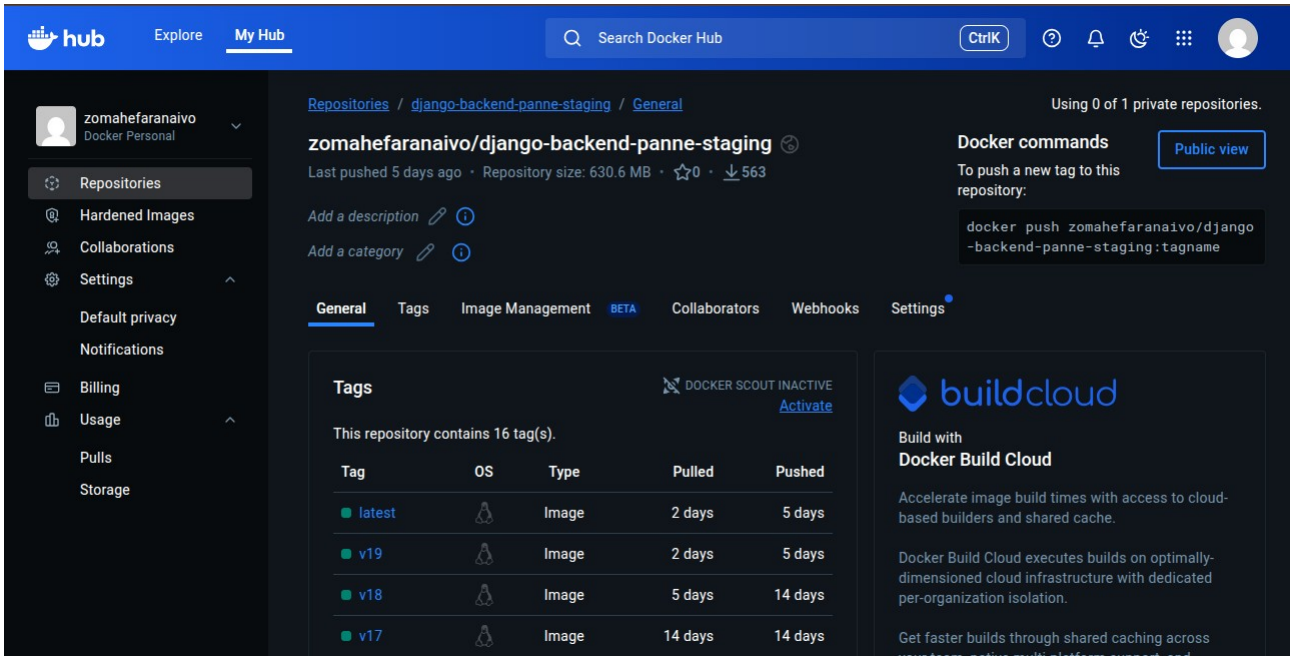



*Figure 34: visualisation des images backend dédié à la staging sur dockerhub*

La figure 35 montre les images frontend pour la phase de staging sur dockerhub

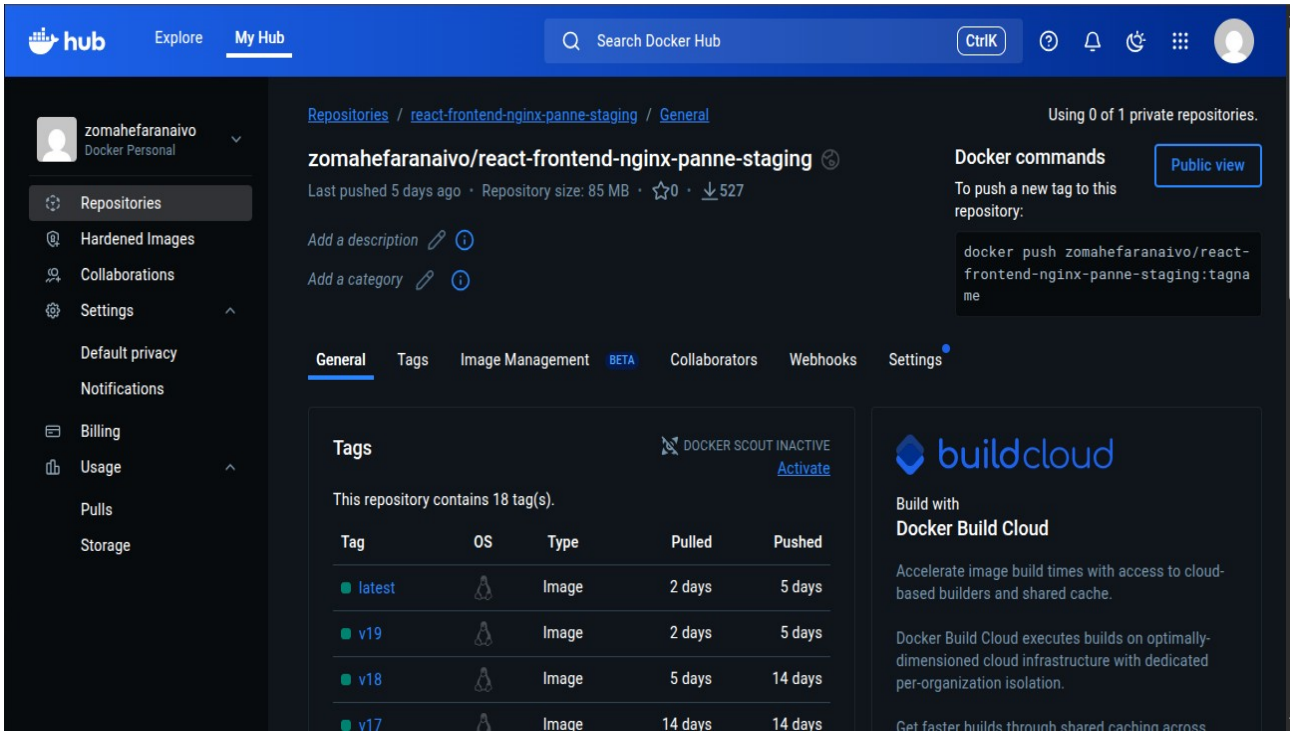



*Figure 35: visualisation des images frontend dédié à la staging sur dockerhub*

- Branche main : les workflows ci.yml et cd.yml assurent le test, la construction, le build&push des images et le déploiement automatisé avec stratégie blue/green.

La figure 36 montre la réussite de l'intégration continue ci.yml  après une pull request sur main.

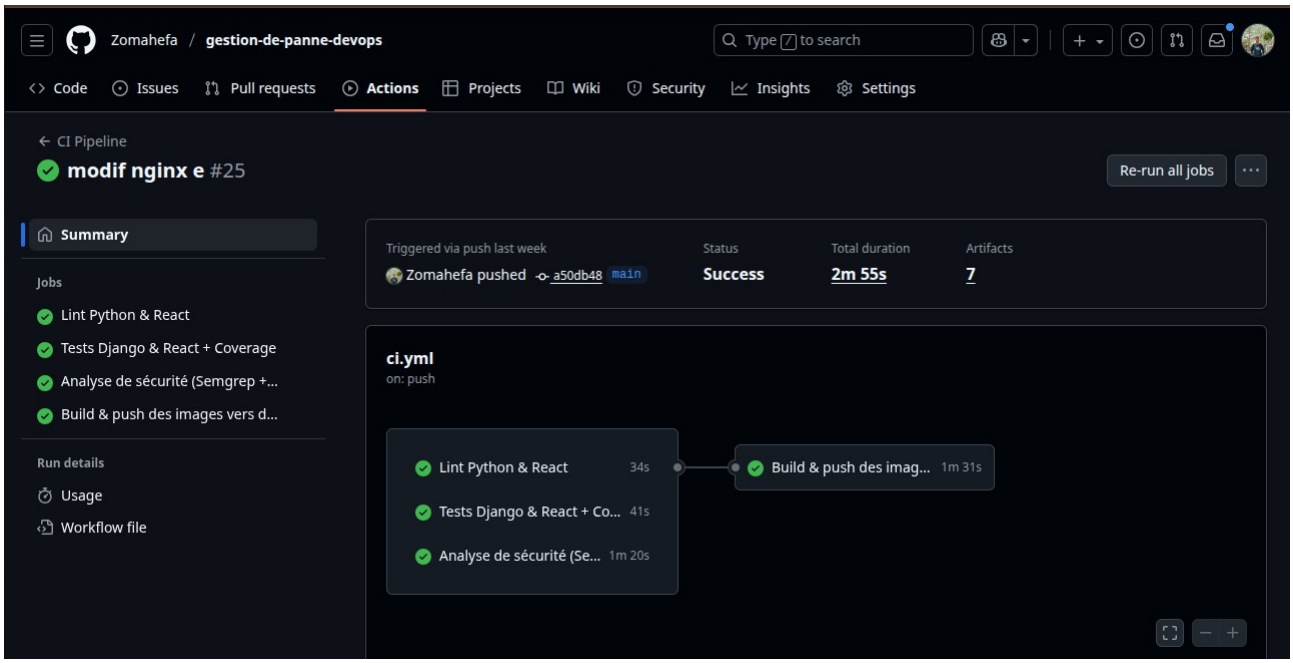



*Figure 36: validation du workflow ci.yml*

La figure 37 montre le cd.yml qui s'exécute à condition que le ci.yml est en réussite

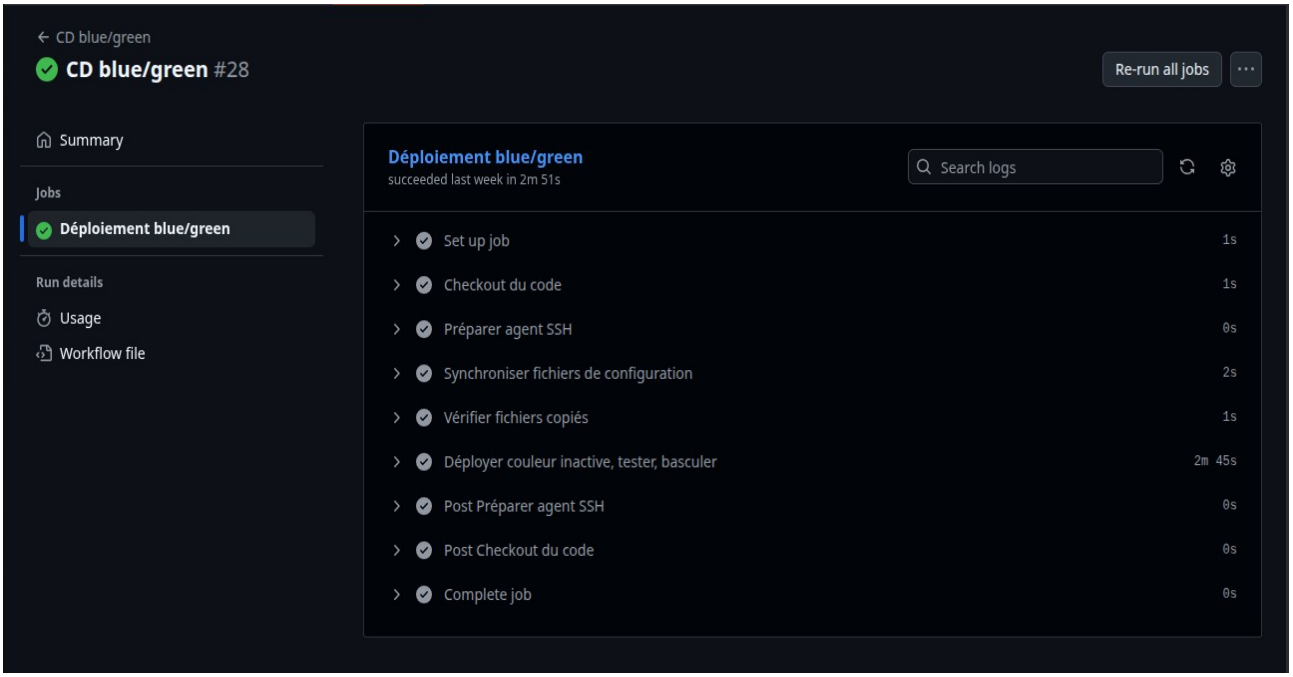



*Figure 37: validation du workflow cd.yml*

Voici les images backend et frontend dédié à la production sur dockerhub qui contient le tag latest et les tag qui s'incrémente à chaque push sur github.

La figure 38 montre les images frontend pour la phase de production sur dockerhub

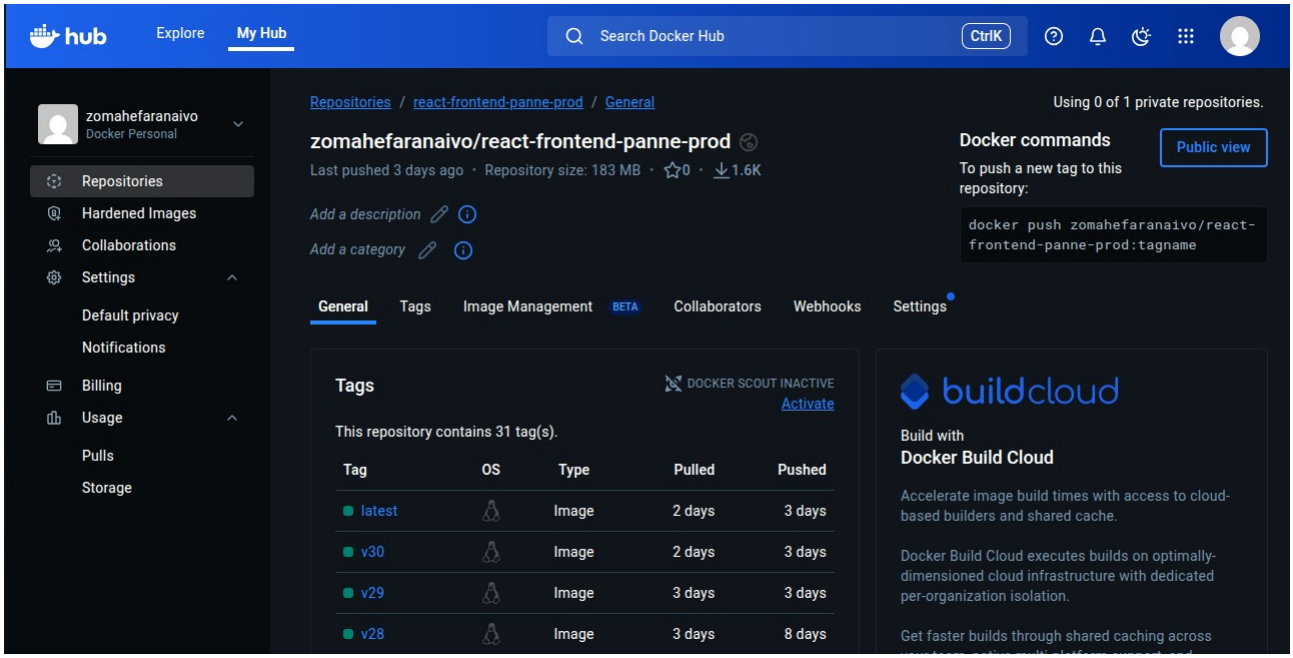



*Figure 38: visualisation des images backend dédié à la production sur dockerhub*

La figure 39 montre les images backend pour la production sur dockerhub

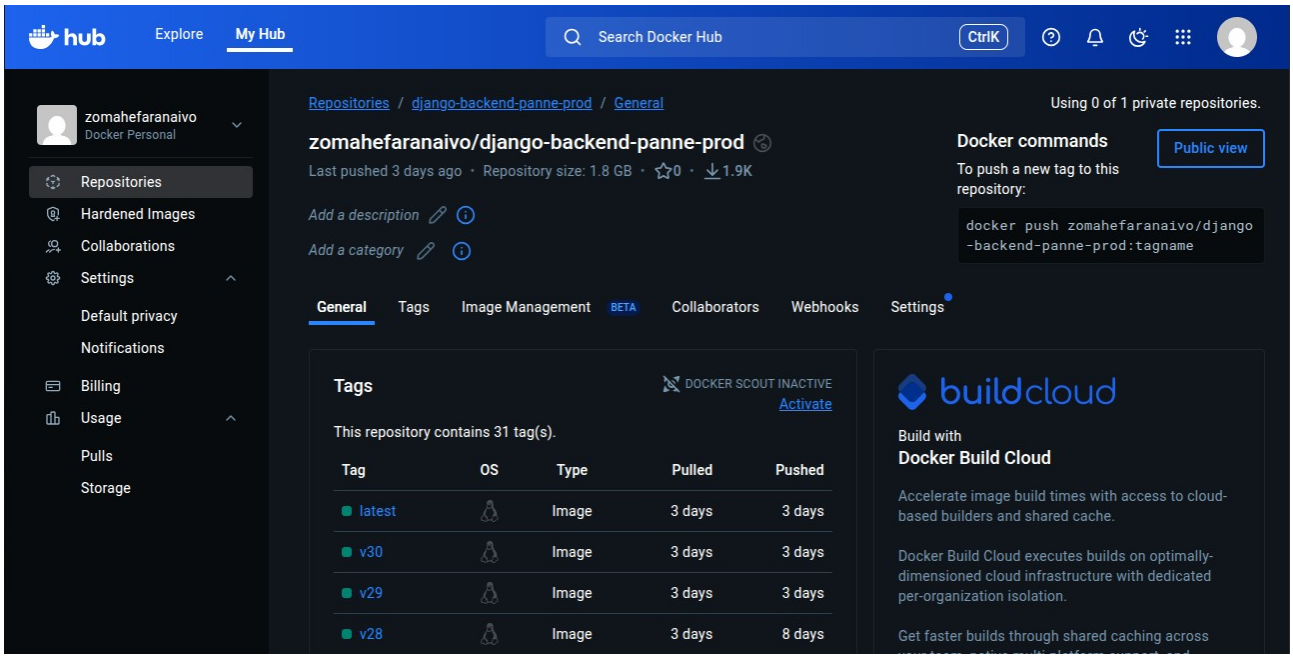



*Figure 39: visualisation des images backend dédié à la production sur dockerhub*

### 7.2.1. *Accès à l'interface de l'application en développement et en staging*

Avant de valider le déploiement sur le serveur de production, des tests fonctionnels ont été réalisés dans les environnements de développement et de staging pour s'assurer du bon fonctionnement de l'interface utilisateur.

- **Environnement de développement**

Dans l'environnement de développement, l'application est lancée localement via la commande :

`make dev-up` ,cette commande utilise le Makefile pour exécuter docker-compose.dev.yml, qui démarre les services avec un **bind mount** entre le code source local et les conteneurs. Cela permet le développement avec **hot reload**.

La figure 40 illustre le fonctionnement du makefile

```
root@ranaivo-Aspire-E1-571G:/home/ranaivo/Documents/gestion-de-panne-devops# make dev-up
docker compose -f docker-compose.dev.yml --env-file .env -p panne-dev up -d
[+] Building 55.9s (11/27)
 => [backend internal] load metadata for docker.io/library/python:3.13-slim                                                  7.3s
 => [frontend internal] load metadata for docker.io/library/node:20-alpine                                                   7.3s
 => [auth] library/python:pull token for registry-1.docker.io                                                                0.0s
 => [auth] library/node:pull token for registry-1.docker.io                                                                  0.0s
 => [backend internal] load .dockerignore                                                                                    0.5s
 => => transferring context: 633B                                                                                            0.1s
 => [frontend internal] load .dockerignore                                                                                   0.6s
 => => transferring context: 426B                                                                                            0.1s
 => [backend base 1/5] FROM docker.io/library/python:3.13-slim@sha256:05b118ecc93ea09e30569706568fb251c71b77d2a3908d338b  45.8s
 => => resolve docker.io/library/python:3.13-slim@sha256:05b118ecc93ea09e30569706568fb251c71b77d2a3908d338b77be033e162b59  1.1s
 => => sha256:05b118ecc93ea09e30569706568fb251c71b77d2a3908d338b77be033e162b59 10.37kB / 10.37kB                             0.0s
 => => sha256:f461762489abc4585b950859793dd1834dfa3b661217b539c767302eb8ce2522 1.75kB / 1.75kB                               0.0s
 => => sha256:287290e873d1fb5d54efdf73ca90f791e7f25755e9d03a78ae912beb038b3811 5.53kB / 5.53kB                               0.0s
 => => sha256:13a347a59782d57000d9d8f542cae148af6b9027fdce630d1412401301eeca5f 250B / 250B                                   1.7s
 => => sha256:c9e669a16e74d8022e55e0c6239a6b0898b1e7edea584946ed700da9ec3ca479 1.29MB / 1.29MB                               6.4s
 => => sha256:f2ec8236a2e7d9404af3e612305d3206a01b5f4062e924100374ef93dfa602f4 8.39MB / 11.79MB                             43.7s
 => => extracting sha256:c9e669a16e74d8022e55e0c6239a6b0898b1e7edea584946ed700da9ec3ca479                                    0.6s
 => [backend internal] load build context                                                                                    1.4s
 => => transferring context: 57.11kB                                                                                         0.8s
 => [frontend 1/5] FROM docker.io/library/node:20-alpine@sha256:16858294071a56ffd4cce9f17b57136cc39e41507b40e245b4f8e906  45.5s
 => => resolve docker.io/library/node:20-alpine@sha256:16858294071a56ffd4cce9f17b57136cc39e41507b40e245b4f8e906f7a19463    1.8s
 => => sha256:16858294071a56ffd4cce9f17b57136cc39e41507b40e245b4f8e906f7a19463 7.67kB / 7.67kB                               0.0s
 => => sha256:bcd88137d802e2482c9df3cdec71e0431857ebbbdba6973776b5593214056d86 1.72kB / 1.72kB                               0.0s
 => => sha256:6fd15b1a333d8412f468ba380d71274e0059ebf8828b37926bff43ba7ca03898 6.50kB / 6.50kB                               0.0s
 => => sha256:537c6f956080bc5b44a38de0951224e37050aa22b362f4e54156e36e46c2e173 6.29MB / 42.76MB                             43.1s
 => => sha256:d31fb19c9d765a3839243f009b51eb48b4d9e104cebc2ec210a649d47c538bef 1.26MB / 1.26MB                               31.0s
 => => sha256:d4cda882e05ad8415027412fb534631d499c5216411b3d162ebac40e824f048 446B / 446B                                   39.3s
 => [frontend internal] load build context                                                                                   1.4s
```

*Figure 40: utilisation du commande makefile*

L’application est accessible à l’adresse : **http://localhost:3000**

La figure 41 montre l’interface de l’application en développement

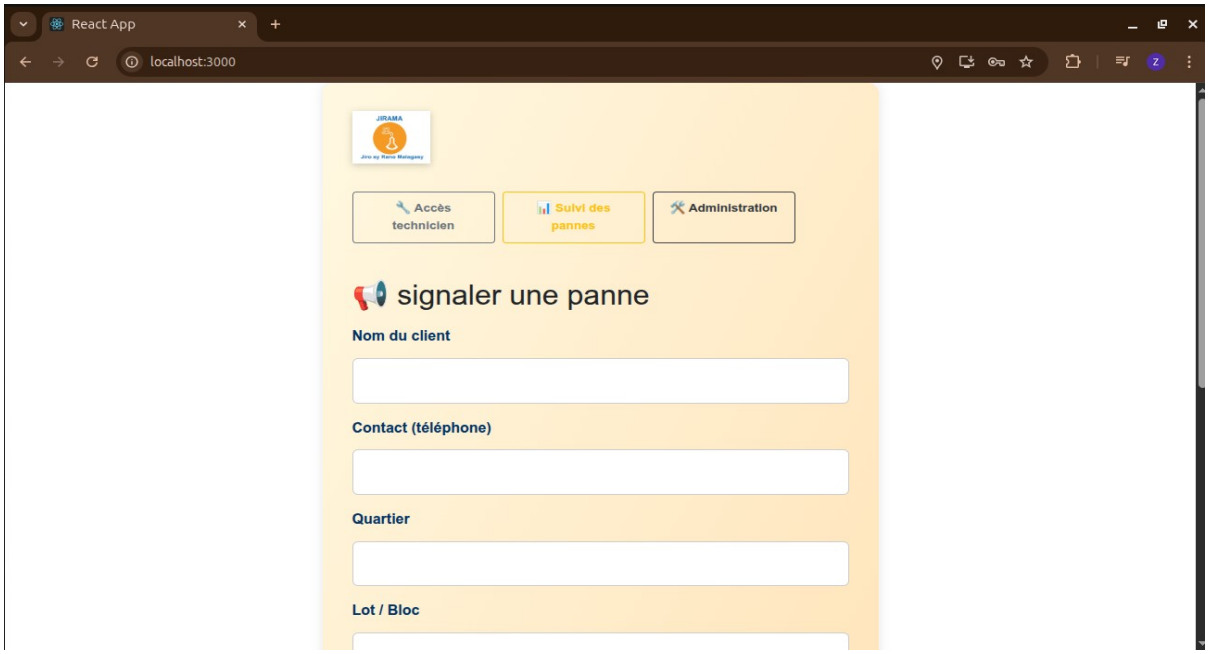



*Figure 41: Visualisation de l'application en phase de développement*

- **Environnement de staging**

Dans l’environnement de staging, l’application est simulée dans des conditions proches de la production.

La figure 42 montre l'interface de l'application en staging

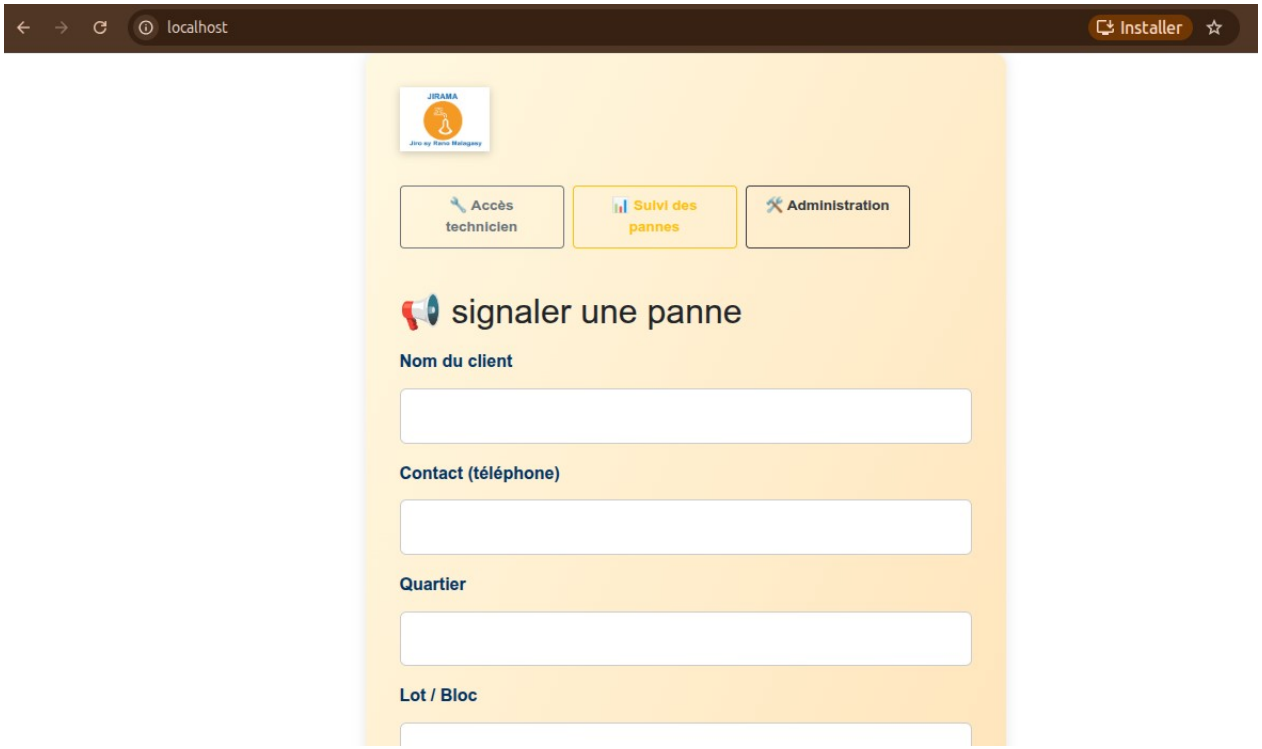



*Figure 42: visualisation de l'application en phase de staging*

### 7.2.2. Vérification du déploiement sur le serveur

Une fois le pipeline CD exécuté, les services ont été déployés sur le VPS dans le répertoire **/opt/panne-2025**. Les vérifications suivantes ont été effectuées :

- Tous les conteneurs sont en état **healthy** confirmé dans les résultats du workflows
- Les services sont accessibles via le domaine DuckDNS configuré : https://ambohimahsoa-panne.duckdns.org/ sécurisé avec certificat valide
- Les fichiers frontend_active.conf et backend_active.conf ont bien été mis à jour
- Le rechargement de Nginx s'est effectué sans erreur
- L'ancienne version a été supprimer lorsque les différents tests de la nouvelle version sont réussis

La figure 43 montre le résultat du workflow cd.yml

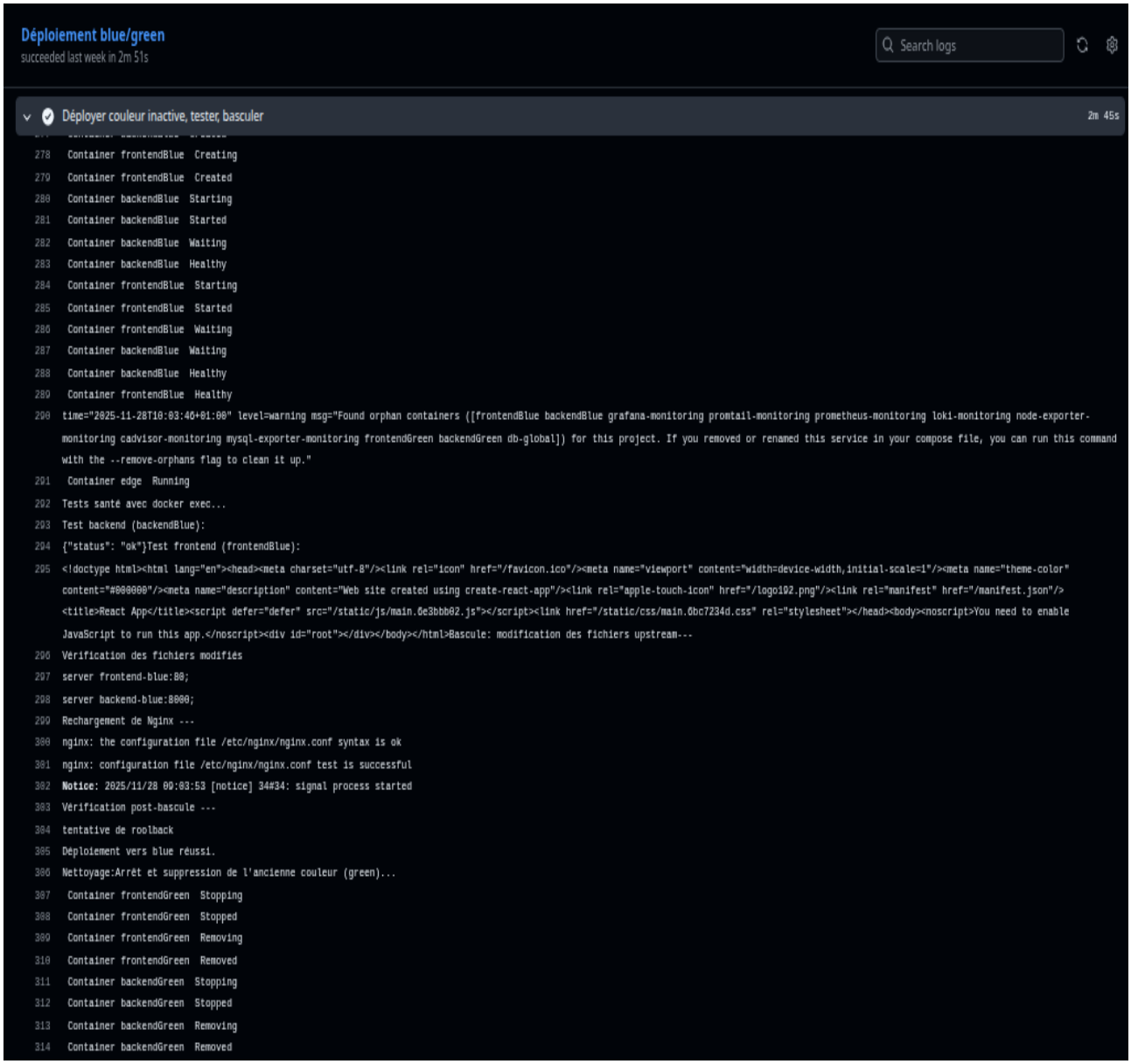



*Figure 43: Visualisation du résultat du workflow cd.yml*

Voici l'application avec un certificat TLS qui est bien valide, le site est sécurisé sur l'url https://ambohimahsoa-panne.duckdns.org/

La figure 44 montre l'interface de l'application en production

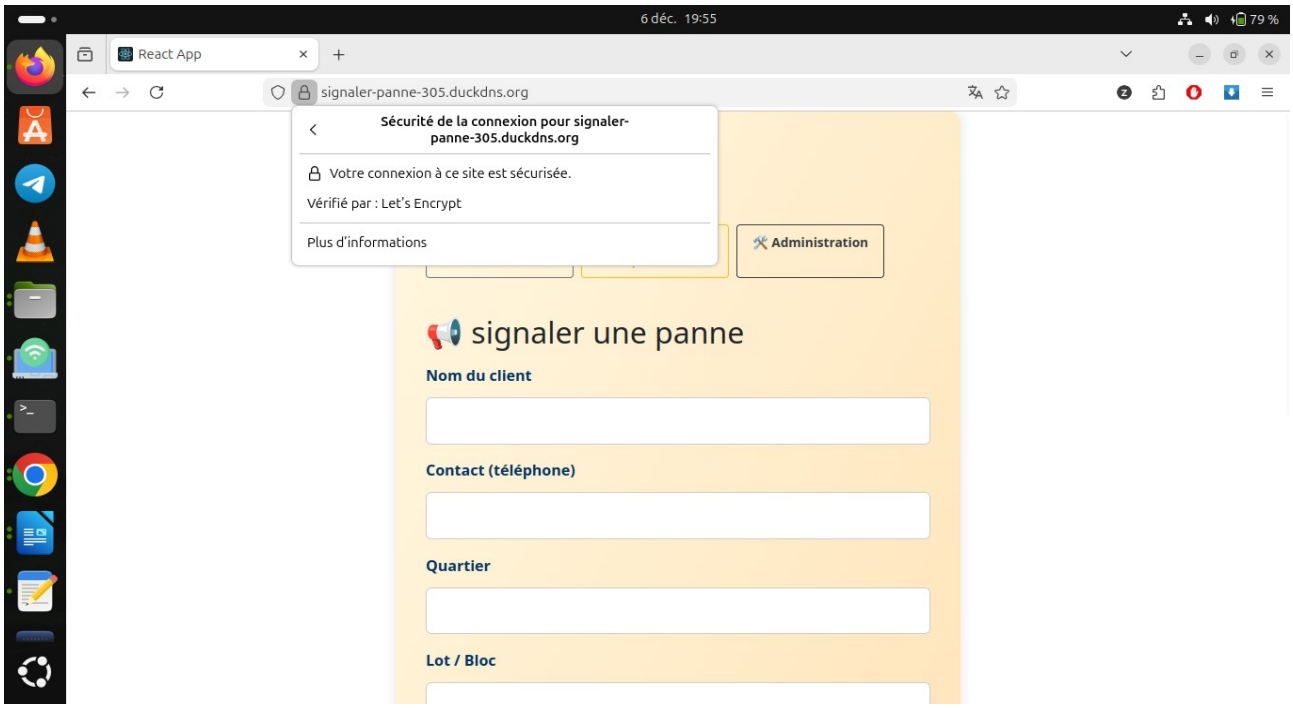



*Figure 44: visualisation de l'interface de l'application en production*

## 7.3. Tests de supervision et alerting

Les dashboards Grafana ont permis de visualiser en temps réel l'état des services. Les tests suivants ont été réalisés :

- Vérification des métriques dans Prometheus (/targets) : tous les jobs sont UP
- Visualisation des logs dans Grafana via Loki
- Déclenchement d'alertes simulées
- Réception des alertes dans Slack via le contact point configuré
- confirmation du sécurisation de nginx pour l'accès à grafana et prometheus

### 7.3.1. Vérification de la sécurité des accès

Lorsqu'on accède à https://signaler-panne-305.duckdns.org/grafana/ et https://signaler-panne-305.duckdns.org/grafana/ on nous demande d'abord une authentification, c'est l'effet du fichier *.htpaswd* mise en place.

La figure 45 montre la demande d’authentification pour accéder à l’interface de grafana

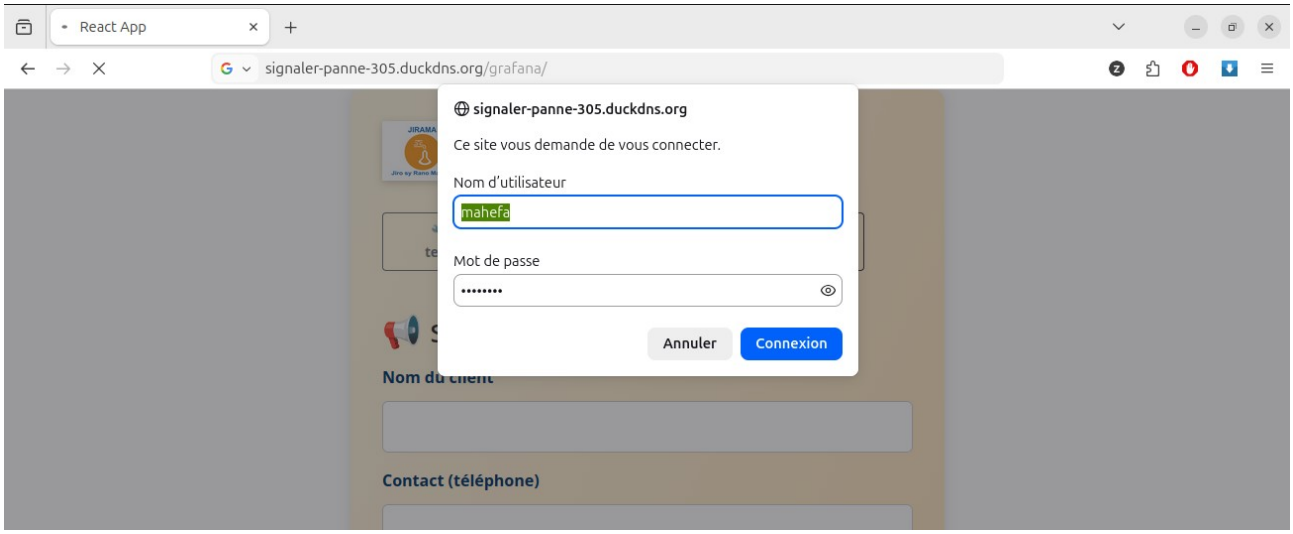



*Figure 45: Demande d'authentification pour accédez à l’interface de grafana*

La figure 46 montre la demande d’authentification pour accéder à l’interface de prometheus

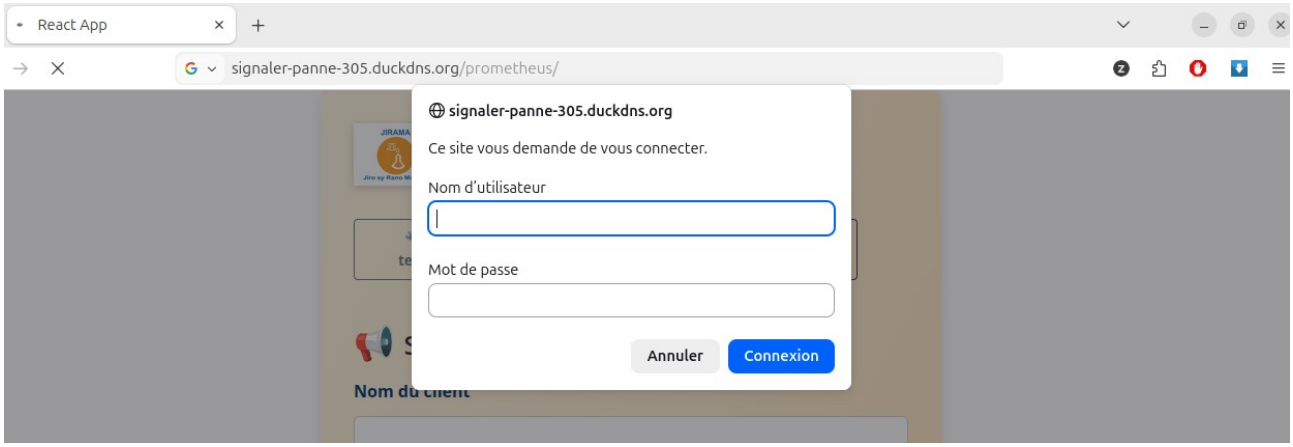



*Figure 46: Demande d'authentification pour accédez à l’interface de prometheus*

### 7.3.2. *Validation de prometheus*

Cette vue permet de vérifier que tous les services exposent correctement leurs métriques et que Prometheus les scrape sans erreur. Elle est essentielle pour valider la connectivité et la disponibilité des endpoints de monitoring. Dans l'image on voit bien que tous les services sont en état « UP ».

La figure 47 montre l'état des jobs dans l'interface de prometheus

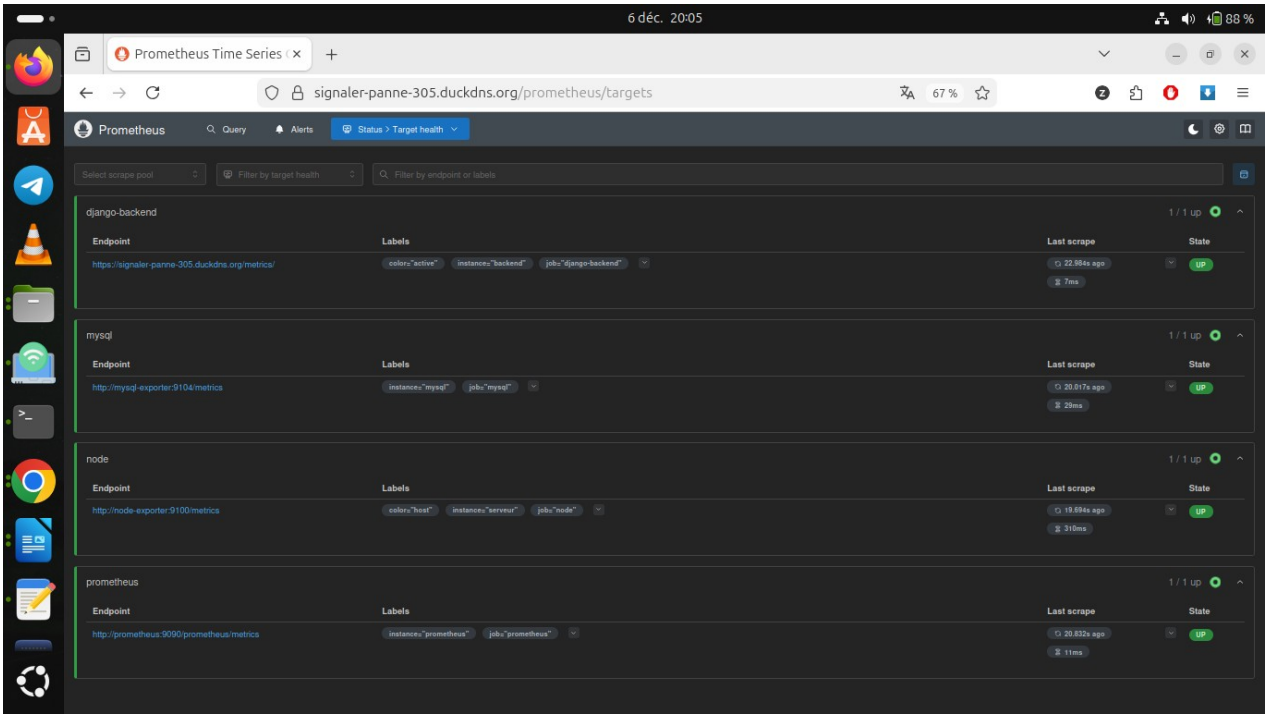



*Figure 47: Vérification des job dans l'interface de prometheus*

### 7.3.3. *Validation de grafana*

#### a. Visualisation Les dashboards

Le premier dashboard est celle du supervision de prometheus, c'est essentiel de le surveillé parce que c'est le responsable du collecte de tous les metrics avec 90 % de notre dashboard l'utilise comme source de données, cela garantit une supervision totale car même le superviseur est superviser.

La figure 48 illustre le dashboard de supervsion de prometheus

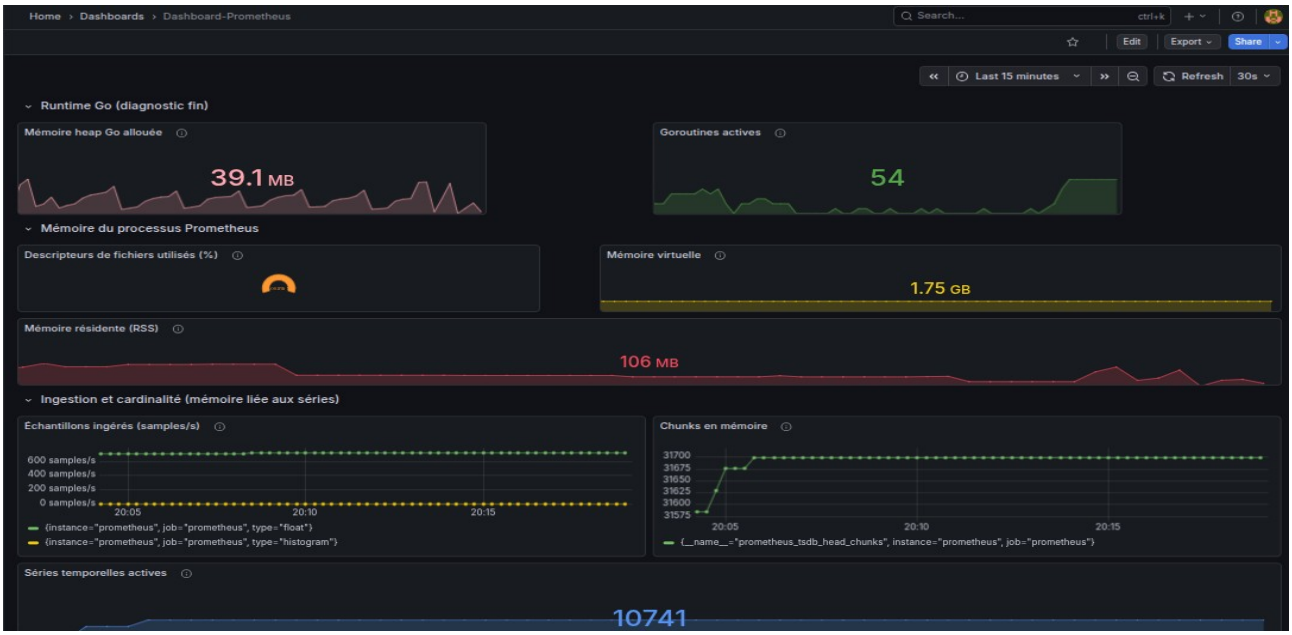



*Figure 48: dashboard de supervision de prometheus*

Le second dashboard affiche les métriques système du VPS hôte (CPU, RAM, disques, réseau) via Node Exporter. Il permet de surveiller la charge globale du serveur et d'anticiper les besoins en ressources.

La figure 49 illustre le dashboard node-exporter pour la supervision du VPS

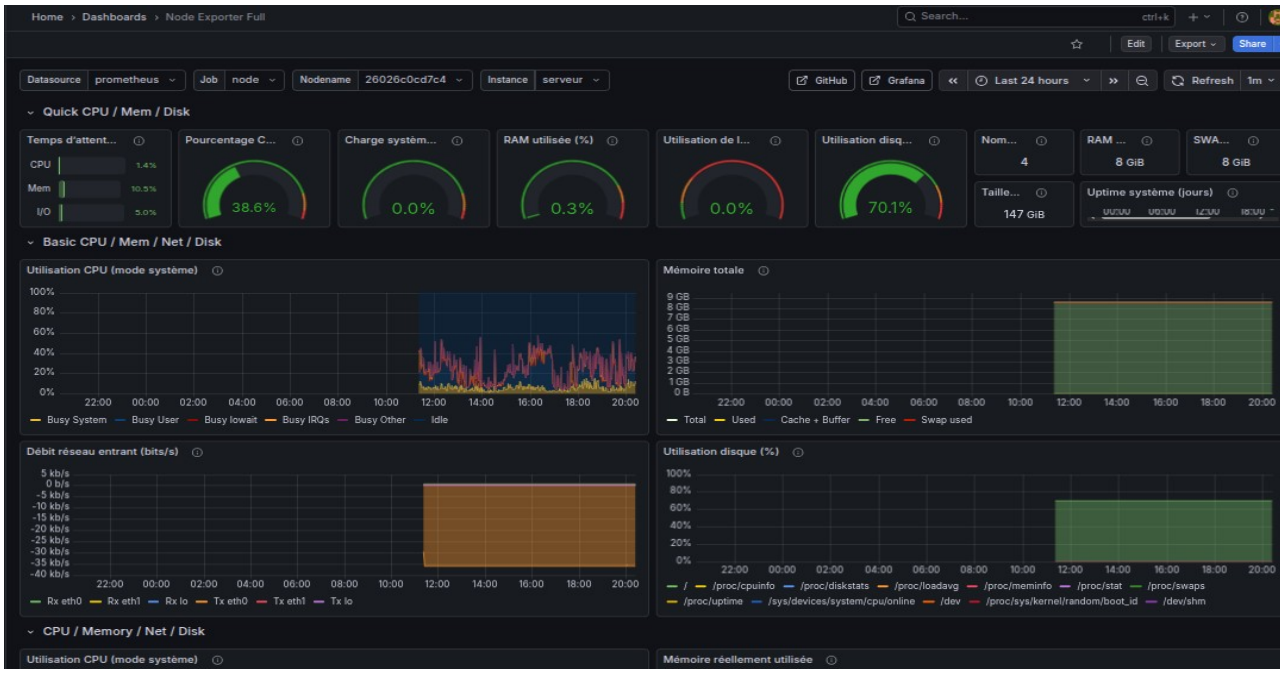



*Figure 49: Dashboard node-exporter pour la supervision du VPS*

Ce prochain dashboard est dédié au backend Django. Il expose des métriques spécifiques comme le temps de réponse, le nombre de requêtes, les erreurs HTTP, et la charge du serveur, permettant de surveiller la performance de l'API.

La figure 50 montre le dashboard de supervision du backend Django

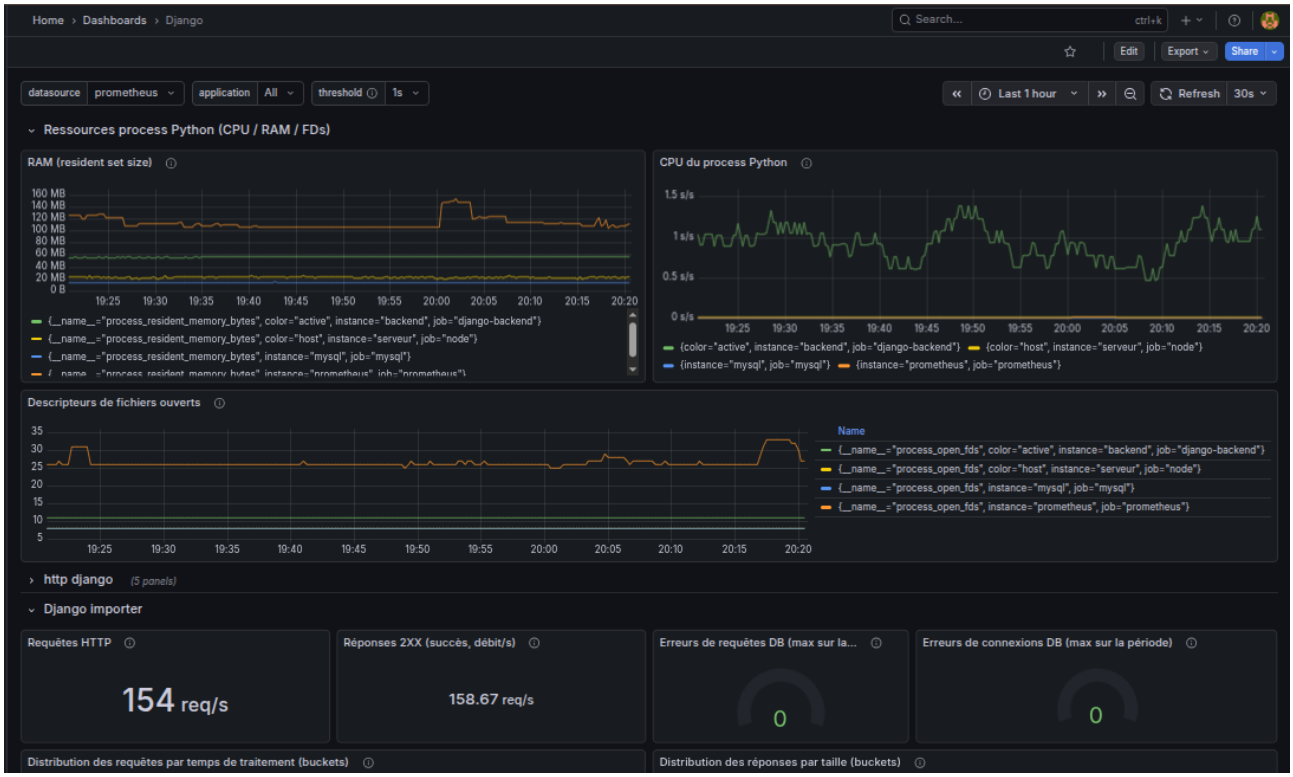



*Figure 50: Dashboard de supervision du backend Django*

Le dashboard suivant présente les métriques de la base de données MySQL, telles que les connexions actives, les requêtes par seconde, les verrous ou encore l'espace disque utilisé.

La figure 51 montre le dashboard mysql-exporter pour la supervision de la base de données mysql.

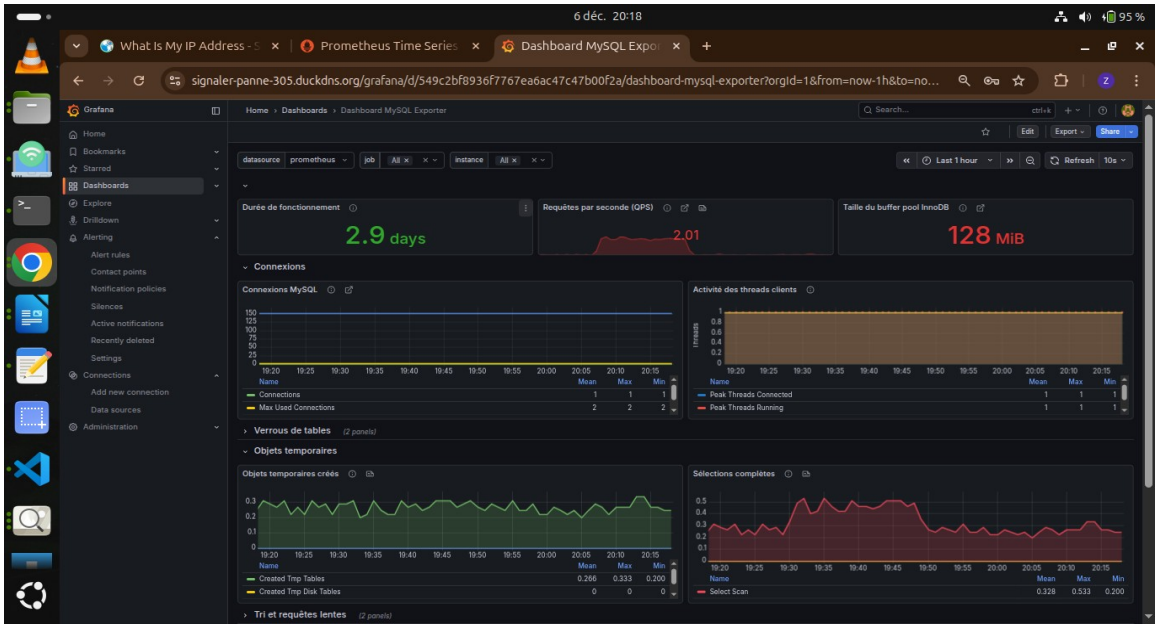



*Figure 51: dashboard mysq-exporter pour la supervision de la base de données mysql*

Le dashboard suivant affiche les logs des services collectés par Promtail et agrégés par Loki. Il permet de rechercher, filtrer et analyser les événements générés par les services, facilitant le diagnostic en cas d'incident.

La figure 52 montre le dashboard Loki pour la supervision des logs docker et système

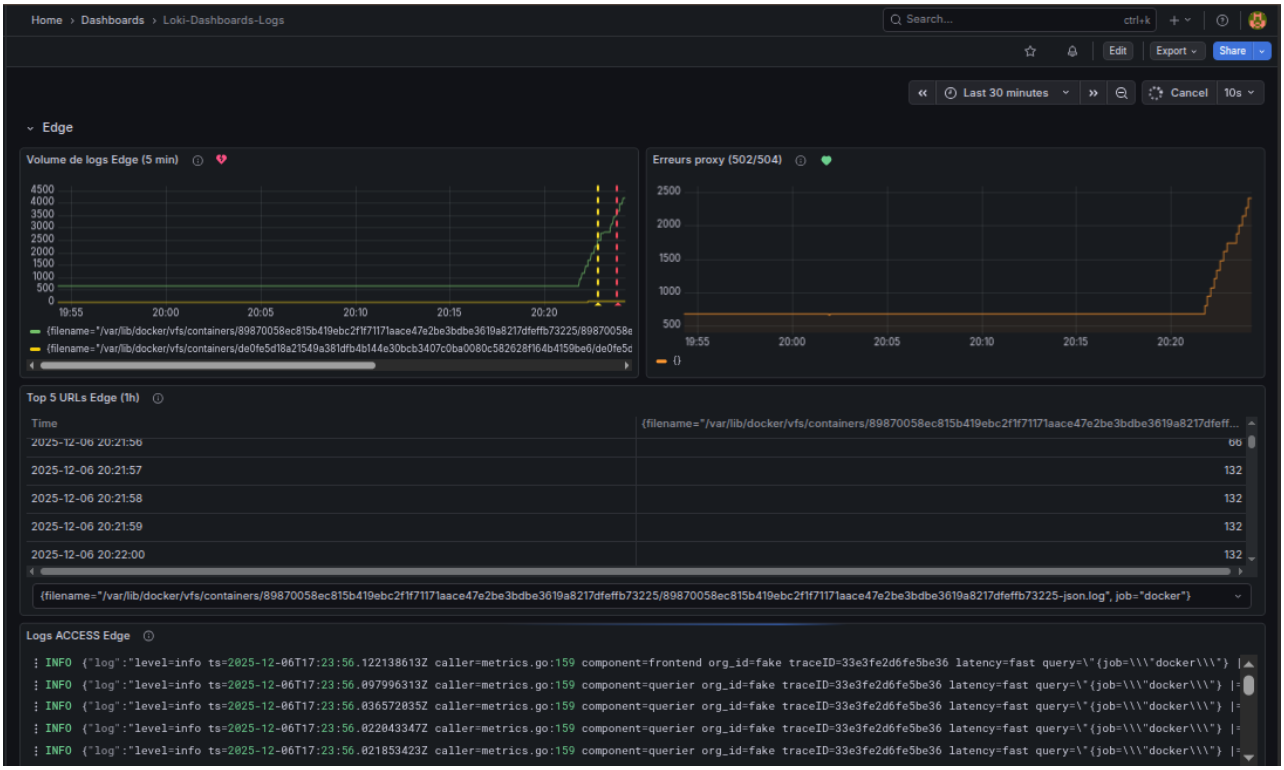



*Figure 52: Dashboard Loki pour la supervision des logs docker et système*

Dans certains panel lié à des alertes, il y a le deux bar en tiré et des cœurs colorés dynamiquement pour montré l'état du panel en fonction de l'alerte mise en place.Le cœur est coloré en vert si l'alerte est en mode normale, coloré en orange si c'est en cas critique et coloré en rouge sous forme de cœur brisé si c'est en cas critique (firing) dans l'alerte.

### b. Validation des alertes mis en place

On peut visualisez les alertes directement dans l'interface grafana dédié pour voir les détails de tous les alertes ou des alertes spécifique.

La figure 53 montre l'ensemble des alertes créer dans grafana

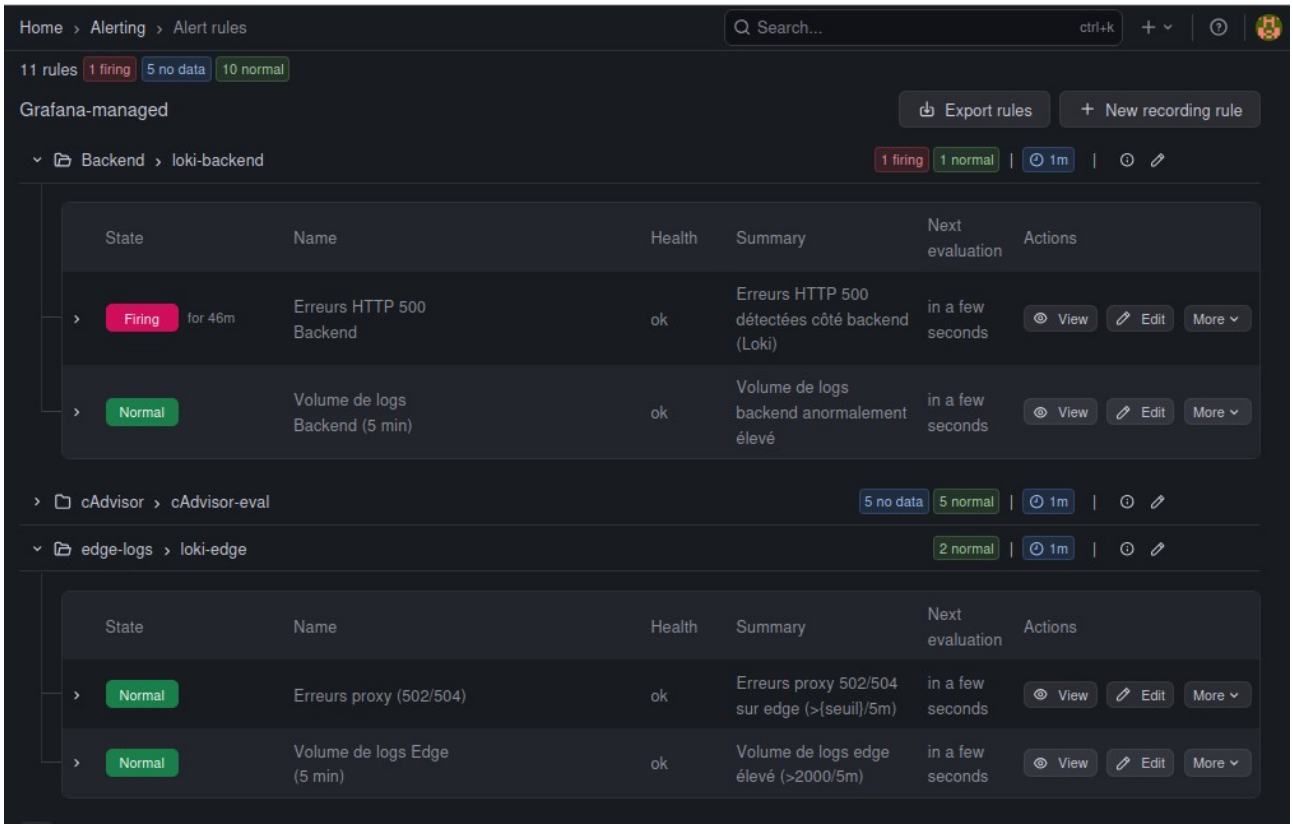



*Figure 53: Visualisation des alertes dans grafana*

Les résultats de ces alertes sont envoyé en temps réel dans le canal de l'espace de travail créer dans l'application slack. On a configuré tous les alertes pour se déclencher qu'en cas critique ce qui permet la résolution rapide des erreurs sans encore que le système tombe

La figure 54 montre les alertes envoyés dans l’application slack

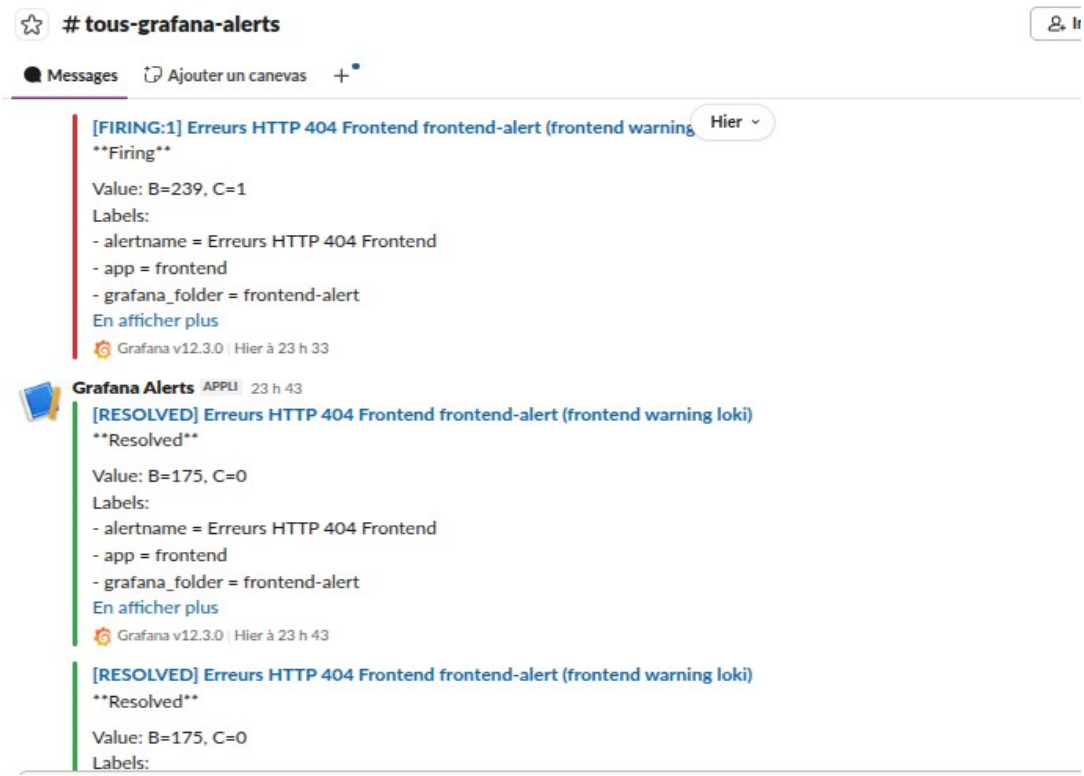



*Figure 54: visualisation des alertes dans l'application Slack*

# CONCLUSION

Ce mémoire s'inscrit dans un contexte de modernisation des pratiques de développement et de déploiement des applications web, où la fiabilité, la sécurité et la disponibilité des services sont devenues des enjeux majeurs, en particulier pour les organisations fournissant des services publics. La problématique abordée portait sur les limites des déploiements manuels et des solutions peu automatisées, qui entraînent des risques d'erreurs, des interruptions de service et un manque de visibilité sur l'état des applications en production.

Pour répondre à ces enjeux, une solution basée sur une approche DevOps a été mise en œuvre. Cette démarche a permis d'automatiser l'ensemble du cycle de vie applicatif, depuis l'intégration du code jusqu'au déploiement en production, tout en assurant une supervision continue et une meilleure maîtrise des environnements. La solution s'appuie notamment sur la conteneurisation des services, l'automatisation des pipelines CI/CD, la mise en place d'une stratégie de déploiement sécurisée ainsi que l'intégration d'outils de supervision et de centralisation des logs tels que Docker, GitHub Actions, Nginx, Prometheus, Grafana et Loki. Les résultats obtenus montrent une amélioration significative de la fiabilité des déploiements, une réduction des interventions manuelles et une meilleure capacité de détection et de résolution des incidents.

En conclusion, ce projet a permis de démontrer l'apport concret des pratiques DevOps dans la professionnalisation du déploiement des applications web. Il constitue une base solide pour l'évolution future de l'infrastructure mise en place.

En perspective, ce travail pourrait être enrichi par l'introduction de mécanismes avancés d'orchestration, l'optimisation des performances à grande échelle, ainsi que l'intégration de solutions de sécurité et d'automatisation encore plus poussées afin de répondre à des besoins opérationnels plus complexes.

# REFERENCES BIBLIOGRAPHIQUES


[1] M. Fowler, Continuous Integration. Boston, MA, USA: Addison-Wesley, 2012.

[2] J. Humble and D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Boston, MA, USA: Addison-Wesley, 2010.

[3] G. Kim, J. Humble, P. Debois, and J. Willis, The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations. Portland, OR, USA: IT Revolution Press, 2016.

[4] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Site Reliability Engineering: How Google Runs Production Systems. Sebastopol, CA, USA: O'Reilly Media, 2016.

[5] Docker Inc., "Docker Documentation," 2023. [En ligne]. Disponible sur : https://docs.docker.com. [Consulté le : 10 déc. 2025].

[6] Prometheus Authors, "Prometheus Documentation," 2025. [En ligne]. Disponible sur : https://prometheus.io/docs. [Consulté le : 10 déc. 2025].

[7] Grafana Labs, "Grafana Documentation," 2025. [En ligne]. Disponible sur : https://grafana.com/docs. [Consulté le : 10 déc. 2025].

[8] S. Newman, Building Microservices, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.

[9] L. Bass, P. Clements, and R. Kazman, Software Architecture in Practice, 4th ed. Boston, MA, USA: Addison-Wesley, 2021.

[10] K. Morris, Infrastructure as Code: Managing Servers in the Cloud. Sebastopol, CA, USA: O'Reilly Media, 2016.

# REFERENCES WEBOGRAPHIQUES

[11] JIRAMA, "Site officiel de la JIRAMA," 2025. [En ligne]. Disponible sur : https://www.jirama.mg. [Consulté le : 02 déc. 2025].

[12] Docker Inc., "Docker Documentation," 2025. [En ligne]. Disponible sur : https://docs.docker.com. [Consulté le : 29 Septembre 2025].

[13] GitHub Inc., "GitHub Actions Documentation," 2025. [En ligne]. Disponible sur : https://docs.github.com/en/actions. [Consulté le : 14 Octobre. 2025].

[14] Prometheus Authors, "Prometheus Documentation," 2025. [En ligne]. Disponible sur : https://prometheus.io/docs. [Consulté le : 03 décembre 2025].

[15] Grafana Labs, "Grafana Documentation," 2025. [En ligne]. Disponible sur : https://grafana.com/docs. [Consulté le : 03 décembre].

[16] Grafana Labs, "Loki Documentation," 2025. [En ligne]. Disponible sur : https://grafana.com/oss/loki. [Consulté le : 03 décembre 2025].

[17] NGINX Inc., "NGINX Documentation," 2025. [En ligne]. Disponible sur : https://nginx.org/en/docs. [Consulté le : 28 Novembre. 2025].

[18] DuckDNS, "Free Dynamic DNS Service," 2025. [En ligne]. Disponible sur : https://www.duckdns.org. [Consulté le : 28 Novembre 2025].

[19] Electronic Frontier Foundation, "Certbot - Automatically enable HTTPS," 2025. [En ligne]. Disponible sur : https://certbot.eff.org. [Consulté le : 02 déc. 2025].

[20] Aqua Security, "Trivy - Vulnerability Scanner," 2025. [En ligne]. Disponible sur : https://aquasecurity.github.io/trivy. [Consulté le : 15 Octobre 2025].

[21] Semgrep Inc., "Semgrep Documentation," 2025. [En ligne]. Disponible sur : https://semgrep.dev/docs. [Consulté le : 15 Octobre 2025].

[22] Astral, "Ruff - An extremely fast Python linter," 2025. [En ligne]. Disponible sur : https://docs.astral.sh/ruff. [Consulté le : 16 Octobre 2025].

[23] OpenJS Foundation, "ESLint Documentation," 2025. [En ligne]. Disponible sur : https://eslint.org/docs. [Consulté le : 16 Octobre 2025].

[24] Cloud Native Computing Foundation, "CNCF Cloud Native Interactive Landscape," 2025. [En ligne]. Disponible sur : https://landscape.cncf.io. [Consulté le : 01 décembre 2025].

# GLOSSAIRE

- **API**: Interface de programmation permettant à différentes applications de communiquer entre elles via des requêtes HTTP standardisées.
- **Artifact** : Fichier ou ensemble de fichiers générés par un processus de build (compilation, test, etc.) et conservés pour analyse ou déploiement ultérieur.
- **Backend** : Partie serveur d'une application qui gère la logique métier, les bases de données et les API, invisible pour l'utilisateur final.
- **Bind Mount** : Type de montage Docker qui lie directement un répertoire de l'hôte à un conteneur, permettant la synchronisation en temps réel des fichiers.
- **Blue/Green Deployment** : Stratégie de déploiement consistant à maintenir deux environnements identiques (blue et green), permettant de basculer instantanément entre versions et de revenir en arrière en cas de problème.
- **Build** : Processus de compilation et d'assemblage du code source en une application exécutable ou une image Docker.
- **CD (Continuous Deployment)** : Déploiement continu, pratique DevOps qui automatise la mise en production après validation des tests.
- **CI (Continuous Integration)** : Intégration continue, pratique consistant à intégrer fréquemment les modifications de code et à les tester automatiquement.
- **CI/CD** : Combinaison de l'intégration continue et du déploiement continu, automatisant l'ensemble du pipeline de livraison logicielle.
- **Conteneur** : Unité d'exécution isolée contenant une application et toutes ses dépendances, plus légère qu'une machine virtuelle.
- **Conteneurisation** : Méthode de virtualisation logicielle qui encapsule une application avec ses dépendances dans un conteneur portable et isolé.
- **Dashboard** : Tableau de bord visuel permettant de surveiller en temps réel les métriques, logs et états des services.
- **DevOps** : Culture et ensemble de pratiques visant à rapprocher les équipes de développement (Dev) et d'exploitation (Ops) pour accélérer la livraison logicielle.
- **DevSecOps** : Extension du DevOps intégrant la sécurité dès les premières phases du développement.

- **Docker Hub** : Registry public permettant de stocker, partager et distribuer des images Docker.
- **Dockerfile** : Fichier texte contenant les instructions pour construire une image Docker.
- **Entrypoint** : Script ou commande exécuté au démarrage d'un conteneur Docker, souvent utilisé pour initialiser l'environnement.
- **Exporter** : Service exposant les métriques d'un système dans un format compatible Prometheus (Node Exporter, MySQL Exporter, etc.).
- **Frontend** : Partie client d'une application web, visible et interactive pour l'utilisateur,
- **Hot Reload** : Fonctionnalité permettant de recharger automatiquement l'application lors de modifications du code, sans redémarrage complet.
- **HTTPS** : Protocole HTTP sécurisé utilisant TLS/SSL pour chiffrer les communications entre client et serveur.
- **htpasswd** : Outil permettant de créer des fichiers d'authentification HTTP Basic avec mots de passe chiffrés.
- **Job** : Ensemble d'étapes exécutées dans un workflow CI/CD, généralement en parallèle avec d'autres jobs.
- **Linting** : Analyse statique du code source pour détecter les erreurs, les incohérences de style et les mauvaises pratiques.
- **Métriques** : Données quantitatives mesurées en continu (CPU, RAM, requêtes/seconde) pour surveiller les performances d'un système.
- **Microservices** : Architecture applicative où l'application est décomposée en petits services indépendants communiquant via API.
- **Monitoring** : Surveillance continue des performances, de la disponibilité et des erreurs d'un système pour détecter rapidement les anomalies.
- **MySQL** : Système de gestion de base de données relationnelle open source.
- **Nginx** : Serveur web et reverse proxy haute performance, utilisé pour router le trafic et servir des contenus statiques.
- **Pipeline** : Séquence automatisée d'étapes (build, test, déploiement) dans un processus CI/CD.

- **Reverse Proxy** : Serveur intermédiaire qui reçoit les requêtes clients et les transmet aux services backend appropriés.
- **Pull Request** : Demande de fusion de modifications de code d'une branche vers une autre, permettant la revue de code.
- **Registry** : Serveur stockant et distribuant des images Docker (Docker Hub, GitLab Registry, etc.).
- **Rollback** : Opération consistant à revenir à une version antérieure stable après un déploiement défaillant.
- **Secrets** : Données sensibles (mots de passe, clés API, certificats) stockées de manière sécurisée et chiffrée.
- **SSH** : Protocole réseau sécurisé permettant une connexion chiffrée à distance à un serveur.
- **SSL/TLS** : Protocoles cryptographiques assurant la sécurité des communications sur Internet via chiffrement.
- **Staging** : Environnement de préproduction simulant la production pour tester les changements avant mise en ligne.
- **Tag** : Étiquette versionnée attachée à une image Docker (ex: v1.0, latest) pour identifier les différentes versions.
- **Volume Docker** : Mécanisme de persistance des données permettant de conserver les informations au-delà du cycle de vie d'un conteneur.
- **VPS** : Serveur virtuel dédié isolé sur une infrastructure physique partagée, offrant un contrôle complet.
- **Webhook** : Mécanisme de notification HTTP permettant à une application d'envoyer automatiquement des données à une URL lors d'un événement.
- **Workflow** : Fichier YAML définissant un pipeline automatisé dans GitHub Actions, déclenché par des événements Git.
- **YAML** : Format de sérialisation de données lisible par l'humain, utilisé pour les fichiers de configuration (Docker Compose, GitHub Actions).

# TABLE DES MATIERES

CURRICULUM VITAE.....I
REMERCIEMENTS.....III
SOMMAIRE.....IV
LISTE DES FIGURES.....VI
LISTE DES TABLEAUX.....VIII
LISTE DES ABREVIATIONS.....IX
INTRODUCTION GÉNÉRALE.....1
PARTIE I: PRESENTATION GENERALE.....2
chapitre 1: Présentation de l'Ecole Nationale d'Informatique.....3
1.1. Information d'ordre générale.....3
1.2. Missions et historiques.....3
1.3. Organigramme institutionnel.....5
1.4. Domaine de spécialisation.....6
1.5. Architecture des formations pédagogiques.....6
1.6. Relation de l'ENI avec les organismes externes.....7
1.7. Débouchés professionnels et diplômés.....8
1.8. Ressources humaines.....10
chapitre 2: Présentation de l'entreprise JIRAMA.....11
2.1. Présentation générale de la Société JIRAMA Madagascar.....11
2.2. Historique.....11
2.3. Indentité juridique.....12
2.4. Activité de la société.....13
2.4.1. Production d'électricité.....13
2.4.2. La production d'eau.....14
2.4.3. Le transport et la distribution.....14
2.5. Organigramme de la société.....15
chapitre 3: Description du projet.....16
3.1. Formulation.....16
3.2. Objectifs et besoins de l'entreprise.....16
3.3. Moyens nécessaires à la réalisation du projet.....16
3.3.1. Moyens matériels.....16
3.3.2. Moyens logiciels.....17
3.4. Résultats attendus.....17
3.5. Chronogramme des travaux.....18
PARTIE II: ANALYSE ET CONCEPTION.....19

chapitre 4: Analyse préalable....20
4.1. Analyse de l’existant....20
4.1.1. Topologie réseau....20
4.1.2. Connexion internet....20
4.1.3. Passerelle....21
4.1.4. Postes de travails....21
4.1.5. Analyse du trafic réseau....21
4.1.6. Surveillance....22
4.1.7. Test et sécurité....23
4.1.8. Gestion de code source....23
4.1.9. Déploiement et hébergement de l’ application....23
4.2. Critique de l’existant....24
4.2.1. Point fort....24
4.2.2. Point faible....24
4.3. Proposition de solution....25
4.4. Solution retenue....26
chapitre 5: Choix des outils....27
5.1. Conteneurisation....27
5.1.1. Comparaison des principaux types de conteneurs....27
5.1.2. Choix et justification....28
5.1.3. Présentation de Docker....28
5.2. systèmes de gestion de code source....29
5.2.1. Comparaison des systèmes de gestion de code source....29
5.2.2. Choix et justification....30
5.2.3. Présentation de Github....30
5.3. Système d’intégration et de déploiement continus(CI/CD)....31
5.3.1. Comparaison des outils CI/CD....31
5.3.2. Choix et justification....32
5.3.3. Présentation de GitHub Actions....32
5.4. Orchestration de conteneurs....33
5.4.1. Comparaison des principaux orchestrateurs de conteneurs....33
5.4.2. Choix et justification....34
5.4.3. Présentation de Docker compose....34
a. Fonctionnement....34
b. Avantages dans un contexte mono-hôte....35
5.5. Surveillance et monitoring....35
5.5.1. Comparaison des outils de supervision et d’observabilité....36

5.5.2. Choix et justification....36
5.5.3. Présentation de Prometheus....37
a. Fonctionnement....38
b. Caractéristiques Principales....38
5.5.4. Présentation de Grafana....38
a. Fonctionnement....38
b. Points forts....39
5.6. Serveur proxy....39
5.6.1. Tableau comparatif des solutions de proxy....39
5.6.2. Choix et justification....40
PARTIE 3: REALISATION....41
chapitre 6: Installation et configuration....42
6.1. Configuration de github....42
6.1.1. Organisation des environnements....42
6.1.2. Configuration du repository GitHub....42
a. Création du repository....42
b. Création des branches principales....43
6.1.3. Stratégie de fusion....43
6.2. Mise en place d'un environnement de développement conteneurisé et automatisé....43
6.2.1. Objectif....43
6.2.2. Architecture technique de l'environnement de développement....44
6.2.3. Conteneurisation des service....45
a. Backend Django....45
a.1. Script entrypoint.sh....46
b. Frontend React....47
c. Mise en place des fichiers .dockerignore : optimisation et sécurité....48
d. Base de données Mysql....49
6.2.4. Orchestration avec Docker Compose....49
6.2.5. Automatisation des commandes avec Makefile....52
6.2.6. Mise en place du workflow develop.yml pour la branche develop....53
a. Objectif du workflow....53
b. Structure générale du fichier develop.yml....53
c. Job 1 : Lint Python & React....53
d. JobTests Django & React....55
e. Job 3 : Analyse de sécurité (Semgrep + Trivy)....58
f. Rapport générés....60
6.3. Phase de staging : validation locale avant mise en production....60

6.3.1. Objectif.....60
6.3.2. Conteneurisation des services.....60
a. Conteneurisation du backend Django.....60
b. Conteneurisation du frontend React.....61
c. Conteneurisation de la Base de données.....62
6.3.3. Mise en place de docker-compose pour l'orchestration des services.....62
a. Mise en place d'une architecture multi-compose en staging.....62
b. Configuration de docker-compose.blue.yml.....63
c. Configuration docker-compose.green.yml.....64
d. Configuration docker-compose.db.yml.....65
e. Configuration docker-compose.edge.yml.....66
e.1. Configuration du fichier nginx.conf.....67
e.2. Configuration des upstream.....69
e.3. Sécurisation d'accès des services de monitoring.....70
f. docker-compose.monitoring.yml.....71
f.1. Fichier prometheus.yml.....73
f.2. Fichier promtail-config.yml.....74
f.3. Configuration de mysql-exporter.....74
g. Configuration des variable d'environnement.....75
h. Avantages de cette orchestration modulaire.....75
6.3.4. Mise en place du workflow GitHub Actions : staging.yml.....76
a. Objectif.....76
b. Les jobs du staging.yml.....76
c. Configuration des variables d'environnement.....78
6.4. Mise en place de l'environnement de production.....79
6.4.1. Introduction à la mise en production.....79
6.4.2. Choix et configuration du serveur VPS.....79
a. Choix du fournisseur.....79
b. Connexion SSH et sécurité.....80
b.1. Configuration ssh.....80
b.2. *Création et configuration de l'utilisateur dédié*.....80
c. Installation de Docker et Docker Compose..... 81
d. Configuration du pare-feu (UFW).....82
e. Configuration du répertoire de projet.....82
f. Gestion des secrets et fichiers sensibles.....82
f.1. Mise en place du fichier.env.global.....83
f.2. Mise en place du fichier.my-exporter.cnf.....83

f.3. Sécurisation de l’accès à prometheus et grafana avec .htpasswd.....84
g. Mise en place du certificat SSL avec DuckDNS et Certbot.....84
g.1. Configuration du domaine DuckDNS.....84
g.2. Installation de Certbot sur le serveur.....85
g.3. Obtention du certificat SSL.....85
g.4. Renouvellement automatique du certificat.....86
6.4.3. Mise en place de la conteneurisation.....86
a. Objectif de la conteneurisation en production.....86
b. Création manuelle des volumes externes.....86
c. Conteneurisation.....87
d. Mise en place des fichiers docker-compose pour l’orchestration.....87
d.1. Mise en place de docker-compose.blue.yml.....87
d.2. Mise en place de docker-compose.green.yml .....89
d.3. Mise en place de docker-compose.db.yml.....89
d.4. Mise en place du docker-compose.edge.yml.....91
6.4.4. Mise en place du CI/CD avec GitHub Actions : automatisation complète.....94
a. Configuration des secrets GitHub.....95
b. Mise en place du Ci Pipeline : qualité, sécurité, fiabilité.....95
c. Mise en place du CD Pipeline : déploiement blue/green.....97
c.1. Connexion SSH sécurisée.....97
c.2. Synchronisation des fichiers.....97
c.3. Mise en place du script de déploiement intelligent.....98
c.4. Nettoyage de l’ancienne version.....98
6.4.5. Mise en place du monitoring et de la supervision.....101
a.Architecture de supervision.....101
b. Configuration de Prometheus.....105
c. Configuration de Promtail.....107
d. Mise en place de Grafana.....108
d.1. Configuration des dashboards.....108
d.2. Configuration du contact point.....109
d.3. Création de l’application Slack.....110
d.4. Configuration du contact point dans Grafana.....112
d.5. Configuration des alertes.....113
chapitre 7: Test et validation.....115
7.1. Architecture finale du projet.....115
7.1.1. Description du fonctionnement de l’architecture.....116
7.1.2. Avantages de cette architecture.....118

7.2. Validation des workflows CI/CD....119
7.2.1. Accès à l'interface de l'application en développement et en staging....123
7.2.2. Vérification du déploiement sur le serveur....125
7.3. Tests de supervision et alerting....127
7.3.1. Vérification de la sécurité des accès....127
7.3.2. Validation de prometheus....129
7.2.3. Validation de grafana....129
a. Visualisation des dashboards....129
b. Validation des alertes mis en place....133
CONCLUSION....135
REFERENCES BIBLIOGRAPHIQUES....X
REFERENCES WEBOGRAPHIQUES....XI
GLOSSAIRE....XIII
TABLE DES MATIERES....XVI

# RESUME

Ce mémoire présente la mise en place d'une infrastructure DevOps complète pour le déploiement, la supervision et la sécurisation d'une application web de gestion de pannes. L'objectif principal est de professionnaliser le processus de livraison continue en automatisant les étapes de développement, de test, de déploiement et de supervision. L'application, développée avec ReactJS et Django, est conteneurisée à l'aide de Docker et orchestrée via Docker Compose sur un serveur VPS. L'automatisation est assurée par GitHub Actions, tandis que la supervision repose sur la stack Prometheus, Grafana, Loki et Promtail. Ce projet démontre qu'il est possible de mettre en œuvre une architecture DevOps robuste, évolutive et sécurisée, même dans un environnement à ressources limitées.

**Mots-clés :** DevOps, CI/CD, automatisation, retrogradation, supervision.

# ABSTRACT

This thesis presents the implementation of a complete DevOps infrastructure for deploying, monitoring, and securing a web-based incident management application. The main objective is to professionalize the continuous delivery process by automating the development, testing, deployment, and monitoring stages. The application, built with ReactJS and Django, is containerized using Docker and orchestrated with Docker Compose on a VPS. Automation is handled through GitHub Actions, while monitoring is ensured by the Prometheus, Grafana, Loki, and Promtail stack. This project demonstrates that a robust, scalable, and secure DevOps architecture can be implemented even in resource-constrained environments.

**Keywords:** DevOps, CI/CD, automation, rollback, monitoring.